

CURSO TÉCNICO EM INFORMÁTICA · EIXO: INFORMAÇÃO E COMUNICAÇÃO

PROGRAMAÇÃO PARA INTERNET I

Apostila completa da disciplina — teoria aula a aula, exercícios práticos, testes rápidos e avaliações (A1 · A2 · A3)

72 aulas · 24 semanas

3 aulas semanais (50 min)

60 h presenciais

10 h não presenciais (AVA)

12 módulos · 6 partes

3 avaliações · 100 pts

VERSÃO DO PROFESSOR · CONTÉM GABARITOS

Material elaborado a partir do Plano de Ensino e do Cronograma oficiais da disciplina · Setembro / 2026 · Docente responsável: _____

Apresentação da apostila

Esta apostila foi elaborada a partir do **Plano de Ensino e do Cronograma oficiais** da disciplina **Programação para Internet I** do Curso Técnico em Informática (Turma 2/2026). Ela reúne, em um único material, a **teoria** de cada aula, **exercícios práticos**, **testes rápidos** de verificação da aprendizagem e as **avaliações** do semestre (A1, A2 e A3), além do roteiro completo do **projeto final integrador**.

O material segue exatamente a sequência das **72 aulas em 24 semanas** (3 aulas semanais de 50 minutos), organizadas em **6 partes** e **12 módulos**. Cada módulo contém:

- **Objetivos de aprendizagem** — o que você deve saber fazer ao final do módulo;
- **Teoria aula a aula** — explicações, exemplos, tabelas e blocos de código comentados;
- **Exercícios práticos** — atividades de laboratório e questões escritas para fixação;
- **Teste rápido** — 5 questões objetivas para você conferir o que aprendeu;
- **Indicação de avaliação** — quando o módulo termina em avaliação ou entrega (A1, A2, A3 ou projeto final).

Como estudar com esta apostila (aula invertida)

Antes da aula, leia a teoria indicada e tente resolver os primeiros exercícios; em sala, participe das atividades práticas e tire dúvidas; depois da aula, refaça o teste rápido e registre suas dúvidas no AVA. Estudar programação é como aprender a andar de bicicleta: não basta ler — é preciso praticar digitando o código e observando o resultado.

DICA

Itens marcados com **(S)** são **propostas pedagógicas** de organização (datas de avaliações, entregas e distribuição das horas não presenciais) — não estão especificados no Plano de Ensino original. Demais informações reproduzem o Plano de Ensino oficial.

Ícones e caixas usadas neste material

Caixa	O que significa	Quando usar
Conceito-chave	Definição importante que provavelmente cairá nas avaliações.	Leia e memorize; anote no caderno.
Analogia	Comparação com situações do cotidiano para facilitar o entendimento.	Use para explicar o conceito com suas palavras.
Dica	Sugestão prática, atalho ou boa prática de quem programa.	Aplique nos exercícios.
Atenção	Erro comum ou ponto em que a maioria dos estudantes tropeça.	Leia com cuidado antes dos exercícios.
Código	Exemplo de código HTML/PHP comentado.	Digite no editor e execute — nunca apenas leia!
Exercícios	Atividades práticas e escritas do módulo.	Resolva em sala e complete em casa (AVA).
Teste rápido	5 questões objetivas de verificação.	Responda sem consultar; depois corrija.

DISCIPLINA · CARGA HORÁRIA · MÓDULOS · AVALIAÇÕES**Visão geral do curso**

Item	Informação
Disciplina	Programação para Internet I — Curso Técnico em Informática
Turma	Turma 2 / 2026 · Eixo Tecnológico: Informação e Comunicação
Carga horária presencial	60 h presenciais (aulas teóricas e práticas)
Carga horária não presencial	10 h não presenciais (atividades orientadas no AVA)
Estrutura	72 aulas · 24 semanas · 3 aulas semanais (períodos de 50 min)

EMENTA (SÍNTESE OFICIAL)

Desenvolvimento de projetos de website. Princípios de aplicações Web e hospedagem: portais, e-business, e-commerce, provedores, registro de domínio e acesso gratuito. Softwares e ferramentas adotados no desenvolvimento de aplicações web. Fundamentos de HTML e principais componentes de um documento HTML (BODY, HEAD, cabeçalhos, separadores). Formatação de textos, blocos e parágrafos. Listas ordenadas e numeradas. Tabelas. Linguagem PHP: configuração e uso. Utilização de ferramentas de desenvolvimento de soluções Web.

Módulos do semestre (distribuição oficial dos conteúdos)

#	Módulo	Conteúdo principal	Aulas	Semanas
1	Introdução à Web	Internet, Web, sites, aplicações Web, cliente/servidor, navegador, servidor, domínio, hospedagem	1-4	1-2
2	Ferramentas de desenvolvimento	Editor de código, navegador, estrutura de projetos, arquivos e pastas	5	2
3	HTML – fundamentos	Estrutura do documento, html, head, body, títulos, parágrafos, quebras, comentários	6-9	2-3
4	HTML – conteúdo	Textos, formatação, links, imagens, listas e organização da informação	10-12	4
5	HTML – tabelas e formulários	Tabelas, campos, input, select, textarea, button, formulários	13-18	5-6
6	Introdução ao PHP	O que é PHP, servidor, instalação/configuração, primeiro programa	19-21	7
7	PHP básico	Variáveis, constantes, tipos de dados, operadores e expressões	22-26	8-9
8	PHP – lógica	if, else, elseif, switch, operadores lógicos	27-29	9-10
9	PHP – repetição	for, while, do while, foreach (com arrays)	30-35	10-12

#	Módulo	Conteúdo principal	Aulas	Semanas
10	PHP + HTML	Mistura de HTML e PHP, geração dinâmica de páginas; organização, include e funções	36 e 43–48	12 e 15–16
11	Formulários + PHP	Recebimento e processamento de dados enviados pelo usuário	37–42	13–14
12	Projeto final	Desenvolvimento, testes, correções, organização e apresentação	49–72	17–24

Metodologia (estratégias de ensino oficiais)

- Exposição dialogada de conceitos, exemplos e atividades em sala.
- Aula invertida: estudo prévio em casa (bibliografia indicada) e aplicação em sala.
- Exercícios individuais e em grupo em cada tópico, com aplicação prática.
- Estudos de caso para análise crítica.
- Atividades extraclasse orientadas pelo Portal Acadêmico (AVA) — 10 h não presenciais.

Recursos didáticos

- Laboratório de informática e aulas presenciais.
- Recursos audiovisuais (projektor) e quadro branco.
- Portal Acadêmico (AVA): tarefas, fóruns e questionários.
- Google Drive, formulários e e-mail.
- Biblioteca virtual e WhatsApp (comunicação).

Atividades extraclasse orientadas — 10 h (proposta — S)

Fase	Semanas	Horas	Atividades previstas
Fase 1 — Web e HTML	Semanas 1 a 6	2,5 h	Quizzes e leituras no AVA, tarefa “Meu curso” (aula 6), continuidade do projeto “Meu Primeiro Site” e finalização do site da empresa fictícia.
Fase 2 — PHP	Semanas 7 a 12	2,5 h	Aula invertida (vídeos/leituras + questionários), listas de exercícios de lógica e repetição, preparação da revisão cumulativa.
Fase 3 — Formulários + PHP e organização	Semanas 13 a 16	2,0 h	Exercícios de GET/POST/validação, leitura sobre funções e include, estudo dirigido para a 2ª avaliação.
Fase 4 — Projeto final	Semanas 17 a 24	3,0 h	Desenvolvimento do projeto fora da aula, elaboração das entregas parciais, exercícios cumulativos de revisão e preparação da apresentação.

Avaliações — 100 pontos no semestre

Avaliação	Pontos	Período	Composição (proposta — S)
1ª Avaliação	30 pts	Semanas 1 a 8	Trabalho prático de HTML “site da empresa fictícia” (10 pts — entrega na semana 6) + atividades extraclasse/AVA e exercícios (5 pts — semanas 1 a 8) + teste escrito-prático sobre Web e HTML (15 pts — semana 8).

Avaliação	Pontos	Período	Composição (proposta — S)
2ª Avaliação	30 pts	Semana 16 (aula 48)	Prova (obrigatoriamente; pode ser em duplas ou com consulta) contemplando HTML (estrutura a formulários), PHP (variáveis a arrays/funções) e GET/POST/validação.
3ª Avaliação	40 pts	Semana 24	Prova escrita final individual e cumulativa (30 pts — aulas 70–71) + projeto integrador: entregas parciais e apresentação final (10 pts — aula 72).

Sumário

Apresentação da apostila	2
Visão geral do curso	4
PARTE I — Fundamentos da Internet e da Web	8
Módulo 1 — Introdução à Web (Aulas 1 a 4)	10
PARTE II — Ferramentas para Desenvolvimento Web (HTML)	18
Módulo 2 — Ferramentas de desenvolvimento (Aula 5)	20
Módulo 3 — HTML – fundamentos (Aulas 6 a 9)	25
Módulo 4 — HTML – conteúdo (Aulas 10 a 12)	34
Módulo 5 — HTML – tabelas e formulários (Aulas 13 a 18)	42
PARTE III — Introdução ao PHP	52
Módulo 6 — Introdução ao PHP (Aulas 19 a 21)	54
Módulo 7 — PHP básico (Aulas 22 a 26)	61
Módulo 8 — PHP – lógica (decisões) (Aulas 27 a 29)	70
Módulo 9 — PHP – repetição e arrays (Aulas 30 a 35)	78
PARTE IV — Formulários + PHP	87
Módulo 11 — Formulários + PHP (Aulas 37 a 42)	89
PARTE V — Organização e Desenvolvimento do Projeto	100
Módulo 10 — PHP + HTML — organização, include e funções (Aulas 36 e 43 a 48)	102
PARTE VI — Desenvolvimento do Projeto Final	115
Módulo 12 — Projeto final integrador (Aulas 49 a 72)	117
Projeto Final Integrador	130
Avaliações — 100 pontos no semestre	132
Glossário	145
Bibliografia	147

PARTE I

FUNDAMENTOS DA INTERNET E DA WEB

AULAS 1 A 4 · SEMANAS 1 E 2

Módulo 1 — Introdução à Web

MÓDULO 1 · AULAS 1 A 4 · SEMANAS 1 E 2

Introdução à Web

PARTE I — FUNDAMENTOS DA INTERNET E DA WEB

OBJETIVOS DE APRENDIZAGEM

- ✓ Explicar o que é a Internet e diferenciá-la da World Wide Web (Web).
- ✓ Descrever o modelo cliente/servidor e o papel do navegador.
- ✓ Entender o ciclo requisição > resposta e a diferença entre HTTP e HTTPS.
- ✓ Identificar as partes de uma URL (protocolo, domínio, caminho e recurso).
- ✓ Explicar os conceitos de endereço IP, DNS, domínio e hospedagem.
- ✓ Reconhecer portais, e-business e e-commerce e o fluxo de uma loja virtual.

AULA 1

O que é a Internet?

Redes de computadores

Uma **rede de computadores** é um conjunto de dispositivos (computadores, celulares, servidores) conectados entre si para **trocar informações**. Redes locais (como a do laboratório da escola) conectam dispositivos próximos; já a **Internet** é uma **rede mundial de redes**: bilhões de dispositivos interligados que se comunicam usando regras comuns, chamadas **protocolos**.

Internet × Web

É muito comum confundir **Internet** com **Web**, mas elas não são a mesma coisa. A Internet é a **infraestrutura** — as “estradas” por onde os dados trafegam. A Web (World Wide Web, ou WWW) é **um dos serviços** que funcionam sobre essas estradas: o conjunto de páginas e sites que acessamos pelo navegador.

Internet × Web

Internet = a rede física e lógica mundial que conecta dispositivos. **Web** = serviço de páginas interligadas (sites) que trafega na Internet. Outros serviços que também usam a Internet: e-mail, streaming de vídeo, jogos on-line, mensagens instantâneas e transferência de arquivos.

Cliente e servidor

Na Web, dois papéis se destacam: o **cliente**, que **pede** uma informação, e o **servidor**, que **fornece** essa informação. O computador ou celular que você usa é o cliente; o computador remoto que guarda o site é o servidor. O programa cliente mais conhecido é o **navegador** (browser), como Chrome, Firefox e Edge.

Cliente × Servidor

	Cliente	Servidor
Papel	Solicita recursos (páginas, imagens, dados)	Armazena e fornece recursos
Onde fica	No dispositivo do usuário	Em máquinas remotas, ligadas 24 h

	Cliente	Servidor
Exemplos	Navegador, celular, computador da escola	Servidor Web, hospedagem de sites
Quem programa	Front-end (HTML, CSS, JavaScript)	Back-end (PHP, banco de dados)

A Internet é como um restaurante

Você (o **cliente**) senta e faz o pedido ao garçom. O pedido chega à cozinha (o **servidor**), que prepara o prato e o envia de volta. Na Web funciona assim: o navegador faz uma **requisição** ("quero a página inicial"), o servidor Web processa o pedido e devolve uma **resposta** (o conteúdo da página). As **estradas** por onde o garçom passa são os cabos, fibras ópticas e sinais wi-fi da Internet.

AULA 2

Como a Web funciona?

Requisição e resposta

Tudo na Web começa com uma **requisição**: você digita um endereço (ou clica em um link) e o navegador **pede** aquele recurso a um servidor. O servidor **responde** enviando o conteúdo (código HTML, imagens, etc.). O navegador então "desenha" a página na tela. Esse ciclo se repete a cada clique.

1. Você digita o endereço do site no navegador;
2. O navegador descobre qual é o servidor responsável pelo endereço;
3. O navegador envia uma **requisição HTTP** ao servidor;
4. O servidor processa o pedido e devolve uma **resposta HTTP** (a página);
5. O navegador interpreta o HTML recebido e exibe a página na tela.

HTTP e HTTPS

HTTP (HyperText Transfer Protocol — Protocolo de Transferência de Hipertexto) é o protocolo de comunicação da Web: o conjunto de regras que define como requisições e respostas são feitas. O **HTTPS** é a versão **segura** do HTTP: os dados trafegam **criptografados**, protegendo senhas e informações pessoais. Sites com HTTPS mostram um **cadeado** na barra de endereços.

Por que HTTPS importa?

Em uma conexão HTTP "pura", qualquer pessoa no caminho poderia ler os dados trocados (senhas, cartões, mensagens). O HTTPS criptografa a comunicação entre navegador e servidor. Regra prática: nunca digite dados pessoais ou senhas em sites sem HTTPS (sem o cadeado).

Anatomia da URL

URL (Uniform Resource Locator) é o **endereço** de um recurso na Web. Cada parte tem uma função:

As partes de uma URL

TEXTO

`https://www.lojaexemplo.com.br/produtos/notebook.php?id=10`

protocolo domínio caminho recurso parâmetro

Componentes da URL

Parte	Exemplo	Para que serve
Protocolo	https://	Regra de comunicação usada (HTTP, HTTPS, FTP...)
Domínio	www.lojaexemplo.com.br	Nome do site — identifica o servidor
Caminho	/produtos/	Pasta (diretório) dentro do site
Recurso	notebook.php	Arquivo/página solicitada
Parâmetros	?id=10	Dados extras enviados à página (após o ?)

DICA

Para memorizar: o **protocolo** é “como” conversar; o **domínio** é “com quem” conversar; o **caminho + recurso** é “o que” você quer; os **parâmetros** são detalhes adicionais do pedido.

AULA 3

Domínio, hospedagem e servidor Web

Endereço IP

Todo dispositivo conectado a uma rede possui um **endereço IP** — um número que o identifica, como **142.250.78.14**. É o “endereço da casa” do dispositivo na rede: para enviar dados, é preciso saber o IP de destino.

DNS — a agenda da Internet

Decorar números IP seria inviável. Por isso existem os **domínios**: nomes amigáveis como **google.com**. O **DNS** (Domain Name System) é o sistema que **traduz o nome do domínio para o endereço IP** do servidor, funcionando como uma grande “agenda de telefones” da Internet.

DNS = agenda telefônica

Você procura “Pizzaria do João” na agenda (nome do domínio) para descobrir o número de telefone (endereço IP) e então fazer a ligação (requisição). O DNS faz exatamente isso, em milissegundos, cada vez que você digita um endereço.

Domínio: registro e escolha

O domínio é **registrado** (alugado por um período) em órgãos competentes: no Brasil, domínios **.com.br** são registrados no **registro.br**; domínios internacionais (.com, .net, .org) em registradores credenciados. Um domínio registrado é seu “endereço oficial” na Web — nenhum outro site pode usar o mesmo nome.

Hospedagem e servidor Web

Para que um site fique no ar, seus arquivos precisam ficar guardados em um computador ligado 24 horas por dia, com software preparado para atender requisições: o **servidor Web**. O serviço que aluga esse espaço é a **hospedagem**. Empresas que oferecem hospedagem e outros serviços de acesso são os **provedores**.

Tipos de hospedagem

Tipo	Como funciona	Indicado para
Gratuita	Espaço limitado, muitas vezes com anúncios e sem domínio próprio	Estudos e testes (como nosso curso!)
Compartilhada	Vários sites dividem o mesmo servidor	Sites pequenos e pessoais
VPS	Servidor virtual com recursos dedicados	Sites médios e aplicações
Dedicada	Um servidor físico inteiro para o cliente	Grandes empresas e alto tráfego
Nuvem (cloud)	Recursos distribuídos em vários servidores, cobrados sob demanda	Aplicações que precisam escalar

O caminho completo de um acesso

1. Você digita **www.meusite.com.br** no navegador;
2. O **DNS** consulta e descobre o **IP** do servidor do site;
3. O navegador envia a **requisição** ao servidor Web naquele IP;
4. O **servidor Web** localiza os arquivos do site na **hospedagem**;
5. A **resposta** (HTML, imagens...) volta ao navegador, que exibe a página.

AULA 4

Portais, e-business e e-commerce

Portais

Portal é um site que **concentra conteúdo e serviços** em um só lugar, funcionando como “porta de entrada” para um conjunto de informações. Exemplos comuns:

- **Portal institucional:** apresenta uma organização — empresa, prefeitura, escola (notícias, serviços, contatos);
- **Portal educacional:** ambiente de estudo — como o nosso **AVA/Portal Acadêmico**, com tarefas, fóruns e questionários;
- **Portal de notícias:** reúne matérias de vários assuntos (G1, UOL etc.).

e-Business — negócios digitais

e-Business (electronic business) é o uso da Internet para **conduzir os negócios de uma empresa** de forma digital: atendimento ao cliente, comunicação com fornecedores, gestão interna, marketing digital. É um conceito **amplo** — abrange toda a operação digital da empresa, não apenas a venda.

e-Commerce — comércio eletrônico

e-Commerce é a parte do e-business que trata da **compra e venda on-line**: a **loja virtual**. O cliente navega pelo catálogo, coloca produtos no carrinho e finaliza o pedido com pagamento eletrônico.

e-Business x e-Commerce

Todo e-commerce faz parte de um e-business, mas e-business é mais amplo: inclui também processos internos, relacionamento com fornecedores e atendimento. Resumindo: **e-business = negócio digital por completo**; **e-commerce = vendas on-line**.

Fluxo de uma loja virtual

1. **Início**: o cliente chega à página inicial da loja;
2. **Catálogo**: navega pelos produtos, busca e filtra;
3. **Ficha do produto**: vê fotos, descrição, preço e avaliações;
4. **Carrinho**: adiciona itens, ajusta quantidades;
5. **Checkout**: identifica-se, informa entrega e pagamento;
6. **Confirmação**: recebe o número do pedido e a confirmação.

DICA

Observe esse fluxo em lojas reais (Mercado Livre, Amazon, Magalu): ele é praticamente o mesmo em todas. Entender esse padrão ajuda a projetar qualquer site de vendas — inclusive o seu projeto final!

EXERCÍCIOS — MÓDULO 1

Atividades práticas e escritas · resolva no laboratório e complete em casa (AVA)

EXERCÍCIO 1 — Observação guiada de sites

PRÁTICO · LABORATÓRIO

Abra **3 sites diferentes** no navegador (ex.: um portal de notícias, o site de uma loja e o portal da escola). Para cada um, registre no caderno:

1. O endereço completo (URL) e as partes: protocolo, domínio, caminho e recurso;
2. Se a conexão é HTTPS (cadeado) — clique no cadeado e anote o que aparece;
3. Que tipo de site é: portal institucional, portal de notícias, e-commerce, outro;
4. Compare as anotações com um colega.

RESULTADO ESPERADO

Tabela com 3 linhas (uma por site) identificando URL correta, presença de HTTPS e classificação do tipo de site.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Verifique se distinguem domínio de caminho. Ótima oportunidade para corrigir o vocabulário ('site' x 'página' x 'URL').

EXERCÍCIO 2 — Análise de 5 URLs**ESCRITO**

Identifique **protocolo, domínio, caminho, recurso e parâmetros** (quando houver) nas URLs abaixo:

- 1) <https://www.empresa.com.br/index.html>
- 2) <http://portal.escola.edu.br/ava/login.php>
- 3) <https://loja.com.br/produtos/celular.php?id=42&cor=azul>
- 4) <https://noticias.uol.com.br/ultimas/>
- 5) <https://registro.br/tecnologia/dns/>

RESULTADO ESPERADO

1) [https | empresa.com.br | / | index.html](https://www.empresa.com.br/index.html) | — · 2) [http | portal.escola.edu.br | /ava/ | login.php](http://portal.escola.edu.br/ava/login.php) | — · 3) [https | loja.com.br | /produtos/ | celular.php | id=42 e cor=azul](https://loja.com.br/produtos/celular.php?id=42&cor=azul) · 4) [https | noticias.uol.com.br | /ultimas/](https://noticias.uol.com.br/ultimas/) | (pasta como recurso) | — · 5) [https | registro.br | /tecnologia/dns/](https://registro.br/tecnologia/dns/) | (pasta como recurso) | —

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Aceite variações razoáveis na separação caminho × recurso; o importante é identificarem domínio e protocolo corretamente e notarem os parâmetros após o '?'.
—

EXERCÍCIO 3 — Pesquisa em grupo: domínio × hospedagem × DNS**EM GRUPO**

Em grupos de 3 a 4 alunos, pesquisem e montem um resumo (cartaz ou 1 página) respondendo:

1. O que é e onde se registra um domínio .com.br? Quanto custa por ano (pesquisa no registro.br)?
2. O que é hospedagem? Citem 2 empresas que oferecem hospedagem gratuita e 2 pagas;
3. Como funciona o DNS? Expliquem com uma analogia criada pelo grupo;
4. Desenhem o fluxo completo: do navegador digitando a URL até a página aparecer na tela.

RESULTADO ESPERADO

Resumo com: registro.br (~R\$ 40/ano, valores podem variar), exemplos de hospedagens (gratuitas: InfinityFree, GitHub Pages; pagas: HostGator, Locaweb, Hostinger), analogia do DNS e desenho do fluxo em 5 etapas.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Valorize a analogia criada pelo grupo e a clareza do desenho do fluxo. Cada grupo apresenta em 2 minutos na aula seguinte.

EXERCÍCIO 4 — Estudo de caso: o fluxo da loja virtual**ESCRITO**

Escolha uma loja virtual conhecida e percorra uma compra (sem pagar!) até o carrinho. Escreva as etapas observadas (início > catálogo > ficha > carrinho > checkout) e responda: em quais etapas são usados formulários? Em qual etapa a loja precisa processar dados no servidor?

RESULTADO ESPERADO

Descrição das 5–6 etapas; identificação de formulários no login/cadastro, busca e checkout; percepção de que catálogo, carrinho e confirmação exigem processamento no servidor.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Conecte com o que virá no curso: formulários (módulo 5) e processamento com PHP (módulos 10 e 11).

TESTE RÁPIDO — MÓDULO 1: Introdução à Web

5 questões objetivas · responda sem consultar o material e depois corrija · gabarito comentado incluído

1. Qual afirmação descreve corretamente a relação entre Internet e Web?

- (A) São sinônimos perfeitos, apenas em idiomas diferentes.
- (B) A Web é a infraestrutura física; a Internet são os sites.
- (C) A Web é um dos serviços que funcionam sobre a Internet.
- (D) A Internet só existe para acessar a Web.

RESPOSTA: C

A Internet é a rede mundial (infraestrutura); a Web é um serviço que trafega nela, assim como e-mail, streaming e jogos.

2. No modelo cliente/servidor da Web, o navegador do usuário é:

- (A) o servidor, pois guarda as páginas.
- (B) o cliente, pois solicita os recursos ao servidor.
- (C) o DNS, pois traduz os endereços.
- (D) o protocolo, pois define as regras.

RESPOSTA: B

O navegador é o software cliente: ele faz a requisição e exibe a resposta enviada pelo servidor Web.

3. A principal função do DNS é:

- (A) criptografar os dados entre navegador e servidor.
- (B) hospedar os arquivos dos sites.
- (C) traduzir o nome do domínio para o endereço IP do servidor.
- (D) acelerar o processamento de páginas PHP.

RESPOSTA: C

O DNS é a 'agenda' da Internet: converte nomes (meusite.com.br) em números IP.

4. Um site com HTTPS, em comparação com HTTP, oferece:

- (A) mais cores e imagens na página.
- (B) comunicação criptografada entre navegador e servidor.
- (C) hospedagem gratuita garantida.
- (D) domínio registrado automaticamente no registro.br.

RESPOSTA: B

HTTPS = HTTP + criptografia. O cadeado na barra indica conexão segura para digitar senhas e dados pessoais.

5. Na URL <https://loja.com.br/produtos/fone.php?id=7>, o trecho [produtos/fone.php](#) representa:

- (A) o protocolo da comunicação.
- (B) o domínio registrado da loja.
- (C) os parâmetros enviados ao servidor.
- (D) o caminho e o recurso (página) solicitado.

RESPOSTA: D

Protocolo = https; domínio = loja.com.br; caminho/recurso = /produtos/fone.php; parâmetro = id=7.

PARTE II

FERRAMENTAS PARA DESENVOLVIMENTO WEB (HTML)

AULAS 5 A 18 · SEMANAS 2 A 6

Módulo 2 — Ferramentas de desenvolvimento

Módulo 3 — HTML – fundamentos

Módulo 4 — HTML – conteúdo

Módulo 5 — HTML – tabelas e formulários

MÓDULO 2 · AULA 5 · SEMANA 2

Ferramentas de desenvolvimento

PARTE II — FERRAMENTAS PARA DESENVOLVIMENTO WEB (HTML)

OBJETIVOS DE APRENDIZAGEM

- ✓ Conhecer as ferramentas básicas do desenvolvedor Web: editor de código, navegador e terminal.
- ✓ Instalar e configurar o ambiente de desenvolvimento (sugestão: VS Code).
- ✓ Criar e organizar a estrutura de pastas de um projeto Web.
- ✓ Entender a importância das extensões de arquivo e do texto puro.

AULA 5**Ambiente de desenvolvimento****As 4 ferramentas do desenvolvedor Web**

Para que serve cada ferramenta

Ferramenta	Papel no desenvolvimento	Exemplos
Editor de código	Escrever e editar os arquivos do site (HTML, PHP, CSS)	VS Code (sugestão do curso), Notepad++, Sublime Text
Navegador	Testar e visualizar o resultado; inspecionar com as DevTools (F12)	Chrome, Firefox, Edge
Gerenciador de arquivos	Criar e organizar pastas e arquivos do projeto	Explorador de Arquivos (Windows), Files (Linux)
Terminal	Executar comandos de texto: navegar em pastas, criar arquivos	Prompt de Comando, PowerShell, bash

Editor de código × processador de texto

Sites são escritos em **texto puro**. Um processador de texto (como o Word) adiciona formatação invisível que **quebra o código**. Por isso usamos um **editor de código**, que salva arquivos limpos e ainda ajuda com **cores de sintaxe**, **autocompletar** e **extensões**.

ATENÇÃO

Nunca escreva HTML/PHP no Word! Use sempre um editor de código. E atenção às extensões: o Windows pode escondê-las — um arquivo **index.html** deve terminar em **.html**, e não em **index.html.txt**.

Estrutura de um projeto Web

Todo projeto deve ter sua **própria pasta**, com os arquivos organizados por tipo. Essa organização facilita encontrar arquivos, reutilizar código e hospedar o site:

Estrutura inicial do projeto (crie no laboratório)

TEXT0

```
meu-site/  
├── index.html ← página inicial (sempre 'index!')  
├── sobre.html  
├── contato.html  
├── imagens/ ← fotos, logos, ícones  
│   └── foto.jpg  
└── css/ ← folhas de estilo (mais adiante)
```

Por que a página inicial se chama index?

O servidor Web procura automaticamente um arquivo chamado **index** (index.html ou index.php) quando alguém acessa o site sem informar a página. Por isso a página inicial **sempre** se chama index.

Terminal: comandos essenciais

Comandos básicos (Windows / Linux)

SHELL

```
cd documentos # entra na pasta 'documentos' (change directory)  
cd .. # volta para a pasta anterior  
dir # lista arquivos e pastas (Linux: ls)  
mkdir meu-site # cria a pasta 'meu-site'  
cd meu-site # entra na pasta criada
```

DICA

No VS Code, use **Arquivo > Abrir Pasta** e selecione **meu-site/**: o editor mostra toda a estrutura do projeto na barra lateral, e o terminal integrado (Ctrl+') já abre dentro da pasta certa.

EXERCÍCIOS — MÓDULO 2

Atividades práticas e escritas · resolva no laboratório e complete em casa (AVA)

EXERCÍCIO 1 — Montando o ambiente

PRÁTICO · LABORATÓRIO

Prepare seu ambiente de desenvolvimento completo:

1. Instale (ou abra) o VS Code e explore: barra lateral, área de edição e terminal integrado;
2. No terminal, crie a pasta do curso: **mkdir programacao-web** e entre nela;
3. Dentro dela, crie a pasta do projeto: **mkdir meu-site** e entre nela;
4. Crie a pasta **imagens/** e um arquivo de teste **teste.txt**;
5. Abra a pasta **meu-site** no VS Code (Arquivo > Abrir Pasta) e confira a estrutura na lateral.

RESULTADO ESPERADO

Estrutura: programacao-web/meu-site/ contendo imagens/ e teste.txt, aberta no VS Code.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Circule pela sala conferindo a estrutura de cada aluno. Corrija erros comuns: pasta criada no lugar errado, extensão .txt oculta, projeto aberto como arquivo e não como pasta.

EXERCÍCIO 2 — Explorando o navegador como ferramenta**PRÁTICO · LABORATÓRIO**

Abra qualquer site no navegador, pressione **F12** (DevTools) e explore:

1. Na aba **Elements/Elementos**, encontre o título da página (tag <title>) e um texto qualquer;
2. Na aba **Network/Rede**, recarregue a página e observe as requisições sendo feitas;
3. Anote: quantas requisições a página fez? Qual foi a primeira?

RESULTADO ESPERADO

Aluno localiza o HTML real da página na aba Elementos e observa dezenas de requisições (HTML, imagens, CSS, JS) na aba Rede — a primeira normalmente é o documento HTML.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Este exercício ‘abre a cabeça’ do aluno: o que aparece na tela é resultado de código. Guarde isso para o módulo de HTML.

EXERCÍCIO 3 — Desafio — mapa do meu projeto**ESCRITO**

Desenhe no caderno a estrutura de pastas que você usará no projeto “Meu Primeiro Site” (módulo 4): página inicial, página sobre, página de contato e pasta de imagens. Escreva o nome exato de cada arquivo.

RESULTADO ESPERADO

Árvore com meu-site/ contendo index.html, sobre.html, contato.html e imagens/.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Confira os nomes: index.html (não ‘inicio.html’ ou ‘Home.html’). Nomes sem acento, sem espaços e em minúsculas — explique o porquê (servidores e URLs).

TESTE RÁPIDO — MÓDULO 2: Ferramentas de desenvolvimento

5 questões objetivas · responda sem consultar o material e depois corrija · gabarito comentado incluído

1. Qual ferramenta é a mais adequada para escrever os arquivos HTML e PHP do projeto?

- (A) Microsoft Word, por causa do corretor ortográfico.
- (B) Um editor de código, como o VS Code.
- (C) O Bloco de Notas do celular.
- (D) O navegador Chrome.

RESPOSTA: B

Código é texto puro: editores de código oferecem cores de sintaxe e autocompletar sem inserir formatação invisível (como faz o Word).

2. A página inicial de um site deve se chamar index.html (ou index.php) porque:

- (A) é uma regra gramatical da linguagem HTML.
- (B) index significa 'índice' em latim e soa profissional.
- (C) o servidor Web procura automaticamente esse arquivo ao acessar o site.
- (D) o navegador não abre arquivos com outros nomes.

RESPOSTA: C

Ao receber um acesso sem página especificada, o servidor busca index.html/index.php na pasta do site.

3. No terminal, o comando mkdir imagens serve para:

- (A) abrir a pasta imagens no navegador.
- (B) criar uma pasta chamada imagens.
- (C) apagar a pasta imagens.
- (D) listar os arquivos da pasta atual.

RESPOSTA: B

mkdir = make directory (criar diretório/pasta). Listar arquivos é dir (Windows) ou ls (Linux); navegar é cd.

4. Por que não se deve usar nomes de arquivo com acentos ou espaços (ex.: "Página Inicial.html")?

- (A) Porque o VS Code não salva arquivos assim.
- (B) Porque ocupam mais espaço em disco.
- (C) Porque URLs e servidores tratam acentos/espacos de forma inconsistente, podendo quebrar os links.
- (D) Porque o HTML proíbe letras maiúsculas.

RESPOSTA: C

Boa prática: nomes curtos, minúsculos, sem acento, separados por hífen ou sublinhado (ex.: pagina-inicial.html).

5. As DevTools do navegador (tecla F12) permitem, entre outras coisas:

- (A) editar o código HTML exibido da página e ver as requisições de rede.
- (B) criar novas pastas no projeto automaticamente.
- (C) registrar domínios gratuitamente.
- (D) traduzir o site para outro idioma.

RESPOSTA: A

As DevTools mostram o HTML recebido, permitem inspecionar elementos, ver a rede, o console e muito mais — são a principal ferramenta de teste do desenvolvedor.

MÓDULO 3 · AULAS 6 A 9 · SEMANAS 2 E 3

HTML – fundamentos

PARTE II — FERRAMENTAS PARA DESENVOLVIMENTO WEB (HTML)

OBJETIVOS DE APRENDIZAGEM

- ✓ Criar um documento HTML5 completo com DOCTYPE, head e body.
- ✓ Identificar elementos, tags, atributos e elementos sem fechamento.
- ✓ Usar cabeçalhos (h1–h6), parágrafos (p), quebras (br) e separadores (hr).
- ✓ Aplicar formatação semântica de textos (strong, em, mark, small).
- ✓ Diferenciar semântica de aparência (strong × b, em × i).

AULA 6**Primeiro documento HTML****O que é HTML?**

HTML (*HyperText Markup Language* — Linguagem de Marcação de Hipertexto) é a linguagem usada para **estruturar o conteúdo** de toda página Web: textos, imagens, links, tabelas e formulários. HTML **não é linguagem de programação**: é uma linguagem de **marcação** — ela apenas “marca” o conteúdo, indicando o que cada trecho é (um título, um parágrafo, uma imagem...).

Marcação × Programação

Uma **linguagem de marcação** (HTML) descreve a estrutura do conteúdo, sem tomar decisões. Uma **linguagem de programação** (como PHP, que veremos na Parte III) executa lógica: faz cálculos, toma decisões e repete ações. Um site completo usa as duas juntas.

A estrutura básica do HTML5

Todo documento HTML5 segue a mesma “esqueleto”. Digite o exemplo no VS Code, salve como **index.html** dentro de **meu-site/** e abra no navegador (clique duplo ou arraste para a janela do navegador):

ola.html – o clássico “Olá, mundo!”

HTML

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
<meta charset="UTF-8">
<title>Minha primeira página</title>
</head>
<body>
<h1>Olá, mundo!</h1>
<p>Esta é a minha primeira página Web.</p>
</body>
</html>
```

Para que serve cada parte

Parte	Função
<!DOCTYPE html>	Declara que o documento usa HTML5 — deve ser a 1ª linha
<html lang="pt-br">	Elemento raiz; lang informa o idioma da página
<head>	Cabeça: configurações e metadados (não aparecem na página)
<meta charset="UTF-8">	Define a codificação — permite acentos (ç, ã, é...)
<title>	Título que aparece na aba do navegador e nos favoritos
<body>	Corpo: todo o conteúdo visível da página

head x body

O **head** guarda informações **sobre** a página (título da aba, codificação, links para CSS — veremos depois). O **body** guarda o conteúdo **da** página (tudo o que o usuário vê). Analogia: o head é a ficha catalográfica de um livro; o body são as páginas com a história.

DICA

No VS Code, crie um arquivo **.html** vazio, digite **!** e pressione **Tab**: o editor gera o esqueleto completo automaticamente (recurso Emmet). Depois digite **Ctrl+Espaço** para ver sugestões de tags.

Desafio “Sobre mim”

Como atividade da aula, crie a página **sobre.html** com um título **<h1>** com seu nome e um parágrafo contando: seu nome completo, idade, cidade onde mora e por que escolheu o curso de Informática. Salve e abra no navegador para conferir.

AULA 7

Estrutura do HTML: elementos, tags e atributos

Anatomia de um elemento

A unidade básica do HTML é o **elemento**, formado por **tag de abertura** + **conteúdo** + **tag de fechamento**:

Anatomia de um elemento

HTML

```
<p>Este texto é o conteúdo do elemento.</p>
```

| | _____ conteúdo _____ |
| | tag de abertura | tag de fechamento (com /)
| o símbolo '<' inicia a tag

Atributos

Atributos são configurações extras escritas **na tag de abertura**, no formato **nome="valor"**. Eles dão informações adicionais sobre o elemento:

Exemplos de atributos

HTML

```
<html lang="pt-br"> <!-- atributo lang: idioma da página -->
<meta charset="UTF-8"> <!-- atributo charset: codificação -->
<a href="sobre.html"> <!-- atributo href: destino do link -->
 <!-- src: arquivo; alt: descrição -->
```

Elementos sem fechamento (vazios)

Alguns elementos **não têm conteúdo** entre tags e, por isso, **não usam tag de fechamento**. São chamados de **elementos vazios**:

- **
** — quebra de linha;
- **<hr>** — separador horizontal (linha);
- **** — imagem (o conteúdo está nos atributos src/alt);
- **<meta>** — metadados (charset etc.);
- **<input>** — campo de formulário (veremos no módulo 5).

Aninhamento correto

Elementos podem conter outros elementos (**aninhamento**), mas as tags devem fechar **na ordem inversa** em que abriram — como caixas dentro de caixas:

Aninhamento: certo × errado

HTML

```
<!-- CERTO: fecha na ordem inversa -->
<p>Texto com <strong>destaque</strong> interno.</p>

<!-- ERRADO: tags cruzadas -->
<p>Texto com <strong>destaque</p></strong>
```

Comentários

Comentários são anotações que **não aparecem** na página — servem para organizar e documentar o código:

Comentário em HTML

HTML

```
<!-- Isto é um comentário: o navegador ignora -->
<!-- Início do cabeçalho do site -->
```

ATENÇÃO

Sem **meta charset="UTF-8"**, os acentos podem aparecer quebrados (ex.: "AulaÃ§Ã£o"). Se isso acontecer, verifique a codificação do arquivo no VS Code (canto inferior direito > UTF-8).

AULA 8

Cabeçalhos e parágrafos

Cabeçalhos: h1 a h6

Os cabeçalhos (ou títulos) organizam a página em **níveis hierárquicos**: **h1** é o título principal; **h2** são as seções; **h3** subseções, e assim por diante até **h6**. Quanto menor o número, maior o destaque.

Hierarquia de títulos

HTML

```
<h1>Minha Cidade</h1> <!-- título principal (único!) -->
<h2>História</h2> <!-- seção -->
<h3> Fundação </h3> <!-- subseção -->
<h2>Pontos turísticos</h2> <!-- outra seção -->
<h2>Culinária</h2>
```

Hierarquia não é tamanho!

Os cabeçalhos definem a **estrutura lógica** do conteúdo (como capítulos e subcapítulos de um livro). Não use h1 só porque a letra é grande: use o nível certo para cada trecho. Boas práticas: **um único h1 por página** e não pular níveis (h1 > h3 sem h2).

Parágrafos, quebras e separadores

- **<p>** — parágrafo: agrupa um bloco de texto com espaçamento próprio;
- **
** — quebra de linha **dentro** do mesmo parágrafo (elemento vazio);
- **<hr>** — separador horizontal: marca uma mudança de assunto (elemento vazio).

pagina “Minha Cidade”

HTML

```
<h1>Minha Cidade</h1>
<p>Curitiba é a capital do Paraná.<br>
Fica na região Sul do Brasil.</p>
<hr>
<h2>História</h2>
<p>Fundada em 1693, cresceu ao redor da matriz.</p>
```

ATENÇÃO

O navegador **ignora** espaços e “Enter” extras no código HTML: dois parágrafos digitados sem <p> viram um texto só na tela. Para separar textos, use <p>; para quebra de linha simples, use
 — não “Enter” no código.

AULA 9

Formatação de textos: semântica × aparência

Tags de formatação semântica

Principais tags de formatação

Tag	Significado (semântica)	Aparência
	Importante, forte ênfase	negrito
	Ênfase na leitura	itálico
<mark>	Trecho destacado/marcado	fundo amarelo

Tag	Significado (semântica)	Aparência
<small>	Texto secundário, letras miúdas	menor
 / <ins>	Removido / inserido (edições)	tachado / sublinhado
<sub> / <sup>	Subscrito / sobrescrito	H ₂ O / 10 ²

Formatação na prática

HTML

```
<p>0 curso é <strong>gratuito</strong> e as vagas são
<em>limitadas</em>.</p>
<p>Não esqueça a <mark>data da prova</mark>:</p>
<p><small>Sujeito a alterações sem aviso.</small></p>
<p>A fórmula da água é H<sub>2</sub></sub>0 e 10<sup>2</sup></sup> = 100.</p>
```

Semântica × aparência: strong × b, em × i

As tags e <i> deixam o texto em negrito e itálico, respectivamente — **apenas aparência**, sem significado. Já e têm o mesmo visual, mas **transmitem significado**: dizem ao navegador, aos leitores de tela (acessibilidade) e aos buscadores (SEO) que aquele trecho é **importante** ou deve ser **ênfatisado na leitura**.

Por que preferir tags semânticas?

1) **Acessibilidade**: leitores de tela avisam pessoas cegas que o trecho é importante; 2) **SEO**: buscadores entendem melhor o conteúdo; 3) **Manutenção**: o significado fica no código, não na aparência. Regra do curso: use **strong** e **em**; evite **b** e **i**.

Aparência é maquiagem; semântica é identidade

Dizer “isto é importante” (strong) é diferente de apenas engrossar a voz (b). No primeiro caso, quem não pode ver a formatação (leitor de tela) ainda assim entende o recado.

EXERCÍCIOS — MÓDULO 3

Atividades práticas e escritas · resolva no laboratório e complete em casa (AVA)

EXERCÍCIO 1 — Página “Sobre mim”

PRÁTICO · LABORATÓRIO

Crie **sobre.html** na pasta meu-site/ com a estrutura HTML5 completa e:

1. Título da aba: “Sobre mim — [seu nome]”;
2. h1 com seu nome completo;
3. Um parágrafo sobre você (idade, cidade, hobbies);
4. Uma segunda seção com h2 “Meus objetivos no curso” e um parágrafo com strong e em;
5. Um hr separando as seções e um small no final: “Página criada na aula 6”.

RESULTADO ESPERADO

Documento HTML5 válido, charset UTF-8, hierarquia h1>h2 correta, formatação semântica aplicada e página abrindo sem erros no navegador.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Checklist de correção: DOCTYPE presente? lang='pt-br'? charset UTF-8? title na aba? um único h1? tags fechadas na ordem? Passe pela sala conferindo o código (não só a tela).

EXERCÍCIO 2 — Página “Minha cidade”**PRÁTICO · LABORATÓRIO**

Crie **cidade.html** apresentando sua cidade com, no mínimo: um h1, dois h2, quatro parágrafos, um br, um hr, um mark e um strong.

RESULTADO ESPERADO

Página com hierarquia correta de títulos e todos os elementos solicitados presentes e bem aplicados (ex.: mark destacando um dado curioso, strong em informação importante).

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Peça que troquem de máquina com o colega e verifiquem o checklist. Erro comum: usar vários h1 — corrija na hora.

EXERCÍCIO 3 — Semântica x aparência**ESCRITO**

Reescreva o trecho abaixo trocando as tags de aparência por tags semânticas e justifique cada troca:

Trecho para corrigir

HTML

```
<p>A prova será <b>na sexta-feira</b> e o conteúdo  
inclui <i>HTML básico</i>. Traga <b>documento</b>.  
<i>Informações sujeitas a alteração.</i></p>
```

RESULTADO ESPERADO

na sexta-feira › (informação importante); <i>HTML básico</i> › (ênfase); o último <i> › <small> (texto secundário) — aceita-se em com justificativa.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

O importante é a justificativa: semântica ajuda acessibilidade e SEO.

EXERCÍCIO 4 — Caça-erros**ESCRITO**

O código abaixo tem **5 erros**. Encontre todos e reescreva o código corrigido:

Código com erros

HTML

```
<html>
<head>
<title>Teste
</head>
<body>
<h1>Título</h2>
<p>Um parágrafo com <strong>destaque</p></strong>
<br></br>
<p>Fim da página</p>
</body>
```

RESULTADO ESPERADO

1) falta <!DOCTYPE html> (e lang no html); 2) falta charset UTF-8 (meta); 3) <title> sem fechamento; 4) <h1> fechado com </h2>; 5) strong/p cruzados +
</br> (br é vazio, não tem fechamento).

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Corrija no quadro com participação da turma. Ótimo exercício pré-prova (cai na A1).

TESTE RÁPIDO — MÓDULO 3: HTML – fundamentos

5 questões objetivas · responda sem consultar o material e depois corrija · gabarito comentado incluído

1. A declaração <!DOCTYPE html> serve para:

- (A) criar o título da página.
- (B) informar ao navegador que o documento usa HTML5.
- (C) definir a codificação UTF-8.
- (D) iniciar o corpo visível da página.

RESPOSTA: B

O DOCTYPE deve ser a primeira linha do arquivo e declara a versão do HTML (HTML5).

2. As informações de configuração da página (charset, title) ficam dentro do elemento:

- (A) <body>
- (B) <head>
- (C) <title>
- (D) <meta>

RESPOSTA: B

O head guarda metadados e configurações; o body guarda o conteúdo visível. title e meta são filhos do head.

3. Qual grupo contém apenas elementos vazios (sem tag de fechamento)?

- (A) p, h1, strong
- (B) br, hr, img
- (C) a, em, mark
- (D) title, meta, body

RESPOSTA: B

br, hr, img (e também meta, input) não envolvem conteúdo — não têm fechamento.

4. Sobre cabeçalhos, é correto afirmar:

- (A) podemos usar quantos h1 quisermos por página.
- (B) h6 é o título de maior destaque.
- (C) o ideal é um único h1 por página, representando o assunto principal.
- (D) cabeçalhos servem apenas para aumentar a fonte do texto.

RESPOSTA: C

h1 é o título principal (único por página); h2..h6 organizam seções em ordem hierárquica — não são apenas tamanhos de fonte.

5. A diferença entre e é que:

- (A) strong deixa itálico e b deixa negrito.
- (B) strong transmite importância (semântica); b apenas altera a aparência.
- (C) b é uma tag obsoleta que quebra a página.
- (D) não existe diferença: são sinônimos perfeitos.

RESPOSTA: B

Ambos exibem negrito, mas strong tem significado (importância) — usado por leitores de tela e buscadores.

MÓDULO 4 · AULAS 10 A 12 · SEMANA 4

HTML – conteúdo

PARTE II — FERRAMENTAS PARA DESENVOLVIMENTO WEB (HTML)

OBJETIVOS DE APRENDIZAGEM

- ✓ Criar links internos e externos com a tag `a` (`href`, `target`).
- ✓ Inserir imagens com caminhos relativos e texto alternativo (`alt`).
- ✓ Construir listas não ordenadas, ordenadas e de definição, inclusive aninhadas.
- ✓ Interligar as páginas de um pequeno site (navegação).

AULA 10

Links — conectando as páginas

A tag de link

O **hipertexto** é a essência da Web: textos que se conectam a outros por **links**. O elemento `<a>` (âncora) cria um link; o atributo **href** indica o **destino**; o conteúdo entre as tags é o texto clicável:

Tipos de link

HTML

```
<!-- Link interno: outra página do MESMO site (caminho relativo) -->
<a href="sobre.html">Sobre mim</a>

<!-- Link externo: site de OUTRO domínio (URL absoluta) -->
<a href="https://www.google.com">Google</a>

<!-- Abrir em nova aba -->
<a href="https://developer.mozilla.org" target="_blank">Documentação MDN</a>
```

Atributos do elemento `a`

Atributo	Função	Exemplo
<code>href</code>	Endereço de destino do link	<code>href="contato.html"</code>
<code>target</code>	Onde abrir (<code>_blank</code> = nova aba; padrão = mesma aba)	<code>target="_blank"</code>
<code>title</code>	Texto de dica ao passar o mouse	<code>title="Fale conosco"</code>

Projeto “Meu Primeiro Site”

A partir desta aula, construímos um site com **3 páginas interligadas** — `index.html` (inicial), `sobre.html` (sobre você) e `contato.html`. Cada página terá um **menu** com links para as outras duas:

Menu de navegação (presente em todas as páginas)

HTML

```
<nav>
<a href="index.html">Início</a> |
<a href="sobre.html">Sobre</a> |
<a href="contato.html">Contato</a>
</nav>
```

Link interno x link externo

Interno: aponta para arquivo do próprio site — usa **caminho relativo** (sobre.html, imagens/foto.jpg).

Externo: aponta para outro site — usa **URL absoluta** com protocolo e domínio (https://...). Confundir os dois é o erro nº 1 de iniciantes: link interno com https:// não funciona!

DICA

Teste **todos** os links após criá-los. Link quebrado (erro 404) quase sempre é nome de arquivo digitado errado: o HTML diferencia maiúsculas de minúsculas em muitos servidores — **Sobre.html** ≠ **sobre.html**.

AULA 11

Imagens

Inserindo imagens

O elemento vazio **** exibe imagens. O atributo **src** indica o **arquivo** da imagem; o atributo **alt** traz um **texto alternativo** que aparece se a imagem não carregar:

Imagem com caminho relativo

HTML

```
<!-- imagem dentro da pasta imagens/ do projeto -->


<!-- controlando o tamanho (em pixels) -->

```

Caminhos relativos

O caminho da imagem é **relativo à página atual**:

Sintaxe de caminhos

Caminho	Significado	Exemplo
arquivo.jpg	Mesma pasta da página	src="foto.jpg"
pasta/arquivo.jpg	Subpasta da pasta atual	src="imagens/foto.jpg"
../arquivo.jpg	Pasta anterior (subir um nível)	src="../index.html"
https://...	URL externa (evite depender de imagens de outros sites)	src="https://site.com/img.png"

Formatos de imagem

Quando usar cada formato

Formato	Extensão	Melhor para
JPEG	.jpg / .jpeg	Fotografias (muitas cores, arquivo leve — sem transparência)
PNG	.png	Imagens com fundo transparente, logos, capturas de tela
GIF	.gif	Animações simples
WebP	.webp	Versão moderna e leve de JPEG/PNG
SVG	.svg	Desenhos vetoriais (ícones) que não perdem qualidade ao ampliar

O atributo alt é obrigatório de verdade

O texto alternativo tem 3 funções: 1) **acessibilidade** — leitores de tela o descrevem para pessoas cegas; 2) aparece quando a imagem **não carrega**; 3) ajuda no **SEO**. Escreva descrições reais: alt="Cachorro labrador brincando na grama", e não alt="imagem1".

ATENÇÃO

Imagem quebrada (ícone de 'foto rasgada')? Checklist: 1) o arquivo existe na pasta certa? 2) o nome está idêntico (maiúsculas/minúsculas)? 3) o caminho relativo está certo a partir da página atual? Use o F12 > aba Rede para ver o erro 404.

AULA 12

Listas

Três tipos de lista

Tipos de lista em HTML

Tipo	Tags	Uso
Não ordenada	ul + li	Itens sem sequência (compras, menu, características)
Ordenada	ol + li	Itens com sequência (passo a passo, ranking)
De definição	dl + dt + dd	Termo + definição (glossário)

Listas na prática

HTML

```
<h2>Lista de compras (não ordenada)</h2>
<ul>
<li>Pão</li>
<li>Leite</li>
<li>Café</li>
</ul>

<h2>Receita — passos (ordenada)</h2>
<ol>
<li>Misture os ingredientes</li>
<li>Leve ao forno por 30 min</li>
<li>Deixe esfriar e sirva</li>
</ol>
```

Lista de definição

HTML

```
<dl>
<dt>HTML</dt>
<dd>Linguagem de marcação que estrutura páginas Web.</dd>
<dt>Navegador</dt>
<dd>Programa cliente que exibe páginas Web.</dd>
</dl>
```

Listas aninhadas

Listas podem conter outras listas **dentro dos itens** — útil para subcategorias e menus:

Lista aninhada

HTML

```
<ul>
<li>Hardware
<ul>
<li>Processador</li>
<li>Memória RAM</li>
</ul>
</li>
<li>Software
<ul>
<li>Sistema operacional</li>
<li>Navegador</li>
</ul>
</li>
</ul>
```

DICA

Listas não servem só para “listas de compras”: menus de navegação, sumários e até o carrinho de um e-commerce são semanticamente listas. Use **ol** quando a ordem importa e **ul** quando não importa.

EXERCÍCIOS — MÓDULO 4

Atividades práticas e escritas · resolva no laboratório e complete em casa (AVA)

EXERCÍCIO 1 — Projeto “Meu Primeiro Site”

PRÁTICO · LABORATÓRIO

Construa um site com 3 páginas interligadas (index.html, sobre.html, contato.html):

1. Todas as páginas com a estrutura HTML5 completa e o mesmo menu de navegação no topo;
2. **index**: h1 de boas-vindas, um parágrafo com strong/em e uma imagem (imagens/);
3. **sobre**: reaproveite/complete a página “Sobre mim” do módulo 3;
4. **contato**: seus dados em uma lista não ordenada (e-mail, cidade, redes) e um link externo para o site da escola com target="_blank";
5. Teste todos os links partindo de qualquer página.

RESULTADO ESPERADO

3 páginas navegando entre si sem links quebrados, imagem carregando por caminho relativo, listas corretas e formatação semântica.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Este projeto continua como atividade extraclasse (AVA, Fase 1). Checklist: menu igual nas 3 páginas? alt nas imagens? links testados? title diferente em cada página?

EXERCÍCIO 2 — Galeria de imagens

PRÁTICO · LABORATÓRIO

Dentro do Meu Primeiro Site, crie **galeria.html** com 3 imagens na pasta imagens/, cada uma com: caminho relativo correto, alt descritivo e largura ajustada. Adicione a página ao menu das demais.

RESULTADO ESPERADO

Galeria com 3 imagens visíveis, alts descritivos e navegação integrada ao site.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Provocar erros de propósito (mover uma imagem de pasta) para treinarem o diagnóstico de imagem quebrada com F12.

EXERCÍCIO 3 — Receita em lista ordenada

PRÁTICO · LABORATÓRIO

Crie **receita.html** com: h1 (nome do prato), uma ul com os ingredientes, uma ol com o modo de preparo e uma dl explicando 2 termos culinários. Link no menu.

RESULTADO ESPERADO

Uso correto dos 3 tipos de lista na mesma página, com hierarquia de títulos.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Verificar: ingredientes em ul (ordem não importa) e preparo em ol (ordem importa). Pergunte por que a escolha de cada tipo.

EXERCÍCIO 4 — Caminhos relativos**ESCRITO**

A página **pages/produtos.html** precisa exibir a imagem **imagens/foto.jpg** (ambas dentro da pasta do projeto). Qual caminho correto no src? E se a imagem estivesse em **pages/img/foto.jpg**?

RESULTADO ESPERADO

1º caso: src="../../imagens/foto.jpg" (sair de pages/ e entrar em imagens/). 2º caso: src="img/foto.jpg" (subpasta da pasta atual).

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Desenhe a árvore de pastas no quadro. O '../' é o ponto que mais gera erro — reforce com mais exemplos se necessário.

TESTE RÁPIDO — MÓDULO 4: HTML – conteúdo

5 questões objetivas · responda sem consultar o material e depois corrija · gabarito comentado incluído

1. Para criar um link que abre a página sobre.html do próprio site, usamos:

- (A) <link src="sobre.html">
- (B) Sobre
- (C) Sobre
- (D) <href a="sobre.html">Sobre</href>

RESPOSTA: B

Link interno usa a tag a com href e caminho relativo — sem https://.

2. O atributo target="_blank" faz o link:

- (A) abrir em uma nova aba/janela.
- (B) ficar em negrito.
- (C) abrir apenas se o site for HTTPS.
- (D) ser invisível na página.

RESPOSTA: A

_blank = nova aba. Sem target, o link abre na mesma aba (comportamento padrão).

3. Uma página em `pages/noticia.html` quer exibir `imagens/foto.jpg` (pasta `imagens/` na raiz do projeto). O `src` correto é:

- (A) `imagens/foto.jpg`
- (B) `pages/imagens/foto.jpg`
- (C) `../imagens/foto.jpg`
- (D) `/foto.jpg`

RESPOSTA: C

É preciso 'subir' um nível (`..`) a partir de `pages/` para alcançar a pasta `imagens/` da raiz.

4. A função do atributo `alt` em uma imagem é:

- (A) definir o tamanho da imagem na tela.
- (B) descrever a imagem para acessibilidade e para quando ela não carregar.
- (C) criar um link ao clicar na imagem.
- (D) indicar o formato do arquivo (`jpg`, `png`...).

RESPOSTA: B

`alt` = texto alternativo: lido por leitores de tela, exibido se a imagem falhar e usado por buscadores.

5. Para exibir os passos de uma receita, em que a ordem importa, o tipo de lista adequado é:

- (A) `ul` — não ordenada, com marcadores.
- (B) `ol` — ordenada, com numeração automática.
- (C) `dl` — lista de definição.
- (D) `table` — tabela de passos.

RESPOSTA: B

`ol` numera os itens automaticamente (1, 2, 3...) — ideal para sequências e rankings.

MÓDULO 5 · AULAS 13 A 18 · SEMANAS 5 E 6

HTML – tabelas e formulários

PARTE II — FERRAMENTAS PARA DESENVOLVIMENTO WEB (HTML)

OBJETIVOS DE APRENDIZAGEM

- ✓ Construir tabelas com `table`, `tr`, `th` e `td`.
- ✓ Organizar tabelas com `thead`, `tbody`, `tfoot` e mesclar células com `colspan`.
- ✓ Criar formulários com `form`, `label` e os principais tipos de `input`.
- ✓ Usar `select`, `textarea` e `button` em formulários completos.
- ✓ Desenvolver o projeto HTML de um site de empresa fictícia (entrega A1).

AULA 13**Tabelas****Estrutura básica**

Tabelas organizam dados em **linhas** e **colunas**. No HTML: **<table>** é a tabela; cada **<tr>** é uma linha (*table row*); cada **<th>** é uma célula de **cabeçalho** (negrito, centralizada); cada **<td>** é uma célula de **dados** (*table data*):

Primeira tabela

HTML

```
<table border="1">
<tr>
<th>Nome</th>
<th>Curso</th>
</tr>
<tr>
<td>Ana Silva</td>
<td>Informática</td>
</tr>
<tr>
<td>João Souza</td>
<td>Informática</td>
</tr>
</table>
```

Receita de tabela: tr sempre envolve th/td

Nunca coloque `<td>` solto dentro da `<table>`: toda célula (`th` ou `td`) precisa estar dentro de uma linha (`tr`). A ordem é sempre: `table > tr > th/td`. O atributo `border="1"` mostra as bordas (para estudo; em projetos reais, bordas vêm do CSS).

Tabela = planilha

Pense em uma planilha: cada **tr** é uma linha; cada **td** é uma célula; a primeira linha com **th** são os títulos das colunas (Nome, Curso, Nota).

AULA 14**Tabelas na prática: thead, tbody, tfoot e colspan****Seções da tabela**

Tabelas organizadas separam o cabeçalho (**<thead>**), o corpo (**<tbody>**) e o rodapé (**<tfoot>**). Isso dá significado à estrutura e permite, no futuro, estilizar cada parte:

Tabela de alunos com seções e colspan

HTML

```
<table border="1">
<thead>
<tr> <th>Nome</th> <th>Curso</th> <th>Nota</th> </tr>
</thead>
<tbody>
<tr> <td>Ana Silva</td> <td>Informática</td> <td>9,0</td> </tr>
<tr> <td>João Souza</td> <td>Informática</td> <td>7,5</td> </tr>
</tbody>
<tfoot>
<tr>
<td colspan="2">Média da turma</td>
<td>8,25</td>
</tr>
</tfoot>
</table>
```

colspan — mesclando colunas

O atributo **colspan="N"** faz uma célula **ocupar N colunas**. No exemplo acima, "Média da turma" ocupa 2 colunas (Nome + Curso), e a última célula traz a média. Existe também o **rowspan** (ocupar N linhas), usado com menos frequência.

ATENÇÃO

Ao usar colspan, **remova** as células que foram "engolidas" — senão a linha fica com mais células que as outras e a tabela desalinha. Conte as colunas: se a tabela tem 3 colunas, cada linha precisa somar 3 (células normais + colspans).

DICA

Tabelas servem para **dados tabulares** (notas, horários, produtos com preço). Não use tabelas para montar o layout da página — para isso existe CSS.

AULA 15**Formulários: form, label e input****Para que servem formulários**

Formulários são a forma de o usuário **enviar dados** para o site: cadastros, logins, buscas, contatos. São a ponte entre o visitante e o servidor — e a base de qualquer aplicação Web (inclusive do seu projeto

final).

Estrutura: form, label e input

Primeiro formulário

HTML

```
<form action="processar.php" method="post">
<label for="nome">Nome:</label>
<input type="text" id="nome" name="nome">

<label for="email">E-mail:</label>
<input type="email" id="email" name="email">

<button type="submit">Enviar</button>
</form>
```

Atributos principais

Atributo	Onde	Função
action	form	Arquivo que vai receber/processar os dados
method	form	Como enviar: get ou post (módulo 11)
for/id	label/input	Associa o rótulo ao campo (clicar no label foca o campo)
name	input	Nome do campo — é assim que o PHP o identifica!
type	input	Tipo do campo (tabela abaixo)
placeholder	input	Dica de preenchimento (texto cinza dentro do campo)
required	input	Torna o preenchimento obrigatório

Tipos de input

Principais tipos (type)

type	O que faz	Exemplo de uso
text	Texto simples de uma linha	Nome, cidade
email	E-mail (valida o formato ao enviar)	Cadastro, contato
password	Senha (oculta os caracteres)	Login
number	Números (com min/max/step)	Idade, quantidade
date	Seletor de data	Data de nascimento
checkbox	Caixa de marcação — opções independentes	Aceito os termos

type	O que faz	Exemplo de uso
radio	Botão de opção — escolha única no grupo (mesmo name)	Turno: manhã/tarde
submit	Botão de envio do formulário	Enviar cadastro

radio x checkbox

radio: o usuário escolhe **uma** opção do grupo — todos os radios do grupo devem ter o **mesmo name**.

checkbox: o usuário marca **quantos quiser** — cada opção pode ter name próprio. Ex.: turno de estudo = radio; hobbies = checkboxes.

ATENÇÃO

Sem o atributo **name**, o campo **não é enviado** ao servidor — o PHP não conseguirá ler o valor. id é para o label/CSS; **name é para o processamento**.

AULA 16**Formulários na prática: cadastro de aluno****Construindo um cadastro completo**

Vamos montar o **Cadastro de Aluno** com: nome, e-mail, telefone, data de nascimento, curso (radio) e senha. Repare na organização: cada campo tem seu label, id e name; os radios de curso compartilham o mesmo name:

cadastro.html – cadastro de aluno

HTML

```
<form action="processar.php" method="post">
<label for="nome">Nome completo:</label>
<input type="text" id="nome" name="nome" required>

<label for="email">E-mail:</label>
<input type="email" id="email" name="email" required>

<label for="fone">Telefone:</label>
<input type="text" id="fone" name="telefone" placeholder="(41) 99999-0000">

<label for="nasc">Data de nascimento:</label>
<input type="date" id="nasc" name="nascimento">

<p>Curso:</p>
<input type="radio" id="inf" name="curso" value="Informática">
<label for="inf">Informática</label>
<input type="radio" id="adm" name="curso" value="Administração">
<label for="adm">Administração</label>

<label for="senha">Senha:</label>
<input type="password" id="senha" name="senha" required>

<button type="submit">Cadastrar</button>
</form>
```

Boas práticas de formulários

- Todo campo com **label** associado (for/id);
- **required** nos campos essenciais;
- **placeholder** com exemplo de formato (não substitui o label!);
- Radios do mesmo grupo com o **mesmo name** e **value** em cada opção;
- Ordem lógica dos campos: identificação > contato > dados específicos > senha > botão.

DICA

O atributo **value** define o valor que será enviado ao servidor. Em radios e checkboxes ele é essencial: sem value, o servidor recebe apenas 'on'.

AULA 17

Select, textarea e buttons

Lista suspensa: select + option

select – escolha em lista suspensa

HTML

```
<label for="turno">Turno:</label>
<select id="turno" name="turno">
  <option value="" disabled selected>Selecione...</option>
  <option value="manha">Manhã</option>
  <option value="tarde">Tarde</option>
  <option value="noite">Noite</option>
</select>
```

A primeira option (disabled + selected) funciona como “placeholder” da lista: aparece selecionada, mas não pode ser escolhida de volta.

Texto longo: textarea

textarea – mensagem de várias linhas

HTML

```
<label for="msg">Mensagem:</label>
<textarea id="msg" name="mensagem" rows="4" cols="40"
  placeholder="Escreva sua mensagem..."></textarea>
```

Diferente do input, o **textarea** tem tag de abertura e fechamento; o texto inicial (se houver) fica entre elas. rows e cols definem o tamanho visível.

Botões

Botões do formulário

HTML

```
<button type="submit">Cadastrar</button> <!-- envia o formulário -->
<button type="reset">Limpar</button> <!-- apaga os campos -->
<input type="submit" value="Enviar"> <!-- alternativa ao button -->
```

select x radio — qual usar?

Situação	Melhor campo
2 a 4 opções, todas importantes (sim/não, turno)	radio — todas ficam visíveis
Muitas opções (cidades, cursos, estados)	select — economiza espaço
Várias escolhas independentes (hobbies)	checkbox
Texto livre de várias linhas (mensagem)	textarea

AULA 18

Projeto HTML: site da empresa fictícia (entrega A1)

O projeto

Hora de juntar **tudo** o que aprendemos de HTML em um projeto completo: o site de uma **empresa fictícia** com **4 páginas** (index, empresa, produtos e contato). Este é o **trabalho prático da Avaliação 1 (10 pts)** — entrega na semana 6 (S).

Requisitos do projeto

1. **4 páginas:** index.html (início), empresa.html (quem somos), produtos.html (catálogo) e contato.html (fale conosco);
2. **Menu de navegação** igual em todas as páginas (links testados!);
3. **Estrutura HTML5** completa em todas (DOCTYPE, lang, charset, title);
4. **index:** h1, textos com formatação semântica (strong/em), imagem com alt e uma lista;
5. **empresa:** história da empresa com h1/h2, parágrafos, br/hr e uma imagem;
6. **produtos: tabela** com pelo menos 4 produtos (nome, categoria, preço, estoque) usando thead/tbody/tfoot e colspan na linha de total/rodapé;
7. **contato: formulário completo** — nome, e-mail, assunto (select), mensagem (textarea), checkbox “quero receber novidades” e botão de envio;
8. Pasta **imagens/** organizada, nomes de arquivo sem acento/espço.

O que será avaliado (10 pts — S)

Estrutura e navegação (2 pts) · Elementos de conteúdo: títulos, textos, imagens e listas (2 pts) · Tabela completa e correta (2 pts) · Formulário completo e correto (2 pts) · Organização dos arquivos e qualidade do código (2 pts). Critérios detalhados na seção **Avaliação A1** desta apostila.

DICA

Planeje antes de codificar: desenhe o mapa do site e o esboço de cada página no caderno. Desenvolva uma página por vez e teste no navegador a cada etapa. Reutilize o menu: copie e cole entre as páginas, ajustando o destaque da página atual.

EXERCÍCIOS — MÓDULO 5

Atividades práticas e escritas · resolva no laboratório e complete em casa (AVA)

EXERCÍCIO 1 — Tabela de horários

PRÁTICO · LABORATÓRIO

Crie **horarios.html** com a tabela do seu horário escolar: 1ª linha com th (dias da semana) e as demais com as aulas (td). Use thead e tbody.

RESULTADO ESPERADO

Tabela alinhada com cabeçalho em th, dias como colunas e horários/aulas nas células.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Confira se th está apenas no cabeçalho. Desafio extra: colspan para 'intervalo' ocupando todos os dias.

EXERCÍCIO 2 — Tabela de alunos com tfoot e colspan

PRÁTICO · LABORATÓRIO

Construa a tabela de alunos (Nome, Curso, Nota) com 4 registros, usando thead, tbody e tfoot. No tfoot, uma célula com colspan="2" (ex.: “Média da turma”) e a média na última coluna.

RESULTADO ESPERADO

Tabela com 3 colunas; linha do tfoot com apenas 2 células (a primeira com colspan=2); média calculada corretamente.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Erro típico: manter 3 células no tfoot + colspan > linha desalinha. Faça-os contar as colunas.

EXERCÍCIO 3 — Cadastro de aluno completo**PRÁTICO · LABORATÓRIO**

Monte **cadastro.html** com: nome (text, required), e-mail (email, required), telefone (text, placeholder), data de nascimento (date), curso (radio: Informática, Administração, Redes), turno (select), observações (textarea), aceite dos termos (checkbox) e botão Cadastrar.

RESULTADO ESPERADO

Formulário completo com labels associados, nomes corretos, radios com mesmo name e values definidos, select com opção 'Selecione...', textarea e checkbox funcionais.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Testar envio (vai dar erro/abrir o processar.php inexistente — esperado; explique que o back-end vem no módulo 11). Confira name em TODOS os campos.

EXERCÍCIO 4 — Projeto A1 — site da empresa fictícia**EM GRUPO**

Em duplas (ou individual, a critério do professor), desenvolva o site completo da empresa fictícia conforme os requisitos da aula 18. **Trabalho avaliativo — 10 pts da A1 (S)**. Entrega: semana 6, via AVA/Drive + pasta do laboratório.

1. Planejar: nome da empresa, ramo, esboço das 4 páginas no caderno;
2. Criar a estrutura de pastas (empresa-x/, imagens/);
3. Desenvolver página por página, testando no navegador;
4. Revisar o checklist de requisitos antes de entregar.

RESULTADO ESPERADO

Site com 4 páginas navegáveis atendendo todos os requisitos da aula 18.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Usar o rubrica da seção A1 para a nota. Sugestão: aula de revisão com autoavaliação por checklist antes da entrega final.

TESTE RÁPIDO — MÓDULO 5: HTML – tabelas e formulários

5 questões objetivas · responda sem consultar o material e depois corrija · gabarito comentado incluído

1. Em uma tabela HTML, a diferença entre th e td é:

- (A) th é célula de dado; td é célula de cabeçalho.
- (B) th é célula de cabeçalho (destacada); td é célula de dado.
- (C) th só pode ser usado no tfoot.
- (D) não há diferença: são sinônimos.

RESPOSTA: B

th marca cabeçalhos de coluna/linha (negrito, centralizado por padrão); td marca dados.

2. O atributo colspan="3" em uma célula faz com que ela:

- (A) seja repetida em 3 linhas.
- (B) ocupe o espaço de 3 colunas.
- (C) tenha 3 pixels de borda.
- (D) exiba 3 textos diferentes.

RESPOSTA: B

colspan mescla colunas (a célula ocupa 3); rowspan mesclaria linhas.

3. Em um formulário, o atributo label for="nome" deve casar com qual atributo do input?

- (A) name="nome"
- (B) id="nome"
- (C) value="nome"
- (D) type="nome"

RESPOSTA: B

O for do label aponta para o id do campo. O name serve para o envio ao servidor — são funções diferentes.

4. Para o usuário escolher apenas uma opção entre “Manhã, Tarde ou Noite”, o campo adequado é:

- (A) 3 checkboxes com nomes diferentes.
- (B) 3 inputs type="radio" com o mesmo name.
- (C) 3 inputs type="text".
- (D) 1 textarea listando as opções.

RESPOSTA: B

Rádios com o MESMO name formam um grupo de escolha única. Checkboxes permitem múltiplas escolhas.

5. No form, os atributos action e method definem, respectivamente:

- (A) o estilo do formulário e a cor do botão.
- (B) o arquivo que processará os dados e o modo de envio (get/post).
- (C) o título da página e o idioma.
- (D) quais campos são obrigatórios e seus tipos.

RESPOSTA: B

action = destino do processamento (ex.: processar.php); method = forma de envio dos dados (GET ou POST — módulo 11).

AVALIAÇÃO À VISTA

AVALIAÇÃO 1 (A1 — 30 pts): trabalho “site da empresa fictícia” (10 pts) + atividades AVA (5 pts) + teste escrito-prático (15 pts). Enunciado completo e rubrica na seção AVALIAÇÕES.

PARTE III

INTRODUÇÃO AO PHP

AULAS 19 A 36 · SEMANAS 7 A 12

Módulo 6 — Introdução ao PHP

Módulo 7 — PHP básico

Módulo 8 — PHP – lógica (decisões)

Módulo 9 — PHP – repetição e arrays

Observação: a aula 36 (revisão cumulativa + projeto PHP), ministrada na semana 12, pertence ao Módulo 10 — seu conteúdo completo está na Parte V.

MÓDULO 6 · AULAS 19 A 21 · SEMANA 7

Introdução ao PHP

PARTE III — INTRODUÇÃO AO PHP

OBJETIVOS DE APRENDIZAGEM

- ✓ Explicar o que é PHP e diferenciá-lo do HTML (servidor × navegador).
- ✓ Configurar o ambiente com servidor local (XAMPP ou similar).
- ✓ Escrever e executar o primeiro programa PHP (echo, blocos <?php ?>).
- ✓ Misturar HTML e PHP no mesmo arquivo para gerar conteúdo dinâmico.

AULA 19

O que é PHP?

Uma linguagem do lado do servidor

PHP (PHP: Hypertext Preprocessor) é uma **linguagem de programação** criada para a Web. Diferente do HTML — que é interpretado no **navegador** —, o código PHP é executado no **servidor**: lá ele faz cálculos, toma decisões, lê formulários e **gera HTML**, que então é enviado pronto ao navegador.

HTML × PHP — quem faz o quê?

HTML: estrutura o conteúdo; processado/exibido no **navegador** (lado do cliente); o resultado é sempre o mesmo para todos. **PHP**: executa lógica; processado no **servidor** (lado do servidor); o resultado pode mudar a cada acesso (conteúdo **dinâmico**): hora atual, dados do usuário, registros de um banco...

O fluxo com PHP

TEXTO

```
NAVEGADOR (cliente)  SERVIDOR
|                     |
|                     |
| — requisicao: pagina.php → |
| 1. interpreta o PHP      |
| 2. executa a lógica      |
| 3. gera HTML             |
| ← resposta: HTML puro — |
|                     |
| exibe a página          |
```

Configuração: servidor local

Para executar PHP no computador do laboratório, instalamos um **pacote com servidor Web + PHP** (sugestão: **XAMPP**, gratuito). Após instalar:

1. Abra o painel do XAMPP e **inicie o Apache** (o servidor Web);
2. Coloque a pasta do projeto dentro de **htdocs/** (pasta que o Apache publica);
3. Acesse pelo navegador: **http://localhost/meu-site/arquivo.php**.

ATENÇÃO

Arquivo **.php** aberto com dois cliques (file://) **não executa** o PHP — o navegador mostra o código ou uma página em branco. PHP só roda passando pelo servidor: **http://localhost/...**

O que dá para fazer com PHP

- Gerar **conteúdo dinâmico** (data/hora, saudações personalizadas, listas de dados);
- **Receber e processar formulários** (cadastros, logins, buscas — módulo 11);
- Tomar **decisões** e **repetir** tarefas (módulos 8 e 9);
- Guardar dados em **bancos de dados** (Programação para Internet II);
- Grandes sites usam/ usaram PHP: Wikipédia, WordPress, Facebook (no início)...

AULA 20

Primeiro programa PHP

Blocos PHP, echo e ponto e vírgula

O código PHP vive dentro de **blocos** `<?php ... ?>`. O comando **echo** envia texto para a página. Toda instrução termina com **ponto e vírgula (;)** — esquecer-lo é o erro mais comum de iniciante!

ola.php – o primeiro programa

PHP

```
<?php
// comentário de uma linha (igual em muitas linguagens)
echo "Olá, mundo!";
echo "<br>"; // posso enviar HTML pelo echo
echo "Meu nome é PHP e rodo no servidor.";
?>
```

Salve como **ola.php** em **htdocs/meu-site/** e acesse **http://localhost/meu-site/ola.php**. Resultado na tela: "Olá, mundo! Meu nome é PHP e rodo no servidor."

Regras básicas da sintaxe

- Blocos: `<?php` abre e `?>` fecha (o fechamento é opcional em arquivos só com PHP);
- Instruções terminam com `;`;
- Texto (string) vai entre **aspas duplas** `"..."` ou simples `'...'`;
- Comentários: `//` (uma linha) ou `/* ... */` (várias linhas);
- PHP diferencia maiúsculas/minúsculas em **variáveis** (`$nome` ≠ `$Nome`), mas não em **palavras-chave** (`echo` = `ECHO`).

DICA

Para "ver" o que o servidor fez, use **Ctrl+U** (exibir código-fonte) no navegador: você verá apenas o **HTML gerado** — o código PHP nunca chega ao cliente. Isso prova que ele foi executado no servidor.

ATENÇÃO

Erros de sintaxe (como esquecer o ; ou fechar aspas) fazem o PHP exibir uma mensagem de erro com o **arquivo e a linha**. Leia a mensagem: ela diz exatamente onde está o problema — não tenha medo dela!

AULA 21**PHP + HTML no mesmo arquivo****Misturando as linguagens**

Um arquivo **.php** pode conter **HTML normalmente** + **blocos PHP** intercalados. O servidor executa os blocos PHP e entrega ao navegador um documento HTML único:

pagina.php – HTML + PHP juntos

PHP

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
<meta charset="UTF-8">
<title>Página dinâmica</title>
</head>
<body>
<h1>Bem-vindo ao meu site!</h1>

<?php
echo "<p>Hoje é dia " . date("d/m/Y") . "</p>";
echo "<p>Agora são " . date("H:i") . " horas.</p>";
?>

<p>Este parágrafo é HTML puro.</p>
</body>
</html>
```

Conteúdo dinâmico

A página acima muda **sozinha**: a data e a hora exibidas são as do momento do acesso — calculadas pelo servidor. Isso é **conteúdo dinâmico**: o mesmo arquivo gera resultados diferentes a cada execução. HTML puro nunca faria isso.

O operador de concatenação (.)

Em PHP, o **ponto (.)** junta (concatena) textos: **"Olá, " . \$nome**. No exemplo, juntamos "Hoje é dia " com o resultado de **date("d/m/Y")** e com ".". A função **date()** devolve a data formatada conforme o padrão informado (d = dia, m = mês, Y = ano com 4 dígitos, H = hora, i = minuto).

DICA

Também dá para escrever variáveis diretamente dentro de aspas duplas: **echo "Olá, \$nome!";** — veremos isso no módulo 7.

EXERCÍCIOS — MÓDULO 6

Atividades práticas e escritas · resolva no laboratório e complete em casa (AVA)

EXERCÍCIO 1 — Olá, mundo! em PHP

PRÁTICO · LABORATÓRIO

Com o Apache iniciado e o projeto em `htdocs/meu-site/`, crie **ola.php** que exiba: seu nome em um `h1`, um parágrafo dizendo por que você escolheu o curso e a data de hoje (`date`).

RESULTADO ESPERADO

Página acessível em `http://localhost/meu-site/ola.php` exibindo nome (`h1`), parágrafo e data atual calculada pelo servidor.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Verifique: bloco `<?php` correto, ponto e vírgula, concatenação com ponto. Faça-os dar `Ctrl+U` para confirmar que só há HTML na resposta.

EXERCÍCIO 2 — Cartão de visita dinâmico

PRÁTICO · LABORATÓRIO

Crie **cartao.php** com a estrutura HTML5 completa e, no body, um bloco PHP que gere dinamicamente: uma saudação conforme nada (apenas texto), a data e a hora completas e uma mensagem com strong.

RESULTADO ESPERADO

Documento .php misturando HTML e blocos PHP, exibindo data/hora do servidor e formatação HTML enviada via `echo`.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Reforce que `echo` pode conter tags HTML. Desafio: duas atualizações de página devem mostrar horários diferentes.

EXERCÍCIO 3 — O que o navegador recebe?

ESCRITO

Sem executar, escreva o HTML que o **navegador recebe** (saída) para o código abaixo:

`saida.php`

PHP

```
<?php
echo "<h2>Resultado</h2>";
echo "<p>" . (10 + 5) . "</p>";
echo "2" . "5";
?>
```

RESULTADO ESPERADO

`<h2>Resultado</h2><p>15</p>25` — na tela: título 'Resultado', parágrafo '15' e o texto '25' (concatenação de strings, não soma!).

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Ponto-chave: (10+5) é soma = 15; "2"."5" é concatenação = 25. Discuta a diferença antes de revelar.

TESTE RÁPIDO — MÓDULO 6: Introdução ao PHP

5 questões objetivas · responda sem consultar o material e depois corrija · gabarito comentado incluído

1. O código PHP é executado:

- (A) no navegador do usuário.
- (B) no servidor Web, antes de a página ser enviada.
- (C) no editor de código, ao salvar o arquivo.
- (D) no sistema DNS.

RESPOSTA: B

PHP é server-side: o servidor executa e envia ao navegador apenas o HTML resultante.

2. O comando usado para exibir conteúdo na página em PHP é:

- (A) print-screen
- (B) echo
- (C) display
- (D) show

RESPOSTA: B

echo (e seu primo print) envia conteúdo para a resposta da página.

3. Um arquivo com PHP deve ter qual extensão e como deve ser acessado?

- (A) .html, com dois cliques no arquivo.
- (B) .php, pelo endereço `http://localhost/...`
- (C) .php, com dois cliques no arquivo.
- (D) .txt, pelo bloco de notas.

RESPOSTA: B

Extensão .php e acesso via servidor (localhost). Com dois cliques (`file:///`), o PHP não é executado.

4. O que o comando `echo "3" . "4";` exibe?

- (A) 7
- (B) 34
- (C) 3.4
- (D) erro de sintaxe

RESPOSTA: B

O ponto concatena strings: `"3"."4" = "34"`. Soma seria com o operador + entre números: `3+4 = 7`.

5. Uma página mostra a data de hoje diferente a cada acesso, mesmo sendo o mesmo arquivo. Isso acontece porque:

- (A) o navegador guarda datas aleatórias.
- (B) o HTML foi recompilado.
- (C) o PHP gera conteúdo dinâmico no servidor a cada requisição.
- (D) o DNS altera o conteúdo enviado.

RESPOSTA: C

Conteúdo dinâmico: a cada requisição o servidor executa o PHP (ex.: `date()`) e gera um resultado atualizado.

MÓDULO 7 · AULAS 22 A 26 · SEMANAS 8 E 9

PHP básico

PARTE III — INTRODUÇÃO AO PHP

OBJETIVOS DE APRENDIZAGEM

- ✓ Declarar e exibir variáveis em PHP.
- ✓ Identificar os tipos de dados (string, integer, float, boolean) e inspecioná-los com `var_dump()`.
- ✓ Aplicar operadores aritméticos e de concatenação em expressões.
- ✓ Diferenciar operadores de comparação (`==` × `===`).
- ✓ Combinar condições com operadores lógicos (`&&`, `||`, `!`).

AULA 22

Variáveis

Declarando variáveis

Uma **variável** é um “espaço na memória com nome” para guardar dados que o programa usa. Em PHP, toda variável começa com **cifrão (\$)** e não precisa declarar o tipo — o PHP descobre sozinho pelo valor atribuído:

variaveis.php

PHP

```
<?php
$nome = "Maria"; // atribuição de texto
$idade = 17; // número inteiro
$altura = 1.65; // número com casas decimais

echo $nome; // exibe: Maria
echo "<br>";
echo "Nome: $nome - Idade: $idade"; // variável dentro de aspas duplas
?>
```

Regras de nomes de variáveis

- Começam com \$, seguidos de letra ou underscore: `$nome`, `$_total`;
- **Não** podem começar com número nem conter acentos ou espaços: `$2valores`, `$meu nome`;
- Diferenciam maiúsculas: **`$nota` ≠ `$Nota`**;
- Use nomes com significado: `$precoFinal` em vez de `$x` (boas práticas!).

Atribuição × comparação

Um sinal de igual (`=`) **atribui**: `$x = 5` guarda 5 em `$x`. Dois/três iguais (`==`, `===`) **comparam**: `$x == 5` pergunta se `$x` vale 5. Confundir os dois é erro clássico — voltaremos nisso na aula 25.

Constantes

Constantes guardam valores que **não mudam** durante o programa. Define-se com **define()** e usa-se sem \$:

Constantes

PHP

```
<?php
define("PI", 3.14159);
define("ESCOLA", "Instituto Técnico");

echo "Escola: " . ESCOLA . "<br>";
echo "Área do círculo (r=2): " . PI * 2 * 2;
?>
```

DICA

Convenção: constantes em **MAIÚSCULAS** (PI, ESCOLA, TAXA_DESCONTO) — fica fácil reconhecê-las no código.

AULA 23

Tipos de dados e var_dump()

Os 4 tipos principais

Tipos de dados em PHP

Tipo	O que guarda	Exemplos
string	Texto (entre aspas)	"Maria", 'PHP', "123"
integer	Número inteiro	17, -5, 2026
float (double)	Número com casas decimais	1.65, 9.8, -0.5
boolean	Verdadeiro ou falso (lógica)	true, false

Veremos também **arrays** (listas de valores) no módulo 9 — outro tipo fundamental.

var_dump() — a lupa do programador

var_dump() mostra o **tipo** e o **valor** de uma variável. É a ferramenta nº 1 para entender o que está guardado:

tipos.php

PHP

```
<?php
$nome = "Maria";
$idade = 17;
$altura = 1.65;
$aprovado = true;

var_dump($nome); // string(5) "Maria"
echo "<br>";
var_dump($idade); // int(17)
echo "<br>";
var_dump($altura); // float(1.65)
echo "<br>";
var_dump($aprovado); // bool(true)
?>
```

ATENÇÃO

Cuidado: "17" (entre aspas) é **string**; 17 (sem aspas) é **integer**. Parecem iguais na tela, mas o PHP os trata de forma diferente em cálculos e comparações. Na dúvida: var_dump().

Booleanos na prática

Booleanos (true/false) são a base das **decisões** (módulo 8): toda condição — `$idade >= 18`, `$nota >= 7` — resulta em true ou false. No PHP, valores "vazios" (0, "", null, false) são considerados falsos em condições.

AULA 24

Operadores aritméticos e expressões

Os operadores

Operadores aritméticos e de texto

Operador	Nome	Exemplo	Resultado
+	adição	7 + 2	9
-	subtração	7 - 2	5
*	multiplicação	7 * 2	14
/	divisão	7 / 2	3.5
%	resto da divisão (módulo)	7 % 2	1
**	potenciação	7 ** 2	49
.	concatenação (junta textos)	"7" . "2"	"72"

Expressões e precedência

Uma **expressão** combina valores e operadores e produz um resultado. Vale a ordem da matemática: **parênteses** > ****** > ***** / **%** > **+** **-**. Na dúvida, use parênteses para deixar claro:

calculadora.php

PHP

```
<?php
$nota1 = 8.0;
$nota2 = 6.5;
$media = ($nota1 + $nota2) / 2; // parênteses garantem a soma antes da divisão

echo "Média: $media <br>"; // 7.25
echo "Resto de 10 / 3: " . (10 % 3); // 1
?>
```

O operador % (resto) é muito útil

O resto da divisão resolve problemas clássicos: **par ou ímpar** ($\$n \% 2 == 0$ > par), **é múltiplo?** ($\$n \% 5 == 0$ > múltiplo de 5), e cálculos de tempo (90 minutos = $90/60$ > 1 hora, resto $90\%60$ > 30 minutos).

ATENÇÃO

Esta é a semana da **1ª Avaliação (A1 — 30 pts)**: entrega do trabalho HTML (10 pts), atividades do AVA (5 pts) e **teste escrito-prático sobre Web e HTML (15 pts)**. O modelo do teste está na seção Avaliações desta apostila — revise os módulos 1 a 5!

AULA 25

Operadores de comparação

Comparando valores

Operadores de comparação **respondem perguntas** e sempre produzem um resultado **booleano** (true ou false):

Operadores de comparação

Significado	Exemplo	Resultado		
Igual (só o valor)	<code>5 == "5"</code>	true		
Idêntico (valor E tipo)	<code>5 === "5"</code>	false		
Diferente (valor)	<code>5 != 3</code>	true		
Não idêntico (valor ou tipo)	<code>5 !== "5"</code>	true		
	<code>>=</code>	<code><=</code>	maior, menor, maior/igual, menor/igual	<code>7 >= 7</code>

== x === : a diferença que derruba iniciantes

comparacao.php

PHP

```
<?php
$a = 5; // integer
$b = "5"; // string

var_dump($a == $b); // bool(true) - valores iguais
echo "<br>";
var_dump($a === $b); // bool(false) - tipos diferentes!
?>
```

Regra prática

`==` compara apenas o **valor** (o PHP converte os tipos antes — "5" vira 5). `===` compara **valor E tipo** (5 inteiro é diferente de "5" texto). Em projetos reais, prefira `===`: evita surpresas.

Tabela-verdade das comparações

Complete mentalmente antes de executar

PHP

```
10 > 3 // true "ana" == "Ana" // false (case-sensitive!)
10 >= 10 // true 7 != 7 // false
"10" === 10 // false "10" == 10 // true
```

ATENÇÃO

Textos também são case-sensitive na comparação: `"Ana" == "ana"` é **false**. E lembre: em condições, use `==` ou `===` — nunca `=` (que atribui!).

AULA 26**Operadores lógicos****Combinando condições**

Operadores lógicos **combinam** comparações em condições compostas:

Operadores lógicos

Operador	Nome	É true quando...	Exemplo
&&	E (and)	os DOIS lados são true	\$idade >= 18 && \$temCnh
	OU (or)	pelo menos UM lado é true	\$dia == "sab" \$dia == "dom"
!	NÃO (not)	inverte o valor	!\$chovendo (não está chovendo)

Tabelas-verdade

&& (E) – só é true se tudo for true

PHP

```
true && true → true false || false → false
true && false → false true || false → true
false && true → false false || true → true
false && false → false true || true → true

!true → false !false → true
```

condicoes.php – exemplo real

PHP

```
<?php
$idade = 17;
$autorizado = false;

// pode entrar se: for maior de idade OU estiver acompanhado
$podeEntrar = ($idade >= 18) || $autorizado;
var_dump($podeEntrar); // bool(false)

// aprovação: média >= 7 E frequência >= 75%
$media = 8.0; $freq = 80;
$aprovado = ($media >= 7) && ($freq >= 75);
var_dump($aprovado); // bool(true)
?>
```

E x OU no dia a dia

E (&&): “vou à festa se terminar a tarefa **e** meus pais deixarem” — precisa dos dois. **OU (||):** “posso pagar com Pix **ou** cartão” — qualquer um serve. **NÃO (!):** inverte — “não está chovendo”.

DICA

Use **parênteses** em cada comparação: `($a > $b) && ($c == $d)`. Além de evitar erros de precedência, o código fica muito mais legível.

EXERCÍCIOS — MÓDULO 7

Atividades práticas e escritas · resolva no laboratório e complete em casa (AVA)

EXERCÍCIO 1 — Laboratório de variáveis e tipos

PRÁTICO · LABORATÓRIO

Crie **variaveis.php** com 4 variáveis sobre você: \$nome (string), \$idade (integer), \$altura (float) e \$estuda (boolean = true). Exiba cada uma com echo em um parágrafo e confirme os tipos com var_dump().

RESULTADO ESPERADO

Saída com os 4 valores exibidos e var_dump mostrando string(N), int(), float() e bool(true).

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Pergunte: 'o que aconteceria se colocasse a idade entre aspas?' — teste e compare.

EXERCÍCIO 2 — Calculadora simples**PRÁTICO · LABORATÓRIO**

Crie **calculadora.php** com duas variáveis \$a = 12 e \$b = 5. Exiba, cada um em uma linha: soma, subtração, multiplicação, divisão, resto (%) e a média.

RESULTADO ESPERADO

17 · 7 · 60 · 2.4 · 2 · 8.5 — cada operação identificada com um texto.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Confira o uso de parênteses na média e a concatenação com ponto. Desafio: converter 90 minutos em 'Xh Ymin' usando / e %.

EXERCÍCIO 3 — Preveja a saída (== x ===)**ESCRITO**

Sem executar, anote true ou false para cada expressão e depois confirme no PHP:

1. a) 10 == "10" b) 10 === "10" c) 7 != 8 d) "abc" === "abc"
2. e) 5 > 3 && 2 > 4 f) 5 > 3 || 2 > 4 g) !(5 > 3) h) (8 % 2 == 0) && true

RESULTADO ESPERADO

a) true b) false c) true d) true e) false f) true g) false h) true

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Correção comentada item a item. Os itens e/f/h consolidam tabelas-verdade; b consolida ===.

EXERCÍCIO 4 — Elegibilidade com condição composta**PRÁTICO · LABORATÓRIO**

Crie **eleicao.php**: variáveis \$idade = 17 e \$temTitulo = true. Com uma única expressão lógica, defina \$podeVotar = (idade >= 16) && \$temTitulo e exiba o resultado com uma mensagem amigável.

RESULTADO ESPERADO

var_dump true (17 ≥ 16 e tem título) e mensagem do tipo 'Pode votar: 1'.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Testar variações: idade 15 (false), sem título (false). Prepara o terreno para o if (próximo módulo).

TESTE RÁPIDO — MÓDULO 7: PHP básico

5 questões objetivas · responda sem consultar o material e depois corrija · gabarito comentado incluído

1. Qual é a forma correta de declarar uma variável em PHP?

- (A) nome = "Ana";
- (B) var nome = "Ana";
- (C) \$nome = "Ana";
- (D) string \$nome = "Ana";

RESPOSTA: C

Variáveis em PHP começam com \$ e não exigem declaração de tipo.

2. O comando var_dump(\$x) serve para:

- (A) apagar a variável \$x.
- (B) exibir o tipo e o valor da variável.
- (C) somar todos os valores guardados.
- (D) converter \$x para string.

RESPOSTA: B

var_dump é a 'lupa' do programador: mostra tipo e valor, essencial para depurar.

3. Qual é o resultado de 10 % 3?

- (A) 3.33
- (B) 3
- (C) 1
- (D) 0

RESPOSTA: C

% é o resto da divisão inteira: $10 \div 3 = 3$ com resto 1.

4. O resultado de 5 === "5" é:

- (A) true, pois os valores são iguais.
- (B) false, pois os tipos são diferentes (integer × string).
- (C) erro de sintaxe.
- (D) a string "5".

RESPOSTA: B

=== exige valor E tipo idênticos. Com ==, o resultado seria true.

5. A expressão (true && false) || true resulta em:

- (A) true
- (B) false
- (C) null
- (D) erro

RESPOSTA: A

true && false = false; false || true = true. Parênteses primeiro, depois o ||.

AVALIAÇÃO À VISTA

Semana 8: TESTE ESCRITO-PRÁTICO da A1 (15 pts) sobre Web e HTML. Modelo completo na seção AVALIAÇÕES.

MÓDULO 8 · AULAS 27 A 29 · SEMANAS 9 E 10

PHP – lógica (decisões)

PARTE III — INTRODUÇÃO AO PHP

OBJETIVOS DE APRENDIZAGEM

- ✓ Usar a estrutura if para executar blocos condicionais.
- ✓ Aplicar if/else/elseif para múltiplas condições (ex.: situação do aluno).
- ✓ Utilizar switch com case, break e default para menus de opções.
- ✓ Escolher a estrutura adequada (if x switch) para cada situação.

AULA 27

Estrutura IF

Decisões no programa

Programas tomam **decisões**: “se a nota for maior ou igual a 7, exibir aprovado”. A estrutura **if** executa um bloco de código **somente se** a condição for **true**:

Sintaxe e primeiro exemplo

PHP

```
if (condição) {  
    // comandos executados quando a condição é TRUE  
}
```

```
<?php  
$idade = 17;  
  
if ($idade >= 18) {  
    echo "Pode tirar a carteira de motorista."  
}  
echo "Fim do programa."; // sempre executa  
?>
```

Com \$idade = 17, a condição (17 >= 18) é **false**: o bloco do if é pulado e só “Fim do programa.” aparece.

Condição = pergunta de sim ou não

Dentro do if vai uma expressão que resulta em **true** ou **false** — normalmente uma comparação (==, >, <=...) ou combinação com operadores lógicos (&&, ||). Se a resposta for “sim”, o bloco entre { } executa; se for “não”, o bloco é ignorado.

ATENÇÃO

Erro clássico: usar = (atribuição) no lugar de == (comparação) dentro do if. **if (\$x = 5)** não compara — atribui 5 e quase sempre resulta em true! Compare com == ou ===.

Blocos com mais de um comando

Vários comandos dentro do if

PHP

```
<?php
$temIngresso = true;

if ($temIngresso === true) {
    echo "Bem-vindo ao cinema!<br>";
    echo "Sala 3, sessão das 19h.";
}

?>
```

DICA

Chaves { } delimitam o bloco. Se houver um único comando, elas são opcionais — mas o curso adota **sempre usar chaves** (evita bugs e melhora a legibilidade). Indente o conteúdo do bloco com 4 espaços.

AULA 28

IF / ELSE / ELSEIF — situação do aluno

Dois caminhos: if/else

Sintaxe if/else

PHP

```
if (condição) {
    // se TRUE
} else {
    // se FALSE
}
```

Vários caminhos: elseif

Quando há **mais de duas possibilidades**, encadeamos **elseif**. O PHP testa as condições **em ordem** e executa **apenas o primeiro** bloco verdadeiro — os demais são pulados, mesmo que também fossem true:

situacao.php – o clássico “situação do aluno”

PHP

```
<?php
$nota = 6.5;

if ($nota >= 7) {
    echo "APROVADO";
} elseif ($nota >= 5) {
    echo "RECUPERAÇÃO";
} else {
    echo "REPROVADO";
}

// Saída: RECUPERAÇÃO

?>
```

A ordem das condições importa!

Se invertêssemos (testar `$nota >= 5` antes de `$nota >= 7`), uma nota 8 cairia em "RECUPERAÇÃO" — pois `8 >= 5` é a primeira condição verdadeira. Regra: em cadeias de `elseif`, teste da condição **mais restritiva para a menos restritiva**.

Condições compostas dentro do if

Aprovação com frequência (módulo 7 + módulo 8 juntos)

PHP

```
<?php
$media = 8.0;
$frequencia = 70;

if (($media >= 7) && ($frequencia >= 75)) {
    echo "Aprovado!";
} elseif (($media >= 7) && ($frequencia < 75)) {
    echo "Reprovado por falta.";
} else {
    echo "Recuperação ou reprovado por nota.";
}
?>
```

elseif é uma fila de guichês

Você passa pelo 1º guichê (if): se resolver, sai do banco — nem olha os outros. Se não, tenta o 2º (elseif)... Se nenhum resolver, vai ao "outros assuntos" (else). **Só um guichê atende você.**

AULA 29**SWITCH — menu de opções****Quando um valor é comparado com várias opções**

switch compara uma **variável** com uma **lista de valores fixos** (cases). É ideal para **menus** e opções fechadas:

menu.php — switch com case, break e default

PHP

```
<?php
$opcao = 2;

switch ($opcao) {
case 1:
echo "Cadastrar aluno";
break;
case 2:
echo "Consultar notas";
break;
case 3:
echo "Sair do sistema";
break;
default:
echo "Opção inválida!";
}
// Saída: Consultar notas
?>
```

Peças do switch

Peça	Função
switch (\$var)	Variável que será comparada com os cases
case valor:	Um valor possível; se bater, executa o bloco abaixo
break;	Encerra o switch — SEM ele, os cases seguintes também executam!
default:	O “else” do switch: nenhum case bateu

ATENÇÃO

Esquecer o **break** causa o efeito “cascata”: o PHP continua executando os cases seguintes até achar um break ou terminar o switch. É o erro nº 1 com switch!

if x switch — qual usar?

Escolhendo a estrutura

Situação	Melhor estrutura
Faixas de valores (nota $\geq$ 7, idade entre...)	if / elseif
Condições variadas e compostas (&&,)	if / elseif
Um valor comparado a opções fixas (menu, dia da semana)	switch
Sim / não simples	if / else

DICA

switch compara com == (frouxo): case "1" bate com 1. Em menus digitados, converta a entrada para número ou use strings consistentes.

EXERCÍCIOS — MÓDULO 8

Atividades práticas e escritas · resolva no laboratório e complete em casa (AVA)

EXERCÍCIO 1 — Situação do aluno (completo)**PRÁTICO · LABORATÓRIO**

Crie **situacao.php** com as variáveis \$nota e \$frequencia. Regras: aprovado se nota ≥ 7 E frequência ≥ 75 ; recuperação se nota entre 5 e 6,9 (frequência ok); caso contrário, reprovado. Se a frequência for < 75 , reprovado por falta (independentemente da nota). Teste com 3 combinações diferentes.

RESULTADO ESPERADO

Programa com if/elseif/else e condições compostas (&&) exibindo a situação correta para cada teste.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Verifique a ordem das condições (mais restritiva primeiro). Peça que documentem os testes feitos em comentários.

EXERCÍCIO 2 — Verificador de maioridade**PRÁTICO · LABORATÓRIO**

Crie **maioridade.php**: dada \$idade, exiba "Maior de idade — pode votar e dirigir" (≥ 18), "Pode votar mas não dirigir" (16–17) ou "Menor de idade" (< 16).

RESULTADO ESPERADO

Cadeia if/elseif/else com as 3 faixas corretas e mensagens claras.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Testar idades 20, 16 e 12. Erro comum: testar ≥ 16 antes de ≥ 18 (ordem errada).

EXERCÍCIO 3 — Menu da lanchonete com switch**PRÁTICO · LABORATÓRIO**

Crie **lanchonete.php** com \$pedido. Cases: 1 = "Sanduíche — R\$ 8,00", 2 = "Suco — R\$ 5,00", 3 = "Combo — R\$ 12,00", default = "Pedido inválido". Teste com os 4 cenários.

RESULTADO ESPERADO

switch com 3 cases + default, breaks presentes, saída correta para cada valor.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Desafio: remover um break de propósito e observar/explicar a cascata.

EXERCÍCIO 4 — O que aparece na tela?**ESCRITO**

Sem executar, escreva a saída exata de cada programa:

Programas para análise

PHP

```
// A) // B)
$n = 5; $op = 1;
if ($n = 3) { switch ($op) {
echo "Aprovado"; case 1: echo "Um";
} else { case 2: echo "Dois";
echo "Reprovado"; break;
} default: echo "Outro";
}
```

RESULTADO ESPERADO

A) "Aprovado" — pegadinha: $\$n = 3$ é atribuição (resultado $3 = \text{true}$), não comparação! B) "UmDois" — falta break no case 1 (cascata até o break do case 2).

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Os dois erros mais clássicos do módulo em um exercício só. Corrija com discussão aberta: como consertar cada um?

TESTE RÁPIDO — MÓDULO 8: PHP – lógica (decisões)

5 questões objetivas · responda sem consultar o material e depois corrija · gabarito comentado incluído

1. O bloco de um if é executado quando:

- (A) a condição resulta em false.
- (B) a condição resulta em true.
- (C) sempre, independentemente da condição.
- (D) apenas se houver um else.

RESPOSTA: B

if executa o bloco somente se a condição for verdadeira (true).

2. Em uma cadeia if/elseif/else, quando uma condição elseif é verdadeira:

- (A) todos os blocos seguintes também executam.
- (B) apenas o bloco dela executa e o restante da cadeia é ignorado.
- (C) o programa gera um erro.
- (D) o bloco do else executa junto.

RESPOSTA: B

O PHP executa apenas o primeiro bloco verdadeiro da cadeia e ignora os demais.

3. No programa da situação do aluno (≥ 7 aprovado; ≥ 5 recuperação; else reprovado), qual é a saída para nota = 8?

- (A) RECUPERAÇÃO
- (B) REPROVADO
- (C) APROVADO
- (D) APROVADO e RECUPERAÇÃO

RESPOSTA: C

8 ≥ 7 é a primeira condição verdadeira > APROVADO. (Se a ordem estivesse invertida, cairia em recuperação — por isso a ordem importa.)

4. A palavra-chave break dentro de um switch serve para:

- (A) reiniciar o switch.
- (B) encerrar o switch, evitando que os cases seguintes executem.
- (C) pular para o default.
- (D) comparar o valor com o próximo case.

RESPOSTA: B

Sem o break, ocorre a cascata: os blocos dos cases seguintes executam mesmo sem bater.

5. O que está errado em `if ($nota = 7) { echo "Aprovado"; }`?

- (A) Falta o ponto e vírgula dentro do if.
- (B) Deveria usar `==` ou `===` para comparar; `=` está atribuindo 7 à variável.
- (C) A condição deveria estar sem parênteses.
- (D) Nada: o código está correto.

RESPOSTA: B

`=` atribui (e o resultado 7 é 'verdadeiro'), então o bloco sempre executa. Comparação correta: `==` ou `===`.

MÓDULO 9 · AULAS 30 A 35 · SEMANAS 10 A 12

PHP – repetição e arrays

PARTE III — INTRODUÇÃO AO PHP

OBJETIVOS DE APRENDIZAGEM

- ✓ Repetir blocos de código com for, while e do...while.
- ✓ Escolher o laço adequado para cada situação e evitar laços infinitos.
- ✓ Criar e acessar arrays indexados.
- ✓ Percorrer arrays com foreach.
- ✓ Utilizar arrays associativos (chave => valor).

AULA 30

Laço for — repetição com contador

Por que repetir?

Exibir os números de 1 a 1000 com echo seria inviável. **Estruturas de repetição** (laços) executam um bloco **várias vezes** automaticamente. O **for** é o laço com **contador** — use quando você sabe **quantas vezes** repetir.

Sintaxe do for

PHP

```
for (início do contador; condição; passo) {  
    // bloco repetido enquanto a condição for true  
}
```

```
<?php  
for ($i = 1; $i <= 10; $i++) {  
    echo "Número: $i <br>";  
}  
?>
```

As 3 partes do for

Parte	Exemplo	Quando acontece
Inicialização	\$i = 1	Uma única vez, no começo
Condição	\$i <= 10	Testada antes de cada volta (true > repete)
Incremento	\$i++	Executado ao final de cada volta

O contador \$i++ (equivalente a \$i = \$i + 1) faz o for andar: 1, 2, 3... até 10. Quando \$i chega a 11, a condição falha e o laço termina.

Tabuada do 7 com for

PHP

```
<?php
$n = 7;
for ($i = 1; $i <= 10; $i++) {
    $resultado = $n * $i;
    echo "$n x $i = $resultado <br>";
}
?>
```

DICA

O for também conta **para trás** (for (\$i = 10; \$i >= 1; \$i--) — contagem regressiva) e **de 2 em 2** (\$i += 2 — só pares).

AULA 31**Laço while****Repetir ENQUANTO uma condição valer**

O **while** repete o bloco **enquanto** a condição for true — teste **antes** de executar. Use quando **não se sabe** quantas voltas serão necessárias (ex.: repetir até o usuário acertar):

while — soma de 1 até 5

PHP

```
<?php
$contador = 1;
$soma = 0;

while ($contador <= 5) {
    $soma = $soma + $contador; // acumula
    $contador++; // AVANÇA o contador!
}

echo "Soma de 1 a 5: $soma"; // 15
?>
```

ATENÇÃO

Laço infinito: se você esquecer o \$contador++ (ou qualquer coisa que torne a condição false), o while nunca termina — a página trava/carrega para sempre. Regra: **todo while precisa de um passo que caminhe para o fim.**

Padrão de uso do while

1. Inicialize a variável de controle **antes** do laço (\$contador = 1);
2. Teste a condição no while (\$contador <= 5);
3. Dentro do bloco, faça o trabalho **e avance o controle** (\$contador++).

DICA

Se a condição já for false na 1ª verificação, o bloco do while **não executa nenhuma vez**. (Isso vai mudar no do...while — próxima aula.)

AULA 32**Laço do...while****Executa primeiro, testa depois**

O **do...while** executa o bloco **ao menos uma vez** e só depois testa a condição. Útil quando a ação deve acontecer antes da primeira verificação (ex.: pedir uma senha e depois checar se está correta):

do...while – comparando com while

PHP

```
<?php
$x = 10;

// WHILE: testa antes – condição falsa, não executa
while ($x < 5) {
    echo "while executou"; // nunca aparece
}

// DO WHILE: executa 1ª vez e só então testa
do {
    echo "do-while executou 1 vez"; // aparece!
} while ($x < 5);

?>
```

Comparação lado a lado

	while	do...while
Teste da condição	Antes de executar o bloco	Depois de executar o bloco
Mínimo de execuções	0 (pode nunca executar)	1 (sempre executa a 1ª vez)
Sintaxe	while (cond) { }	do { } while (cond); — note o ; final!
Uso típico	Repetir enquanto valer	Executar e depois validar

ATENÇÃO

O **ponto e vírgula** depois do while(cond) no do...while é obrigatório — esquecê-lo gera erro de sintaxe.

AULA 33**Arrays indexados****Uma variável, vários valores**

Guardar 5 alunos em 5 variáveis (\$aluno1, \$aluno2...) é inviável. Um **array** guarda **vários valores em uma única variável**, cada um em uma **posição** (índice):

Criando e acessando arrays

PHP

```
<?php
$alunos = ["Ana", "Bruno", "Carla", "Diego"];

echo $alunos[0]; // Ana — o PRIMEIRO é o índice 0!
echo "<br>";
echo $alunos[2]; // Carla
echo "<br>";

$alunos[4] = "Elisa"; // adiciona na posição 4
echo "Total: " . count($alunos); // 5
?>
```

Índices começam em 0

A posição do primeiro elemento é **[0]**, o segundo é **[1]**, e assim por diante. Um array com 4 itens tem índices 0 a 3. Acessar `$alunos[4]` antes de existir gera aviso (undefined) — cuidado!

Funções úteis

- **count(\$array)** — quantidade de elementos;
- **\$array[] = valor** — adiciona no final (sem informar índice);
- **\$array[i] = valor** — altera/cria a posição i;
- **sort(\$array)** — ordena (crescente).

DICA

Escreva o array em várias linhas quando ficar grande — facilita ler e manter:

Array multilinha

PHP

```
$notas = [
8.5,
7.0,
9.25,
];
```

AULA 34

foreach — percorrendo arrays

Item a item, sem se preocupar com índices

O **foreach** percorre um array **do primeiro ao último elemento** automaticamente. A cada volta, o valor do elemento atual fica disponível em uma variável:

foreach – exibindo a lista de alunos

PHP

```
<?php
$alunos = ["Ana", "Bruno", "Carla", "Diego"];

foreach ($alunos as $aluno) {
    echo "Aluno: $aluno <br>";
}
// foreach ($array as $item) – 'para cada $item do $array'
?>
```

A versão com índice: as \$i => \$valor

foreach com chave e valor

PHP

```
<?php
$notas = [8.5, 7.0, 9.25];

foreach ($notas as $indice => $nota) {
    echo "Nota da posição $indice: $nota <br>";
}
?>
```

foreach x for em arrays

As duas formas de percorrer

PHP

```
// com for – controlando o índice
for ($i = 0; $i < count($alunos); $i++) {
    echo $alunos[$i];
}

// com foreach – mais simples e legível
foreach ($alunos as $aluno) {
    echo $aluno;
}
```

Quando usar cada laço?

for: quando o número de voltas é conhecido/definido por um contador (1 a 10, tabuada).
while/do...while: quando depende de uma condição que muda durante a execução. **foreach:** quando o objetivo é percorrer **todos os elementos de um array** — é o laço dos arrays.

DICA

Acumular dentro do foreach é padrão comum: somar todas as notas para calcular a média (\$soma += \$nota; e ao final \$media = \$soma / count(\$notas)).

AULA 35

Arrays associativos

Chaves com significado

No array indexado, a posição é um número (0, 1, 2...). No **array associativo**, você define as chaves (textos com significado), no formato **chave => valor**:

Ficha do aluno

PHP

```
<?php
$aluno = [
    "nome" => "Ana Silva",
    "idade" => 17,
    "curso" => "Informática",
];

echo $aluno["nome"]; // Ana Silva
echo "<br>";
echo "Curso: " . $aluno["curso"];
echo "<br>";

$aluno["media"] = 9.0; // acrescenta um novo par
?>
```

foreach em arrays associativos

Percorrendo chave => valor

PHP

```
<?php
foreach ($aluno as $campo => $valor) {
    echo "$campo: $valor <br>";
}
// nome: Ana Silva
// idade: 17
// curso: Informática
?>
```

Indexado x associativo

Indexado: posição numérica automática (0, 1, 2...) — ideal para **listas de coisas do mesmo tipo** (vários alunos). **Associativo:** chaves com nome — ideal para **fichas/registros** (um aluno com vários campos). Os dois serão combinados no módulo 10 (array de fichas = multidimensional).

Array associativo = formulário

Cada par chave => valor é como um campo preenchido de um formulário: nome: Ana · idade: 17 · curso: Informática. A chave é o label; o valor é o que foi digitado.

EXERCÍCIOS — MÓDULO 9

Atividades práticas e escritas · resolva no laboratório e complete em casa (AVA)

EXERCÍCIO 1 — Tabuada com for

PRÁTICO · LABORATÓRIO

Crie **tabuada.php** que exiba a tabuada de um número \$n (de 1 a 10) formatada: "7 x 1 = 7". Desafio: exibir apenas os números pares de 1 a 20 em outro laço.

RESULTADO ESPERADO

10 linhas de tabuada corretas; desafio com for (\$i = 2; \$i <= 20; \$i += 2) ou if (\$i % 2 == 0) dentro do laço.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Pergunte qual abordagem do desafio é melhor e por quê (menos voltas × mais simples).

EXERCÍCIO 2 — while × do...while**PRÁTICO · LABORATÓRIO**

Crie **somas.php**: com \$limite = 0, some de 1 até \$limite usando while e depois usando do...while, exibindo os dois resultados. Explique (em comentário) por que os resultados diferem.

RESULTADO ESPERADO

while: 0 (não executa, condição falsa desde o início); do...while: 1 (executa a 1ª volta somando 1). Comentário explicando a diferença do momento do teste.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Excelente para fixar 'mínimo 0 × mínimo 1 execução'. Depois testar com \$limite = 5.

EXERCÍCIO 3 — Lista de alunos com média**PRÁTICO · LABORATÓRIO**

Crie **alunos.php** com dois arrays: \$alunos (5 nomes) e \$notas (5 notas). Exiba cada nome com sua nota usando foreach com índice (\$i => \$valor) e, ao final, a média da turma.

RESULTADO ESPERADO

5 linhas 'Nome — Nota' e a média correta (soma acumulada / count).

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Reforce o padrão de acumulação (\$soma += ...). Desafio: exibir 'aprovado/recuperação' por aluno usando o if do módulo 8 dentro do foreach.

EXERCÍCIO 4 — Ficha do aluno (associativo)**PRÁTICO · LABORATÓRIO**

Crie **ficha.php** com um array associativo contendo nome, idade, curso e cidade de um colega. Exiba a ficha formatada percorrendo com foreach (\$campo => \$valor).

RESULTADO ESPERADO

Ficha exibida linha a linha ('nome: ...', 'idade: ...').

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Confira a sintaxe => e aspas nas chaves. Este é o embrião do 'cadastro' do projeto final.

EXERCÍCIO 5 — Preveja a saída**ESCRITO**

Sem executar, escreva a saída exata:

Analise o código

PHP

```
<?php
$numeros = [3, 7, 2];
$soma = 0;
foreach ($numeros as $n) {
    if ($n > 2) {
        $soma = $soma + $n;
    }
}
echo $soma;
?>
```

RESULTADO ESPERADO

10 — soma de 3 e 7 (o 2 não entra porque a condição é $n > 2$).

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Simular no quadro 'mesa redonda': uma linha por vez, acompanhando \$soma. Combina foreach + if (integração de módulos).

TESTE RÁPIDO — MÓDULO 9: PHP – repetição e arrays

5 questões objetivas · responda sem consultar o material e depois corrija · gabarito comentado incluído

1. Quantas vezes o bloco executa em: `for ($i = 0; $i < 5; $i++) { ... }`?

- (A) 4 vezes
- (B) 5 vezes
- (C) 6 vezes
- (D) infinitas vezes

RESPOSTA: B

\$i assume 0, 1, 2, 3, 4 > 5 voltas. Quando \$i chega a 5, a condição $\$i < 5$ falha.

2. O maior risco ao usar while é:

- (A) esquecer o ponto e vírgula do echo.
- (B) criar um laço infinito por não atualizar a variável de controle.
- (C) usar chaves no bloco.
- (D) o laço nunca executar a primeira vez.

RESPOSTA: B

Sem o avanço (\$contador++ ou similar), a condição nunca vira false e o laço não termina.

3. A diferença do do...while para o while é que o do...while:

- (A) só funciona com arrays.
- (B) executa o bloco pelo menos uma vez, pois testa a condição no final.
- (C) não precisa de condição.
- (D) executa sempre duas vezes.

RESPOSTA: B

do...while = faz e depois confere: mínimo de 1 execução; while = confere e depois faz: pode executar 0 vezes.

4. Dado \$frutas = ["maçã", "banana", "uva"], o comando echo \$frutas[1]; exibe:

- (A) maçã
- (B) banana
- (C) uva
- (D) erro: índice fora do array

RESPOSTA: B

Índices começam em 0: [0]=maçã, [1]=banana, [2]=uva.

5. Para percorrer todos os elementos de um array sem controlar índices manualmente, o laço ideal é:

- (A) for
- (B) while
- (C) foreach
- (D) do...while

RESPOSTA: C

foreach foi criado para arrays: foreach (\$arr as \$item). Com chave: as \$k => \$v.

PARTE IV

FORMULÁRIOS + PHP

AULAS 37 A 42 · SEMANAS 13 E 14

Módulo 11 — Formulários + PHP

MÓDULO 11 · AULAS 37 A 42 · SEMANAS 13 E 14

Formulários + PHP

PARTE IV — FORMULÁRIOS + PHP

OBJETIVOS DE APRENDIZAGEM

- ✓ Enviar dados de formulários com os métodos GET e POST.
- ✓ Ler os dados enviados com \$_GET e \$_POST.
- ✓ Validar campos com isset() e empty().
- ✓ Sanitizar dados com htmlspecialchars() (segurança básica).
- ✓ Construir o fluxo completo: formulário > processamento > validação > resultado.

AULA 37

Método GET e a superglobal \$_GET

Enviando dados pela URL

No módulo 5 criamos formulários HTML; agora eles ganham vida: o **PHP recebe e processa** os dados. Com **method="get"**, os dados viajam **na própria URL**, após um **?**, no formato **nome=valor**:

busca.html — formulário GET

HTML

```
<form action="buscar.php" method="get">
<label for="q">Buscar produto:</label>
<input type="text" id="q" name="produto">
<button type="submit">Buscar</button>
</form>
```

buscar.php — recebendo com \$_GET

PHP

```
<?php
// após buscar 'notebook', a URL fica:
// buscar.php?produto=notebook

$produto = $_GET["produto"];
echo "Você buscou por: $produto";
?>
```

\$_GET é um array!

\$_GET é um **array associativo** criado automaticamente pelo PHP com tudo o que veio na URL: a chave é o **name** do campo no formulário; o valor é o que o usuário digitou. Por isso o atributo **name** é obrigatório em todo campo que será processado!

Características do GET

- Dados **visíveis na URL** — qualquer um vê (e pode alterar!);
- Limitado a URLs curtas (poucos dados);

- Fica no **histórico** e pode ser **favoritado**;
- **Bom para**: buscas, filtros, paginação — dados públicos;
- **Ruim para**: senhas e dados pessoais (nunca usar GET para isso!).

DICA

Teste alterar a URL manualmente: digite `buscar.php?produto=celular` direto na barra de endereços — o resultado muda. Isso mostra a transparência (e o perigo) do GET.

AULA 38**Método POST e \$_POST****Dados no corpo da requisição**

Com **method="post"**, os dados viajam **no corpo da requisição HTTP** — invisíveis na URL. O PHP os entrega no array **\$_POST**:

contato.html + contato.php

PHP

```
<!-- contato.html -->
<form action="contato.php" method="post">
<label for="n">Nome:</label>
<input type="text" id="n" name="nome">
<label for="e">E-mail:</label>
<input type="email" id="e" name="email">
<button type="submit">Enviar</button>
</form>

<!-- contato.php -->
<?php
$nome = $_POST["nome"];
$email = $_POST["email"];
echo "Obrigado, $nome! Confirmação enviada para $email.";
?>
```

GET x POST — comparativo

Quando usar cada método

	GET	POST
Onde viajam os dados	Na URL (após ?)	No corpo da requisição
Visibilidade	Todos veem (URL, histórico)	Não aparecem na URL
Limite de tamanho	Sim (URL curta)	Praticamente sem limite
Favoritar/repetir	Sim — a URL guarda os dados	Não (reenvio pede confirmação)
Leitura no PHP	<code>\$_GET["campo"]</code>	<code>\$_POST["campo"]</code>
Uso típico	Buscas, filtros, links	Cadastros, logins, formulários

Regra de ouro

GET para **consultar** (a ação não altera nada: buscar, filtrar). **POST** para **enviar dados sensíveis ou alterar algo** (cadastrar, editar, excluir, login). Na dúvida em formulários de cadastro: **POST**.

AULA 39**Validação: isset() e empty()****Nunca confie nos dados do usuário**

O usuário pode enviar o formulário vazio, alterar a URL (GET) ou usar ferramentas para forjar requisições. Por isso o servidor **sempre valida** o que chegou — mesmo que o HTML tenha required (ele pode ser contornado!).

As duas funções-chave

isset(\$x): verifica se a variável/campo **foi enviado** (existe). **empty(\$x)**: verifica se o valor está **vazio** ("", 0, null, false). Padrão de validação: primeiro **isset** (veio?), depois **empty** (veio preenchido?).

processar.php – validação de campos obrigatórios

PHP

```
<?php
if ($_SERVER["REQUEST_METHOD"] === "POST") {

    // 1) o campo foi enviado?
    if (!isset($_POST["nome"])) {
        die("Campo nome não recebido.");
    }

    // 2) veio preenchido?
    if (empty($_POST["nome"])) {
        die("Erro: o nome é obrigatório.");
    }

    $nome = $_POST["nome"];
    echo "Cadastro recebido, $nome!";
}
?>
```

Validando vários campos

Coletando erros de todos os campos

PHP

```
<?php
$erros = [];

if (empty($_POST["nome"])) $erros[] = "Nome é obrigatório.";
if (empty($_POST["email"])) $erros[] = "E-mail é obrigatório.";

if (count($erros) > 0) {
    foreach ($erros as $erro) echo "$erro <br>";
} else {
    echo "Tudo certo! Processando cadastro...";
}
?>
```

DICA

O padrão do array \$erros (acima) é profissional: valida **tudo** e mostra **todos** os erros de uma vez — em vez de corrigir um, enviar, descobrir outro...

ATENÇÃO

Checkbox não marcado **não é enviado** pelo formulário: `isset($_POST["termos"])` será `false`. Para saber se foi marcado, teste o `isset`.

AULA 40**Sanitização e segurança básica****O risco de exibir dados sem tratamento**

Se exibirmos na página exatamente o que o usuário digitou, alguém pode enviar **código** em vez de texto — por exemplo `<script>` — que seria executado no navegador de outros visitantes. Esse ataque chama-se **XSS** (cross-site scripting). A defesa básica: **sanitizar** a saída.

htmlspecialchars()

A função **htmlspecialchars(\$texto)** converte caracteres especiais de HTML em entidades inofensivas: `<` vira `<`, `>` vira `>`, `&` vira `&` e aspas também. O texto aparece **normal na tela**, mas **nunca é interpretado como código**.

comentarios.php – exibição segura

PHP

```
<?php
// usuário mal-intencionado digitou no campo 'mensagem':
$mensagem = "<script>alert('hackeado!')</script>";

// JEITO PERIGOSO – executa o script na página:
// echo $mensagem;

// JEITO SEGURO – mostra como texto inofensivo:
echo htmlspecialchars($mensagem);
// exibe: <script>alert('hackeado!')</script>
?>
```

Regra do curso

1. **Valide** a entrada (isset/empty — aula 39);
2. **Sanitize** a saída: todo dado do usuário exibido na página passa por **htmlspecialchars()**;
3. Mensagem de erro clara para o usuário (sem detalhes técnicos).

DICA

Guarde o dado **como foi enviado** (validado) e aplique htmlspecialchars **na hora de exibir**. Em Programação para Internet II veremos sanitização avançada para bancos de dados (prepared statements).

ATENÇÃO

Sanitizar não é opcional “porque ninguém vai atacar meu site de estudos”: é **hábito profissional**. Escreva sempre `echo htmlspecialchars($dado);` — vira automático.

AULA 41

Formulário completo com confirmação

O fluxo completo

Juntando as aulas: um formulário de contato (nome, e-mail, idade, curso e mensagem) com POST, validação, sanitização e **página de confirmação**:

contato.html

HTML

```
<form action="confirmar.php" method="post">
<label for="nome">Nome:</label>
<input type="text" id="nome" name="nome" required>

<label for="email">E-mail:</label>
<input type="email" id="email" name="email" required>

<label for="idade">Idade:</label>
<input type="number" id="idade" name="idade" min="1" max="120">

<label for="curso">Curso:</label>
<select id="curso" name="curso">
<option value="informatica">Informática</option>
<option value="redes">Redes</option>
<option value="adm">Administração</option>
</select>

<label for="msg">Mensagem:</label>
<textarea id="msg" name="mensagem" rows="4"></textarea>

<button type="submit">Enviar</button>
</form>
```

confirmar.php – recebe, valida, sanitiza e confirma

PHP

```
<?php
$erros = [];

if (empty($_POST["nome"])) $erros[] = "Informe o nome.";
if (empty($_POST["email"])) $erros[] = "Informe o e-mail.";
if (empty($_POST["mensagem"])) $erros[] = "Escreva a mensagem.";

if (count($erros) > 0) {
echo "<h2>Corrija os erros:</h2>";
foreach ($erros as $e) echo "<p>$e</p>";
exit; // interrompe: não processa
}

// tudo válido → sanitizar e usar
$nome = htmlspecialchars($_POST["nome"]);
$email = htmlspecialchars($_POST["email"]);
$curso = htmlspecialchars($_POST["curso"]);
$msg = htmlspecialchars($_POST["mensagem"]);

echo "<h2>Mensagem recebida!</h2>";
echo "<p>Nome: $nome</p>";
echo "<p>E-mail: $email</p>";
echo "<p>Curso: $curso</p>";
echo "<p>Mensagem: $msg</p>";
?>
```

Fluxo em 4 etapas

1) **Preenchimento** (HTML: form + labels + required) > 2) **Envio** (method=post, action=confirmar.php) > 3) **Processamento** (PHP: isset/empty, \$_POST, validações) > 4) **Resultado** (confirmação sanitizada ou lista de erros). Esse fluxo é o coração de TODO sistema Web — e do seu projeto final.

AULA 42**Projeto: cadastro de alunos (HTML + PHP)****O projeto do módulo**

Sistema de cadastro em 2 arquivos: **cadastro.html** (formulário) + **processar.php** (recebimento, validação e resultado). É o ensaio geral para o projeto final:

cadastro.html

HTML

```
<!DOCTYPE html>
<html lang="pt-br">
<head><meta charset="UTF-8"><title>Cadastro de aluno</title></head>
<body>
<h1>Cadastro de aluno</h1>
<form action="processar.php" method="post">
<label for="nome">Nome:</label>
<input type="text" id="nome" name="nome" required>

<label for="email">E-mail:</label>
<input type="email" id="email" name="email" required>

<label for="idade">Idade:</label>
<input type="number" id="idade" name="idade" min="10" max="120">

<p>Curso:</p>
<input type="radio" id="inf" name="curso" value="Informática">
<label for="inf">Informática</label>
<input type="radio" id="red" name="curso" value="Redes">
<label for="red">Redes</label>

<button type="submit">Cadastrar</button>
</form>
</body>
</html>
```

processar.php

PHP

```
<?php
$erros = [];

if (empty($_POST["nome"])) $erros[] = "Nome é obrigatório.";
if (empty($_POST["email"])) $erros[] = "E-mail é obrigatório.";
if (empty($_POST["curso"])) $erros[] = "Escolha um curso.";

if (!empty($_POST["idade"]) && ($_POST["idade"] < 10
|| $_POST["idade"] > 120)) {
$erros[] = "Idade inválida (10 a 120).";
}

if (count($erros) > 0) {
echo "<h2>Erro de validação</h2>";
foreach ($erros as $e) echo "<p>$e</p>";
echo '<a href="cadastro.html">Voltar</a>';
exit;
}

$nome = htmlspecialchars($_POST["nome"]);
$email = htmlspecialchars($_POST["email"]);
$curso = htmlspecialchars($_POST["curso"]);
$idade = (int) $_POST["idade"];

echo "<h2>Cadastro realizado!</h2>";
echo "<p>Aluno: $nome ($idade anos)</p>";
echo "<p>Curso: $curso</p>";
echo "<p>Confirmação enviada para: $email</p>";
?>
```

Checklist de qualidade do projeto

- Todos os campos com **label + name** (e id);
- **required** nos essenciais; number com min/max;
- PHP valida **todos** os campos (isset/empty) antes de usar;
- Saída sempre com **htmlspecialchars**;
- Mensagens de erro claras + **link para voltar** ao formulário;
- Código indentado e comentado.

Limitação que virá no futuro

Ao recarregar a página, o cadastro “some”: o array vive só durante aquela execução. Para **guardar de verdade** os dados entre acessos usamos arquivos ou **bancos de dados** (MySQL) — conteúdo de Programação para Internet II. No projeto final, simularemos a persistência com arrays/sessões conforme orientação do professor.

EXERCÍCIOS — MÓDULO 11

Atividades práticas e escritas · resolva no laboratório e complete em casa (AVA)

EXERCÍCIO 1 — Busca com GET

PRÁTICO · LABORATÓRIO

Crie **busca.html** (campo produto, method=get, action=buscar.php) e **buscar.php** que receba \$_GET["produto"], valide (isset/empty) e exiba "Resultado da busca por: X" (sanitizado). Teste também digitando a URL manualmente.

RESULTADO ESPERADO

Busca funcionando; envio vazio mostra erro; URL digitada manualmente funciona; saída com htmlspecialchars.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Peça que testem buscar por 'teste' e observem a sanitização em ação.

EXERCÍCIO 2 — Contato com POST e validação completa

PRÁTICO · LABORATÓRIO

Crie o formulário de contato completo (nome, e-mail, idade, curso em select, mensagem em textarea) + confirmar.php com array \$erros, validação de todos os campos, sanitização e confirmação formatada.

RESULTADO ESPERADO

Fluxo completo funcionando: envio válido > confirmação; campos vazios > lista de todos os erros de uma vez.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Testar os 3 cenários (ok / vazio / parcial). Conferir htmlspecialchars em TODAS as saídas.

EXERCÍCIO 3 — Teste de invasão (sanitização)

PRÁTICO · LABORATÓRIO

No formulário do exercício 2, digite no campo mensagem: **<script>alert('XSS')</script>** e também **negrito?**. Envie e observe o resultado. Depois comente a linha do htmlspecialchars e envie de novo. Explique por escrito a diferença.

RESULTADO ESPERADO

Com htmlspecialchars: os textos aparecem literalmente, nada é executado/aplicado. Sem: o alert dispara e 'negrito?' fica em negrito — prova de que o dado foi interpretado como código.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Exercício de impacto: deixa o XSS 'visível'. Restaurar o código seguro ao final e discutir por que validar só no HTML (required) não basta.

EXERCÍCIO 4 — Projeto do módulo: cadastro de alunos**PRÁTICO · LABORATÓRIO**

Desenvolva em dupla o sistema **cadastro.html + processar.php** conforme a aula 42, atendendo ao checklist de qualidade (6 itens). Entrega pelo AVA.

RESULTADO ESPERADO

Sistema completo validando nome/e-mail/curso (obrigatórios) e idade (10–120), com mensagens de erro, link 'voltar' e saída sanitizada.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Corrigir com o checklist da aula 42, item a item. Este projeto é pré-requisito prático para o projeto final (módulo 12).

TESTE RÁPIDO — MÓDULO 11: Formulários + PHP

5 questões objetivas · responda sem consultar o material e depois corrija · gabarito comentado incluído

1. Com `method="get"`, os dados do formulário são enviados:

- (A) no corpo da requisição, invisíveis.
- (B) na URL, após o `?`, no formato `nome=valor`.
- (C) por e-mail automaticamente.
- (D) em um arquivo temporário no servidor.

RESPOSTA: B

GET coloca os dados na URL (visíveis); POST os envia no corpo da requisição.

2. Para ler no PHP o campo `<input name="email">` enviado por POST, usamos:

- (A) `$_GET["email"]`
- (B) `$_POST["email"]`
- (C) `$_FORM["email"]`
- (D) `$_EMAIL`

RESPOSTA: B

O método do form define a superglobal: `method=post > $_POST`; `method=get > $_GET`. A chave é o nome do campo.

3. As funções `isset()` e `empty()` servem, respectivamente, para verificar se o dado:

- (A) é numérico e se é texto.
- (B) foi enviado e se está vazio.
- (C) está criptografado e se é seguro.
- (D) é verdadeiro e se é falso.

RESPOSTA: B

`isset` = existe/foi enviado; `empty` = está vazio (`""`, `0`, `null`, `false`). Juntas, formam a validação básica.

4. A função htmlspecialchars() protege a página porque:

- (A) criptografa a senha do usuário.
- (B) impede o envio do formulário duas vezes.
- (C) converte <, > e aspas em entidades — o dado vira texto, nunca código.
- (D) valida se o e-mail digitado existe.

RESPOSTA: C

Sanitização de saída: neutraliza tags/scripts injetados (ataque XSS).

5. Para um formulário de LOGIN (e-mail e senha), o método correto e o motivo são:

- (A) GET — porque é mais rápido.
- (B) POST — porque os dados não ficam visíveis na URL nem no histórico.
- (C) GET — porque a senha aparece criptografada na URL.
- (D) Tanto faz: os dois são igualmente seguros para senhas.

RESPOSTA: B

Dados sensíveis (senha!) nunca por GET: ficariam visíveis na URL, no histórico e em logs. POST envia no corpo da requisição.

PARTE V

ORGANIZAÇÃO E DESENVOLVIMENTO DO PROJETO

AULAS 43 A 48 · SEMANAS 15 E 16

Módulo 10 — PHP + HTML — organização, include e funções

O Módulo 10 inclui a aula 36 (revisão cumulativa), ministrada na semana 12, ao final da Parte III — conforme a tabela oficial de módulos do cronograma.

MÓDULO 10 · AULAS 36 E 43 A 48 · SEMANAS 12, 15 E 16

PHP + HTML — organização, include e funções

PARTE V — ORGANIZAÇÃO E DESENVOLVIMENTO DO PROJETO

OBJETIVOS DE APRENDIZAGEM

- ✓ Integrar HTML e PHP gerando páginas dinamicamente.
- ✓ Organizar os arquivos do projeto em pastas (css/, imagens/, includes/).
- ✓ Reutilizar partes da aplicação com include (cabeçalho e rodapé).
- ✓ Criar funções com parâmetros e retorno (function, return).
- ✓ Trabalhar com arrays multidimensionais e foreach aninhado.
- ✓ Gerar tabelas HTML dinamicamente a partir de dados em PHP.

AULA 36**Revisão cumulativa e projeto “sistema de cadastro simples”****O que já sabemos**

Antes de avançar, esta aula faz uma **revisão cumulativa** de tudo o que foi visto em PHP até aqui: variáveis, tipos, operadores, decisões (if/elseif/else, switch), laços (for, while, do...while) e arrays (indexados, associativos, foreach).

Checklist da revisão

Tema	Você consegue...
Variáveis e tipos	declarar \$nome, usar var_dump e explicar string × int × float × bool?
Operadores	calcular média com parênteses e usar % (resto)?
Comparações	explicar a diferença entre == e ===?
Decisões	programar a situação do aluno com if/elseif/else?
Switch	montar um menu com case, break e default?
Laços	escolher entre for, while e do...while?
Arrays	criar array indexado e associativo e percorrê-los com foreach?

Projeto integrador: sistema de cadastro simples

Juntando HTML + PHP: uma página que **armazena alunos em um array** (associativo e multidimensional) e os **exibe em uma tabela HTML** gerada dinamicamente:

cadastro_simples.php

PHP

```
<!DOCTYPE html>
<html lang="pt-br">
<head><meta charset="UTF-8"><title>Cadastro simples</title></head>
<body>
<h1>Alunos cadastrados</h1>
<?php
$alunos = [
["nome" => "Ana", "curso" => "Informática", "nota" => 9.0],
["nome" => "Bruno", "curso" => "Redes", "nota" => 7.5],
];
?>
<table border="1">
<tr><th>Nome</th><th>Curso</th><th>Nota</th></tr>
<?php foreach ($alunos as $a): ?>
<tr>
<td><?= $a["nome"] ?></td>
<td><?= $a["curso"] ?></td>
<td><?= $a["nota"] ?></td>
</tr>
<?php endforeach; ?>
</table>
</body>
</html>
```

Sintaxe alternativa e <?= ?>

Misturando HTML e PHP, podemos usar **if (): endif;** e **foreach (): endforeach;** (mais legíveis em templates) e a abreviação **<?= \$valor ?>**, que equivale a **<?php echo \$valor; ?>** — perfeita para “imprimir” dados dentro do HTML.

AULA 43**Organização de arquivos do projeto****Por que organizar?**

Projetos crescem rápido. Sem organização, encontrar arquivos vira um pesadelo. A estrutura padrão separa cada tipo de arquivo em sua **pasta**:

Estrutura profissional do projeto

TEXTO

```
meu-projeto/  
├── index.php ← página inicial  
├── cadastro.php ← formulário de cadastro  
├── listar.php ← listagem dinâmica  
├── processar.php ← recebe e valida os dados  
├── css/  
│   └── estilo.css ← estilos (aparência)  
├── imagens/  
│   └── logo.png ← imagens do site  
├── includes/  
├── cabecalho.php ← parte reutilizável (topo)  
└── rodape.php ← parte reutilizável (rodapé)
```

Organizar = manter

1) **Encontrar rápido**: cada coisa no seu lugar; 2) **Reutilizar**: includes/ evita copiar e colar código; 3) **Hospedar fácil**: a estrutura vai inteira para o servidor; 4) **Trabalhar em equipe**: cada um sabe onde mexer. Esses mesmos princípios valem para o **projeto final** (módulo 12).

DICA

Regras de nomes (relembrando): minúsculas, sem acentos e sem espaços — use **hífen** ou **underline** (processar-dados.php ou processar_dados.php).

AULA 44**include — reutilizando partes da aplicação****O problema da repetição**

Cabeçalho (logo + menu) e rodapé aparecem em **todas** as páginas. Copiar e colar esse código em cada arquivo gera manutenção pesada: mudar o menu exigiria editar 10 arquivos! A solução: escrever **uma vez** e **incluir** onde for preciso.

includes/cabecalho.php

PHP

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
<meta charset="UTF-8">
<title>Meu Projeto</title>
</head>
<body>
<header>
<h1>Sistema de Cadastro</h1>
<nav>
<a href="index.php">Início</a> |
<a href="cadastro.php">Cadastrar</a> |
<a href="listar.php">Listar</a>
</nav>
<hr>
</header>
```

includes/rodape.php

PHP

```
<hr>
<footer>
<small>Curso Técnico em Informática — <? = date("Y") ?></small>
</footer>
</body>
</html>
```

index.php – página usando os includes

PHP

```
<?php include "includes/cabecalho.php"; ?>

<h2>Bem-vindo!</h2>
<p>Este é o sistema de cadastro da turma.</p>

<?php include "includes/rodape.php"; ?>
```

include x require

Comando	Se o arquivo não existir...	Uso recomendado
include	avisa (warning) e a página continua	partes opcionais
require	erro fatal e a página para	partes essenciais (cabecalho!)
include_once / require_once	idem, mas ignora inclusão duplicada	evita incluir 2× o mesmo arquivo

ATENÇÃO

O caminho do include é **relativo ao arquivo que o chama**. De index.php (raiz), usa-se "includes/cabecalho.php". Teste sempre navegando entre as páginas — link errado no menu é o sintoma mais comum.

AULA 45**Funções — function, parâmetro e return****O que são funções**

Uma **função** é um bloco de código com **nome**, que executa uma tarefa e pode **devolver um resultado**. Assim como o include evita repetir HTML, funções evitam repetir **lógica**:

Criando e chamando funções

PHP

```
<?php
// 1) DEFINIR a função
function saudacao($nome) {
return "Olá, $nome! Seja bem-vindo(a).";
}

// 2) CHAMAR a função (quantas vezes quiser)
echo saudacao("Ana"); // Olá, Ana! Seja bem-vindo(a).
echo "<br>";
echo saudacao("Bruno"); // Olá, Bruno! Seja bem-vindo(a).
?>
```

Anatomia da função

Peça	Papel	No exemplo
function nome	Define a função	function saudacao
parâmetro	Valor que ENTRA (dentro dos parênteses)	\$nome
return	Valor que SAI (e encerra a função)	return "Olá, ..."
chamada	Usar a função: nome(valor)	saudacao("Ana")

Parâmetro x argumento x retorno

Parâmetro: a "variável de entrada" na definição (\$nome). **Argumento:** o valor passado na chamada ("Ana"). **Retorno:** o que a função devolve com return — quem chama decide o que fazer com ele (exibir, guardar em variável, comparar...). Sem return, a função devolve null.

Funções com vários parâmetros e valor padrão

Mais exemplos

PHP

```
<?php
function soma($a, $b) {
    return $a + $b;
}

function areaRetangulo($base, $altura = 1) { // altura opcional
    return $base * $altura;
}

echo soma(3, 4); // 7
echo areaRetangulo(5); // 5 (altura padrão = 1)
echo areaRetangulo(5, 2); // 10
?>
```

DICA

Boas práticas: nome de função descreve a ação em verbo (calcularMedia, verificarIdade); uma função faz **uma** tarefa; documentação com comentário antes da definição.

AULA 46**Funções na prática — utilitários do sistema****Funções que usaremos no projeto final**

funcoes.php – caixa de ferramentas

PHP

```
<?php
// Calcula a média de duas notas
function calcularMedia($nota1, $nota2) {
    return ($nota1 + $nota2) / 2;
}

// Verifica se é maior de idade
function verificarMaioridade($idade) {
    return $idade >= 18;
}

// Calcula o preço com desconto
function calcularDesconto($preco, $percentual) {
    return $preco - ($preco * $percentual / 100);
}

// Formata a situação do aluno
function situacao($media) {
    if ($media >= 7) return "Aprovado";
    if ($media >= 5) return "Recuperação";
    return "Reprovado";
}
?>
```

Qualquer página do projeto pode usar essas funções bastando incluir o arquivo:

Usando as funções em outra página

PHP

```
<?php
require_once "funcoes.php";

$media = calcularMedia(8.0, 6.5);
echo "Média: $media <br>"; // 7.25
echo "Situação: " . situacao($media); // Aprovado
echo "<br>";
echo "Maior de idade? ";
echo verificarMaioridade(17) ? "Sim" : "Não"; // Não
echo "<br>Preço final: R$ " . calcularDesconto(100, 15); // 85
?>
```

Funções que retornam true/false

verificarMaioridade devolve um **booleano** — ideal para usar direto em condições: **if (verificarMaioridade(\$idade)) { ... }**. A expressão **cond ? "Sim" : "Não"** é o operador ternário: um if/else de uma linha (condição ? valor se true : valor se false).

DICA

Organização real: salve as funções em **funcoes.php** (ou em includes/) e use **require_once** nas páginas — assim elas ficam disponíveis no projeto inteiro, como uma caixa de ferramentas.

AULA 47**Arrays multidimensionais****Array de arrays**

Um **array multidimensional** é um array cujos elementos **também são arrays**. É assim que guardamos uma **lista de registros** (vários alunos, cada um com seus campos):

Lista de alunos (array de fichas)

PHP

```
<?php
$alunos = [
    ["nome" => "Ana", "nota1" => 8.0, "nota2" => 9.0],
    ["nome" => "Bruno", "nota1" => 6.0, "nota2" => 7.0],
    ["nome" => "Carla", "nota1" => 9.5, "nota2" => 8.5],
];

// acesso direto: aluno da posição 0, campo nome
echo $alunos[0]["nome"]; // Ana
?>
```

foreach aninhado — percorrendo as duas dimensões

Percorrendo a lista completa

PHP

```
<?php
foreach ($alunos as $aluno) { // 1ª dimensão: cada aluno
    echo "Aluno: " . $aluno["nome"] . "<br>";
    foreach ($aluno as $campo => $valor) { // 2ª dimensão: cada campo
        echo "&nbsp;&nbsp;&nbsp;$campo = $valor <br>";
    }
    echo "<hr>";
}
?>
```

Lendo a estrutura

\$alunos é uma **lista** (índices 0, 1, 2). Cada posição guarda uma **ficha** (array associativo com nome, nota1, nota2). Por isso o acesso usa **dois colchetes**: \$alunos[0]["nome"] = "na lista, posição 0; na ficha, campo nome". É exatamente a estrutura de uma **tabela**: linhas (alunos) × colunas (campos).

Calculando com os dados

Média de cada aluno + média da turma

PHP

```
<?php
$somaTurma = 0;
foreach ($alunos as $aluno) {
    $media = ($aluno["nota1"] + $aluno["nota2"]) / 2;
    echo $aluno["nome"] . ": média $media <br>";
    $somaTurma += $media;
}
echo "Média da turma: " . ($somaTurma / count($alunos));
?>
```

AULA 48**Projeto: listagem dinâmica de alunos (semana da A2)****PHP gerando tabela HTML**

O projeto da aula junta tudo: um array multidimensional de alunos + foreach gerando as linhas da tabela + função de situação:

listar.php — listagem dinâmica completa

PHP

```
<?php require_once "includes/cabecalho.php"; ?>
<?php
$alunos = [
["nome" => "Ana", "curso" => "Informática", "nota" => 9.0],
["nome" => "Bruno", "curso" => "Redes", "nota" => 6.0],
["nome" => "Carla", "curso" => "Informática", "nota" => 7.5],
];
function situacao($nota) {
if ($nota >= 7) return "Aprovado";
if ($nota >= 5) return "Recuperação";
return "Reprovado";
}
?>
<h2>Alunos matriculados (<?= count($alunos) ?>)</h2>
<table border="1">
<thead>
<tr><th>Nome</th><th>Curso</th><th>Nota</th><th>Situação</th></tr>
</thead>
<tbody>
<?php foreach ($alunos as $a): ?>
<tr>
<td><?= $a["nome"] ?></td>
<td><?= $a["curso"] ?></td>
<td><?= $a["nota"] ?></td>
<td><?= situacao($a["nota"]) ?></td>
</tr>
<?php endforeach; ?>
</tbody>
</table>
<?php include "includes/rodape.php"; ?>
```

O poder da geração dinâmica

Repare: escrevemos as tags <tr>/<td> **uma única vez**; o foreach as **repete para cada aluno**. Com 3 ou 300 alunos, o código é o mesmo — só o array muda. É assim que sistemas reais exibem listas de produtos, pedidos, contatos... E no projeto final, é assim que será a sua listagem.

ATENÇÃO

AVALIAÇÃO 2 (A2 — 30 pts) nesta aula/semana 16 (S): prova obrigatória (pode ser em duplas ou com consulta, conforme o Plano de Ensino) sobre HTML (estrutura a formulários), PHP (variáveis a arrays/funções) e GET/POST/validação. Modelo de prova na seção **Avaliações** desta apostila — estudem por ele!

EXERCÍCIOS — MÓDULO 10

Atividades práticas e escritas · resolva no laboratório e complete em casa (AVA)

EXERCÍCIO 1 — Cabeçalho e rodapé reutilizáveis**PRÁTICO · LABORATÓRIO**

Na pasta do seu projeto, crie **includes/cabecalho.php** (DOCTYPE, head, título, menu com 3 links) e **includes/rodape.php** (footer com o ano atual via `date("Y")`). Reescreva `index.php`, `cadastro.php` e `listar.php` usando os includes.

RESULTADO ESPERADO

3 páginas exibindo o mesmo cabeçalho/rodapé; alterar o título do menu uma única vez atualiza todas as páginas.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Testar clicando no menu em todas as páginas (caminhos relativos!). Desafio: destacar no menu a página atual.

EXERCÍCIO 2 — Caixa de ferramentas (funções)

PRÁTICO · LABORATÓRIO

Crie **funcoes.php** com: `calcularMedia($n1, $n2)`, `verificarMaioridade($idade)`, `calcularDesconto($preco, $percentual)` e `situacao($media)`. Crie **teste_funcoes.php** que as chame com valores de teste e exiba os resultados.

RESULTADO ESPERADO

Testes: `media(8, 6.5) = 7.25` · `maioridade(17) = false` · `desconto(100, 15) = 85` · `situacao(7.25) = 'Aprovado'`.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Verificar uso correto de `return` (e não `echo` dentro da função — a função devolve, quem chama exibe). Discutir por que isso importa (reaproveitamento).

EXERCÍCIO 3 — Lista de alunos com médias

PRÁTICO · LABORATÓRIO

Crie um array multidimensional com 4 alunos (`nome`, `nota1`, `nota2`). Com `foreach`: exiba nome e média de cada um (usando `calcularMedia` do exercício 2) e a situação; ao final, a média geral da turma.

RESULTADO ESPERADO

4 linhas 'Nome — média — situação' + média da turma, usando `require_once 'funcoes.php'`.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Este exercício é a base direta do projeto da aula 48 e da listagem do projeto final.

EXERCÍCIO 4 — Listagem dinâmica em tabela

PRÁTICO · LABORATÓRIO

Transforme o exercício 3 em **listar.php**: tabela HTML com `thead`/`tbody` gerada por `foreach` (colunas: Nome, Média, Situação) + `tfoot` com a média da turma em `colspan`. Use `cabecalho/rodape` via `include`.

RESULTADO ESPERADO

Página completa: menu do include, tabela dinâmica com 4 linhas e tfoot com média geral.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Checklist: <?= ?> usado corretamente? endforeach presente? tfoot com colspan? Compare as soluções dos grupos no projetor.

TESTE RÁPIDO — MÓDULO 10: PHP + HTML — organização, include e funções

5 questões objetivas · responda sem consultar o material e depois corrija · gabarito comentado incluído

1. A principal vantagem de usar include para o cabeçalho do site é:

- (A) a página carrega mais rápido no navegador.
- (B) escrever o cabeçalho uma única vez e reutilizá-lo em todas as páginas.
- (C) o Google indexa melhor o site.
- (D) include adiciona estilos CSS automaticamente.

RESPOSTA: B

Reutilização e manutenção: mudar o cabeçalho em um único arquivo atualiza o site inteiro.

2. Qual a diferença entre include e require quando o arquivo não é encontrado?

- (A) Não há diferença.
- (B) include gera erro fatal; require apenas avisa.
- (C) require gera erro fatal (para a página); include emite warning e continua.
- (D) require só funciona com arquivos PHP e include com HTML.

RESPOSTA: C

require = essencial (falhou, para tudo); include = opcional (falhou, avisa e segue).

3. O que a função a seguir retorna? `function dobro($x) { $x * 2; } (chamada: dobro(5))`

- (A) 10
- (B) 5
- (C) 0
- (D) null — falta a palavra-chave return.

RESPOSTA: D

Sem return, a função não devolve nada (null). O correto: `return $x * 2;`

4. Dado `$alunos = [ ["nome" => "Ana", "nota" => 9] ]`, como acessar a nota da primeira aluna?

- (A) `$alunos["nota"]`
- (B) `$alunos[0]["nota"]`
- (C) `$alunos[1]["nota"]`
- (D) `$alunos[0][0]`

RESPOSTA: B

Dois colchetes: posição na lista ([0]) + campo da ficha (["nota"]).

5. Em uma página PHP que gera uma tabela com 50 produtos a partir de um array, as tags `<tr>/<td>` são escritas:

- (A) 50 vezes, uma para cada produto.
- (B) uma única vez dentro do `foreach`, que repete a linha para cada produto.
- (C) no CSS, que monta a tabela.
- (D) pelo navegador, automaticamente.

RESPOSTA: B

Geração dinâmica: o `foreach` repete o bloco HTML para cada item do array — o código não cresce com a quantidade de dados.

AVALIAÇÃO À VISTA

AVALIAÇÃO 2 (A2 — 30 pts): prova obrigatória nesta semana (aula 48) — HTML, PHP e GET/POST. Modelo completo na seção AVALIAÇÕES.

PARTE VI

DESENVOLVIMENTO DO PROJETO FINAL

AULAS 49 A 72 · SEMANAS 17 A 24

Módulo 12 — Projeto final integrador

MÓDULO 12 · AULAS 49 A 72 · SEMANAS 17 A 24

Projeto final integrador

PARTE VI — DESENVOLVIMENTO DO PROJETO FINAL

OBJETIVOS DE APRENDIZAGEM

- ✓ Planejar uma aplicação Web simples: tema, objetivo, usuários e requisitos.
- ✓ Estruturar o projeto em pastas e arquivos profissionais.
- ✓ Desenvolver as páginas: inicial (menu), cadastro (formulário), processamento e listagem dinâmica.
- ✓ Validar dados, tratar erros e testar todas as funcionalidades (tabela de testes).
- ✓ Identificar e corrigir erros de sintaxe, lógica e execução.
- ✓ Preparar e apresentar o projeto final (roteiro, demonstração e avaliação).

AULA 49

Definição do projeto: grupos e tema

O que é o projeto final

Uma **aplicação Web simples e funcional**, desenvolvida **em grupos**, integrando todo o curso: **HTML** (estrutura e conteúdo), **PHP** (processamento e conteúdo dinâmico), **formulários** (entrada de dados), **validação** (controle dos dados), **arrays** (armazenamento temporário) e **organização de arquivos** (estrutura do projeto).

Temas sugeridos

Escolham **UM** tema (ou proponham outro ao professor)

Tema	Cadastro de...	Listagem de...
Cadastro de alunos	nome, idade, curso, e-mail	alunos com situação
Agenda de contatos	nome, telefone, e-mail, categoria	contatos por categoria
Catálogo de produtos	nome, categoria, preço, estoque	produtos com estoque baixo
Sistema de contatos (mensagens)	nome, e-mail, assunto, mensagem	mensagens recebidas
Cadastro de clientes	nome, CPF, cidade, telefone	clientes por cidade
Biblioteca	título, autor, gênero, disponível?	livros e disponibilidade
Controle de tarefas	título, prazo, prioridade, status	tarefas por status

Formação dos grupos e divisão de papéis

- Grupos de 3 a 4 alunos (critério do professor);
- Papéis sugeridos: **líder/organizador**, **front-end** (HTML/visual), **back-end** (PHP/lógica), **testador/documentador** — todos programam, mas cada um lidera uma frente;
- Combinem: encontros fora da aula (AVA/WhatsApp), repositório ou Drive compartilhado da pasta do projeto.

DICA

Escolham um tema que o grupo **entenda do assunto**: saber como funciona uma biblioteca ou uma loja facilita levantar requisitos. Tema simples e bem feito vale mais que tema ambicioso pela metade.

AULA 50**Levantamento de requisitos****Antes de codificar: planejar**

Requisitos são as respostas de 5 perguntas sobre o sistema. Escrevê-las evita retrabalho — é o que profissionais fazem antes de programar:

As 5 perguntas do planejamento

Pergunta	Exemplo (tema: Biblioteca)
1. Qual o OBJETIVO do sistema?	Controlar o acervo e a disponibilidade dos livros
2. QUEM vai usar?	Bibliotecário (cadastra) e aluno (consulta)
3. QUAIS FUNCIONALIDADES?	Cadastrar livro, listar acervo, buscar por título
4. QUAIS PÁGINAS?	index.php, cadastro.php, listar.php, processar.php
5. QUAIS DADOS (campos)?	título, autor, gênero, ano, disponível (sim/não)

Modelo de documento de requisitos (Entrega 1)

requisitos.txt – modelo (1 página)

TEXTO

```

PROJETO FINAL – PROGRAMAÇÃO PARA INTERNET I
Grupo: nomes · Turma: 2/2026 · Tema: Biblioteca da Escola

1. OBJETIVO: controlar o acervo de livros da biblioteca.
2. USUÁRIOS: bibliotecário (cadastro) e alunos (consulta).
3. FUNCIONALIDADES:
- Cadastrar livro (título, autor, gênero, ano, disponível)
- Listar todos os livros em tabela
- Mostrar total de livros e quantos disponíveis
4. PÁGINAS: index.php · cadastro.php · processar.php · listar.php
5. DADOS: título (texto, obrigatório) · autor (texto, obrigatório)
gênero (select) · ano (número, 1900-2026) · disponível (checkbox)
6. VALIDAÇÕES: título/autor obrigatórios; ano entre 1900 e 2026.

```

Requisito bem escrito é testável

“O sistema deve ser bonito” não é requisito (como testar?). “A listagem exibe título, autor e disponibilidade em tabela com thead” é requisito — dá para verificar sim/não. Escrevam requisitos **concretos**: viram a tabela de testes da aula 58!

AULA 51**Estrutura do projeto****Criando o esqueleto**

Com os requisitos definidos, criamos a estrutura de arquivos — exatamente o padrão do módulo 10, agora aplicado ao projeto do grupo:

Estrutura do projeto final

TEXTO

```
projeto-final-grupo1/  
├── index.php ← página inicial com o menu  
├── cadastro.php ← formulário de cadastro  
├── processar.php ← recebe, valida e confirma  
├── listar.php ← listagem dinâmica (tabela)  
├── sobre.php ← sobre o sistema (opcional)  
├── funcoes.php ← funções utilitárias do grupo  
├── css/  
│   └── estilo.css  
├── imagens/  
│   └── logo.png  
├── includes/  
├── cabecalho.php  
└── rodape.php
```

Ordem de construção recomendada

1. Pastas e arquivos vazios (o esqueleto completo);
2. includes/cabecalho.php e rodape.php (menu pronto desde o início!);
3. index.php com o menu navegável (testar os links);
4. cadastro.php (o formulário HTML completo);
5. processar.php (validação + confirmação);
6. listar.php (tabela dinâmica);
7. funcoes.php (utilitários) e ajustes de interface (css).

DICA

Commit mental: ao terminar cada passo, **teste no navegador** antes de passar ao próximo. Construir tudo e testar só no fim é a receita para uma pilha de erros difíceis de achar.

ATENÇÃO

Todo o grupo deve ter a **mesma versão** dos arquivos (Drive/pendrive compartilhado, combinado na aula 49). Arquivo duplicado com nomes diferentes (Cadastro.php × cadastro.php) causa bugs ‘fantasma’.

AULA 52**Página inicial, cadastro e processamento**

Aulas 52 a 54 — as 3 páginas centrais

Semana 18: construímos o núcleo do sistema — **index.php** (menu), **cadastro.php** (formulário completo) e **processar.php** (fluxo de recebimento).

index.php — página inicial com menu

PHP

```
<?php require_once "includes/cabecalho.php"; ?>

<h2>Bem-vindo ao sistema <?= $nomeSistema ?></h2>
<p>Escolha uma opção no menu acima:</p>
<ul>
<li><a href="cadastro.php">Cadastrar</a> — incluir novo registro</li>
<li><a href="listar.php">Listar</a> — ver os cadastros</li>
<li><a href="sobre.php">Sobre</a> — informações do sistema</li>
</ul>

<?php include "includes/rodape.php"; ?>
```

cadastro.php — checklist do formulário

- Todos os campos dos requisitos (item 5) presentes;
- Cada campo com **label + name + id**;
- Tipos certos: text, email, number (min/max), date, select, radio, checkbox;
- **required** nos obrigatórios; placeholder com exemplo;
- method="post" e action="processar.php";
- botão submit com texto claro ("Cadastrar").

processar.php — fluxo completo

1. Confirmar que a requisição é POST (\$_SERVER["REQUEST_METHOD"]);
2. Validar cada campo: isset/empty + regras (faixa, formato) > array \$erros;
3. Se houver erros: exibir todos + link "Voltar ao cadastro" + exit;
4. Sem erros: sanitizar (htmlspecialchars) e montar a confirmação;
5. (Extra do grupo) guardar o registro no array/sessão para a listagem.

DICA

Reproveitem o projeto do módulo 11 (cadastro de alunos) como base — adaptem os campos ao tema do grupo. Não comecem do zero: comecem do que já funciona.

AULA 55

Listagem dinâmica, interface e validação do projeto

Aulas 55 a 57 — completar e polir

Listagem dinâmica (aula 55): a tabela gerada por foreach a partir dos dados cadastrados — padrão da aula 48, com os campos do tema do grupo:

listar.php – núcleo da listagem

PHP

```
<table border="1">
<thead>
<tr><th>#</th><th>Título</th><th>Autor</th><th>Disponível</th></tr>
</thead>
<tbody>
<?php foreach ($livros as $i => $livro): ?>
<tr>
<td><?= $i + 1 ?></td>
<td><?= htmlspecialchars($livro["titulo"]) ?></td>
<td><?= htmlspecialchars($livro["autor"]) ?></td>
<td><?= $livro["disponivel"] ? "Sim" : "Não" ?></td>
</tr>
<?php endforeach; ?>
</tbody>
<tfoot>
<tr><td colspan="4">Total: <?= count($livros) ?> registros</td></tr>
</tfoot>
</table>
```

Interface (aula 56)

- Títulos claros (h1/h2) em cada página;
- Menu consistente (include) e destacado na página atual;
- Mensagens de sucesso/erro visíveis e educadas;
- CSS mínimo de apoio: cores, espaçamento, tabela legível (border-collapse, padding nas células);
- Sem texto 'solto' sem contexto: tudo com label/título.

Validação do projeto (aula 57)

Antes de considerar pronto, o grupo testa o sistema como um todo — checklist:

- Formulário com campos vazios › mensagens corretas?
- Valores inválidos (idade 999, ano 1800) › bloqueados?
- Dados obrigatórios ausentes › erro claro + voltar?
- Navegação: todos os links do menu funcionam de todas as páginas?
- Listagem: mostra os dados cadastrados e o total?
- Texto com ou <script> digitado › exibido como texto (sanitizado)?

AULA 58

Testes: a tabela de testes

Testar é parte de programar

Software profissional é **testado** com método, não "clitando de qualquer jeito". A ferramenta da aula: a **tabela de testes** — cada linha é um **caso de teste** com **entrada** e **resultado esperado**:

Tabela de testes – exemplo (tema Biblioteca)

TEXT0

#	Teste	Entrada	Esperado
1	Cadastro válido	título/autor ok, ano 2020	mensagem de sucesso
2	Campos vazios	nada preenchido	erros de obrigatório
3	Ano inválido	ano = 1800	erro 'ano inválido'
4	Texto com tags	título = teste	exibido como texto
5	Listagem	3 cadastros feitos	tabela com 3 linhas
6	Navegação	menu em todas as páginas	links funcionando

Como montar a sua tabela

1. Liste as funcionalidades do sistema (dos requisitos);
2. Para cada uma, crie ao menos: **1 teste válido**, **1 teste inválido** e **1 teste de limite** (ex.: ano = 1900 e 2026 exatos);
3. Escreva o resultado esperado **antes** de testar;
4. Execute, marque OK/FALHOU e registre o que aconteceu;
5. Toda FALHA vira tarefa de correção (aula 59).

Teste de limite (caso de borda)

Erros gostam de se esconder nas **bordas**: ano exatamente 1900 (mínimo permitido), 2026 (máximo), 1899 (um abaixo), 2027 (um acima), campo com 1 caractere, campo enorme... Testar os limites é o que separa um teste amador de um teste profissional.

AULA 59

Correção de erros (debugging)

Os 3 tipos de erro

Classificação dos erros

Tipo	O que é	Exemplo	Sintoma
Sintaxe	Código fora das regras da linguagem	esquecer ; ou fechar aspas	mensagem de Parse error com arquivo e linha
Lógica	Código válido, mas faz a coisa errada	média = (\$n1+\$n2)/3	sem mensagem — resultado errado
Execução	Falha durante o funcionamento	acessar \$alunos[9] que não existe	Warning/Notice ou página quebrada

A estratégia em 5 passos

1. **Ler** a mensagem de erro COMPLETA (ela diz arquivo e linha!);
2. **Identificar** o arquivo e a linha indicados;
3. **Entender** o que o código quis fazer × o que fez (var_dump é seu amigo);
4. **Corrigir** a causa — não o sintoma;
5. **Testar** de novo (e refazer os testes que já passavam).

Lendo uma mensagem de erro do PHP

TEXTO

```
Parse error: syntax error, unexpected token "echo",
expecting ";" in /htdocs/projeto/listar.php on line 24
```

```
| | | |
tipo do erro | | arquivo e LINHA
o que faltou o que ele encontrou
```

Ferramentas de investigação

- **var_dump(\$variavel)** + exit; — para o programa e mostra o conteúdo;
- **echo** estratégico — “chegou aqui”, “valor = \$x”;
- **Comentar** blocos (//) para isolar a parte problemática;
- **F12 > Console** no navegador — erros de HTML/JS;
- Testar com **entradas simples** (números redondos, textos curtos).

DICA

Erro de lógica é o mais traiçoeiro (não aparece mensagem!). Defesa: escreva o resultado esperado ANTES (tabela de testes) e compare. Se differiu, var_dump nas variáveis do cálculo.

ATENÇÃO

Resista ao impulso de reescrever o arquivo inteiro quando algo falha. Localize primeiro: 90% dos erros estão em 1 linha. Reescrever sem entender = o erro volta.

AULA 60

Revisão geral da ementa

Revisão cumulativa — preparação para a prova final

A aula 60 revisa **toda a ementa**: Internet e Web, HTML (estrutura a formulários) e PHP (variáveis a GET/POST). Use o mapa abaixo como roteiro de estudo:

Mapa da revisão

Bloco	Conceitos essenciais	Módulos
Internet e Web	cliente/servidor, HTTP(S), URL, IP, DNS, domínio, hospedagem, portal, e-commerce	1
Ferramentas	editor de código, estrutura de pastas, index.html	2
HTML estrutura	DOCTYPE, head/body, tags/atributos, h1–h6, p, br, hr, strong/em	3
HTML conteúdo	a href/target, img src/alt, caminhos relativos, ul/ol/dl	4

Bloco	Conceitos essenciais	Módulos
HTML dados	table/tr/th/td, thead/tbody/tfoot, colspan, form, input types, select, textarea	5
PHP básico	blocos, echo, variáveis \$, tipos, var_dump, operadores, == x ===, &&,	6–7
PHP lógica	if/elseif/else, switch/case/break/default	8
PHP repetição	for, while, do...while, arrays indexados/associativos, foreach	9
PHP + HTML	<?= ?>, include/require, function/return, arrays multidimensionais	10
Formulários + PHP	\$_GET/\$_POST, isset/empty, htmlspecialchars, fluxo completo	11

DICA

Método de revisão ativo: para cada linha da tabela, **escreva um exemplo de código sem consultar**. O que não conseguir escrever é o que precisa estudar. Refaça os testes rápidos dos módulos — as questões das provas seguem o mesmo estilo.

As provas do curso

A2 (30 pts): semana 16 (aula 48) — modelo na seção Avaliações. **A3 (40 pts):** prova escrita final individual (30 pts, aulas 70–71 — S) + projeto e apresentação (10 pts, aula 72). Conteúdo cumulativo: tudo o que está nesta tabela.

AULA 61**Aulas 61 a 69 — desenvolvimento guiado do projeto****Três semanas predominantemente práticas**

Semanas 21 a 23: o grupo desenvolve a **versão final** do projeto, aplicando tudo o que foi construído nas etapas anteriores. Roteiro sugerido:

Roteiro das aulas 61 a 69

Semana	Aulas	Metas da semana
21	61–63	Planejamento final (o que falta? quem faz?); estrutura completa revisada; página inicial e menu finalizados

Semana	Aulas	Metas da semana
22	64–66	Formulários completos com validação; processamento PHP robusto (erros + confirmação); listagem dinâmica com todos os campos
23	67–69	Validação de entrada em todos os campos; tratamento de erros com mensagens amigáveis; execução da tabela de testes completa › ENTREGA 5 (versão testada)

Quadro de acompanhamento do grupo

checklist do projeto (cole no caderno/Drive do grupo)

TEXTO

```
[ ] Estrutura de pastas completa (css/, imagens/, includes/)
[ ] cabecalho.php + rodape.php usados em TODAS as páginas
[ ] index.php com menu e apresentação do sistema
[ ] cadastro.php com todos os campos dos requisitos
[ ] processar.php valida todos os campos (isset/empty + regras)
[ ] Mensagens de erro claras + link voltar
[ ] Saídas sanitizadas (htmlspecialchars em tudo)
[ ] listar.php com tabela dinâmica (thead/tbody/tfoot + total)
[ ] funcoes.php com pelo menos 2 funções do grupo
[ ] Tabela de testes executada (mínimo 6 casos) – OK/FALHOU
[ ] Correções das falhas aplicadas e retestadas
[ ] CSS mínimo: páginas consistentes e legíveis
```

DICA

Reunião de 5 minutos no início de cada aula do grupo: o que fizemos? o que falta? quem faz o quê hoje? — é assim que equipes reais trabalham (daily meeting).

AULA 70

Aulas 70 a 72 — finalização e apresentação

Aula 70 — correções e melhorias

Ajustes finais de **código** (nomes, indentação, comentários), **interface** (consistência, mensagens) e **usabilidade** (navegação clara). **(S) Nesta semana aplica-se a prova escrita final individual — 30 pts da A3.**

Aula 71 — preparação da apresentação

Roteiro da apresentação (5 a 8 minutos por grupo)

TEXTO

1. ABERTURA (1 min): nome do sistema, tema, componentes do grupo.
2. OBJETIVO E USUÁRIOS (1 min): o problema que o sistema resolve.
3. DEMONSTRAÇÃO (3-4 min):
 - navegar pelo menu;
 - cadastrar um registro VÁLIDO (mostrar a confirmação);
 - tentar um cadastro INVÁLIDO (mostrar as mensagens de erro);
 - exibir a LISTAGEM dinâmica com os dados;
 - mostrar um trecho do CÓDIGO (foreach da listagem ou validação).
4. DIFICULDADES E APRENDIZADOS (1 min): o que foi difícil e como resolveram.
5. ENCERRAMENTO: agradecimento e espaço para perguntas.

Checklist do ensaio

- Testar o sistema **no computador do laboratório/projetor** (resolução, navegador);
- Ter dados de exemplo **já cadastrados** (a listagem não pode aparecer vazia);
- Cronometrar: dentro dos 5-8 minutos;
- Todos os membros falam (dividir as partes);
- Plano B: prints/vídeo da demonstração, caso algo falhe ao vivo.

Aula 72 — apresentação e avaliação final

Apresentação dos projetos pelos grupos, avaliação final (10 pts — critério do professor, conforme o Plano) e encerramento da disciplina. Critérios sugeridos (S): **funcionalidade** (cadastro, listagem e validação funcionando), **organização dos arquivos**, **qualidade do código** (nomes, indentação, comentários), **uso correto de HTML e PHP** e **apresentação/demonstração**. A rubrica completa está na seção **Avaliação A3** desta apostila.

Apresentar também é habilidade técnica

Na vida profissional, vocês vão demonstrar sistemas para clientes e gestores. Quem apresenta bem um projeto simples se destaca de quem apresenta mal um projeto complexo. Ensaio não é frescura — é parte do trabalho.

EXERCÍCIOS — MÓDULO 12

Atividades práticas e escritas · resolva no laboratório e complete em casa (AVA)

EXERCÍCIO 1 — Documento de requisitos do grupo

EM GRUPO

Preencham o modelo requisitos.txt (aula 50) com o tema escolhido: objetivo, usuários, funcionalidades, páginas e dados (campos com tipo e regras). **Vale como Entrega 1 do projeto final (S).**

RESULTADO ESPERADO

Documento de 1 página com os 6 itens preenchidos de forma concreta e testável (campos com tipos e regras de validação definidos).

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Validar um por um com visto do professor. Requisitos vagos voltam para refinação ('sistema bonito' não é requisito).

EXERCÍCIO 2 — Tabela de testes do sistema**EM GRUPO**

Montem a tabela de testes do sistema do grupo com **no mínimo 6 casos**: para cada funcionalidade, 1 teste válido, 1 inválido e 1 de limite. Colunas: #, teste, entrada, resultado esperado, resultado obtido, OK/FALHOU.

RESULTADO ESPERADO

Tabela completa com esperados definidos ANTES da execução e registro dos resultados reais; falhas viram lista de correções.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Usar na aula 69 (Entrega 5). Teste cruzado entre grupos aumenta o rigor.

EXERCÍCIO 3 — Caça-erros do projeto**ESCRITO**

O trecho abaixo (de um listar.php) tem **3 erros**: um de sintaxe, um de lógica e um de segurança. Identifique-os, classifique-os e corrija:

listar.php com erros

PHP

```
<?php
$livros = [
    ["titulo" => "PHP Básico", "ano" => 2020],
    ["titulo" => "HTML Completo", "ano" => 2022],
]
$total = count($livros) + 1;
foreach ($livros as $livro) {
    echo "<tr><td>" . $livro["titulo"] . "</td></tr>";
}
echo "Total de livros: $total";
?>
```

RESULTADO ESPERADO

SINTAXE: falta ; após o fechamento do array \$livros = [...] (linha 5). LÓGICA: \$total = count(\$livros) + 1 exibe 3 em vez de 2. SEGURANÇA: echo do título sem htmlspecialchars(). Correções: acrescentar ;, remover + 1 e envolver a saída com htmlspecialchars(\$livro["titulo"]).

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Reforça a aula 59. Peça que classifiquem ANTES de corrigir — a classificação é o aprendizado principal.

EXERCÍCIO 4 — Ensaio da apresentação**EM GRUPO**

Sigam o roteiro da aula 71 e ensaiem com cronômetro: abertura, objetivo, demonstração (válido + inválido + listagem + código), dificuldades e encerramento. Gravem um vídeo do ensaio (celular) e autoavaliem: tempo, clareza, todos falaram?

RESULTADO ESPERADO

Apresentação de 5–8 minutos com demonstração funcional e participação de todos os membros.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Professor assiste a um ensaio de cada grupo na aula 71 e dá feedback. Dados de exemplo já cadastrados são obrigatórios.

TESTE RÁPIDO — MÓDULO 12: Projeto final integrador

5 questões objetivas · responda sem consultar o material e depois corrija · gabarito comentado incluído

1. A ordem recomendada de construção do projeto final é:

- (A) listagem > formulário > estrutura > includes.
- (B) estrutura e includes > página inicial > cadastro > processamento > listagem.
- (C) CSS > imagens > formulários > PHP.
- (D) apresentação > código > testes.

RESPOSTA: B

Do alicerce ao acabamento: primeiro o esqueleto e o menu (navegável desde o início), depois as funcionalidades na ordem do fluxo de dados.

2. Um requisito bem escrito se caracteriza por ser:

- (A) curto e genérico, como 'o sistema deve ser bom'.
- (B) concreto e testável, como 'a listagem exibe título, autor e disponibilidade em tabela'.
- (C) escrito em inglês técnico.
- (D) aprovado por todos os grupos da turma.

RESPOSTA: B

Requisito testável permite verificar sim/não na tabela de testes. Generalidades não orientam o desenvolvimento.

3. Na tabela de testes, além dos casos válidos e inválidos, os 'testes de limite' servem para:

- (A) testar o sistema em outros navegadores.
- (B) verificar o comportamento exatamente nas bordas das regras (ex.: ano mínimo e máximo).
- (C) medir a velocidade da página.
- (D) testar apenas campos obrigatórios.

RESPOSTA: B

Erros se escondem nas bordas: valores exatamente no limite (1900/2026) ou logo fora (1899/2027) revelam condições mal escritas ($\geq$ vs $>$).

4. A mensagem “Parse error: syntax error ... in listar.php on line 24” indica um erro de:

- (A) lógica — o resultado do cálculo está errado.
- (B) execução — a variável não existe.
- (C) sintaxe — o código viola as regras da linguagem; deve-se abrir o arquivo na linha indicada.
- (D) segurança — dados não sanitizados.

RESPOSTA: C

Parse error = sintaxe. A mensagem dá arquivo e linha: estratégia ler > identificar > entender > corrigir > testar.

5. Na demonstração da apresentação final (aula 72), o roteiro prevê mostrar:

- (A) apenas a página inicial do sistema.
- (B) somente o código-fonte completo, lido linha a linha.
- (C) um cadastro válido, uma tentativa inválida com as mensagens de erro e a listagem dinâmica.
- (D) um vídeo pronto, sem executar o sistema ao vivo.

RESPOSTA: C

A demonstração prova que o sistema funciona E que trata erros — os dois lados da validação. Plano B (vídeo) existe só para emergências.

AVALIAÇÃO À VISTA

AVALIAÇÃO 3 (A3 — 40 pts): prova escrita final individual (30 pts, aulas 70–71) + projeto e apresentação (10 pts, aula 72). Modelo e rubrica na seção AVALIAÇÕES.

AULAS 49 A 72 · SEMANAS 17 A 24 · 6 ENTREGAS**Projeto Final Integrador**

O que é: Aplicação Web simples e funcional desenvolvida em grupos, integrando HTML (estrutura e conteúdo), PHP (processamento e conteúdo dinâmico), formulários (entrada de dados), validação (controle dos dados), arrays (armazenamento temporário) e organização de arquivos (estrutura do projeto).

Temas sugeridos no material didático: Cadastro de alunos · Agenda de contatos · Catálogo de produtos · Sistema de contatos · Cadastro de clientes · Biblioteca · Controle de tarefas.

Entregas do projeto

Entrega	Descrição	Aulas	Semana
Entrega 1	Proposta do projeto e requisitos (tema, objetivo, usuários, funcionalidades)	Aulas 49–50	Semana 17
Entrega 2	Estrutura de arquivos/pastas e página inicial com menu	Aulas 51–52	Semanas 17–18
Entrega 3	Página de cadastro (formulário completo) e processamento PHP	Aulas 53–54	Semana 18
Entrega 4	Listagem dinâmica, interface e validação do projeto	Aulas 55–57	Semana 19
Entrega 5	Versão testada: testes, tratamento de erros e correções	Aulas 58–70	Semanas 20–24
Entrega 6	Versão final + apresentação do projeto (aula 72)	Aulas 71–72	Semana 24

Critérios sugeridos para os 10 pts (S)

Critério	Pontos	O que se espera
Funcionalidade	2,0	Cadastro, listagem e validação funcionando de ponta a ponta.
Organização dos arquivos	2,0	Estrutura em pastas (css/, imagens/, includes/), includes reutilizados, nomes padronizados.
Qualidade do código	2,0	Nomes significativos, indentação consistente, comentários úteis.
Uso correto de HTML e PHP	2,0	HTML semântico e válido; PHP com validação (isset/empty) e sanitização (htmlspecialchars).
Apresentação e demonstração	2,0	Roteiro seguido, demonstração funcional (válido + inválido + listagem), todos participam, dentro do tempo.

DICA

O desenvolvimento do projeto é guiado aula a aula no Módulo 12 desta apostila: requisitos (aula 50), estrutura (51), páginas (52–55), testes (58), correção de erros (59) e apresentação (70–72).

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Acompanhe as 6 entregas com vistos e feedback curto. A nota de projeto (10 pts) considera o processo (entregas 1–5) e a apresentação final. Sugestão: teste cruzado entre grupos na aula 69 e ensaio geral na 71.

A1 · 30 PTS & A2 · 30 PTS & A3 · 40 PTS**Avaliações — 100 pontos no semestre**

Regras oficiais do Plano de Ensino e proposta de operacionalização do cronograma (S). Os modelos de prova abaixo são **propostas (S)** — o professor pode adaptá-los.

Regras oficiais

Avaliação	Pontos	Regra oficial
1ª Avaliação	30 pts	Distribuídos a critério do professor (trabalhos, seminários, testes etc.), admitindo-se inclusive provas.
2ª Avaliação	30 pts	Obrigatoriamente por meio de prova; poderá ser em duplas ou com consultas.
3ª Avaliação	40 pts	30 pts obrigatoriamente em prova escrita final individual + 10 pts distribuídos a critério do professor.

Proposta de operacionalização no cronograma (S)

Etapa	Período	Proposta
1ª Avaliação	Semanas 1 a 8	Trabalho prático de HTML "site da empresa fictícia" (10 pts — entrega na semana 6) + atividades extraclasse/AVA e exercícios (5 pts — semanas 1 a 8) + teste escrito-prático sobre Web e HTML (15 pts — semana 8).
2ª Avaliação	Semana 16 (aula 48)	Prova (obrigatoriamente; pode ser em duplas ou com consulta) contemplando HTML (estrutura a formulários), PHP (variáveis a arrays/funções) e GET/POST/validação.
3ª Avaliação	Semana 24	Prova escrita final individual e cumulativa (30 pts — aulas 70–71) + projeto integrador: entregas parciais e apresentação final (10 pts — aula 72).

APOIO E RECUPERAÇÃO (PROPOSTA — S)

Identificação precoce de dificuldades pelos exercícios de cada aula; revisões estratégicas ao final dos blocos (aulas 36 e 60); listas de reforço e atividades diferenciadas no AVA; reforço dos pré-requisitos de HTML antes do PHP e de lógica antes de formulários; nova oportunidade de demonstrar aprendizagem por meio das entregas do projeto final.

AVALIAÇÃO 1 (A1) — 30 PONTOS · SEMANAS 1 A 8 (S)

Trabalho HTML (10) + atividades AVA (5) + teste escrito-prático (15)

Componente	Pontos	Quando
Trabalho prático de HTML — “site da empresa fictícia”	10 pts	Entrega na semana 6 (aula 18)
Atividades extraclasse/AVA e exercícios de fixação	5 pts	Semanas 1 a 8
Teste escrito-prático sobre Web e HTML	15 pts	Semana 8

Trabalho prático (10 pts): site da empresa fictícia

Desenvolver o site de uma empresa fictícia com 4 páginas (index.html, empresa.html, produtos.html e contato.html), aplicando todos os conteúdos de HTML do curso até o módulo 5. Pode ser feito em duplas ou individualmente (critério do professor).

Requisitos do trabalho:

- 4 páginas com estrutura HTML5 completa (DOCTYPE, html lang, meta charset, title) e menu de navegação idêntico e funcional em todas;
- index.html: h1, parágrafos com formatação semântica (strong, em), imagem com alt, lista (ul ou ol) e separador hr;
- empresa.html: hierarquia h1/h2, textos, quebras (br) e separadores (hr), imagem;
- produtos.html: tabela com no mínimo 4 produtos (nome, categoria, preço, estoque) usando thead, tbody, tfoot e colspan no rodapé;
- contato.html: formulário com nome (text), e-mail (email), assunto (select), mensagem (textarea), checkbox de novidades e botão de envio, todos os campos com label;
- Organização: pasta imagens/, nomes de arquivos em minúsculas sem acento/espço, código indentado.

Rubrica de correção do trabalho (10 pts)

Critério	Pontos	O que se espera
Estrutura e navegação	2,0	HTML5 completo nas 4 páginas; menu idêntico e sem links quebrados.
Conteúdo e semântica	2,0	Hierarquia de títulos correta; strong/em (não b/i); imagens com alt descritivo; listas adequadas.
Tabela de produtos	2,0	thead/tbody/tfoot, colspan correto, 4+ produtos, células alinhadas.
Formulário de contato	2,0	Todos os campos pedidos, labels associados (for/id), types corretos, select/textarea/checkbox presentes.
Organização e qualidade	2,0	Estrutura de pastas, nomes de arquivos padronizados, indentação e legibilidade do código.

ORIENTAÇÕES DE CORREÇÃO — VERSÃO DO PROFESSOR

Aplique a rubrica item a item com visto no checklist; devolva com feedback escrito. Trabalhos com links quebrados perdem o critério 'Estrutura e navegação' integralmente. Combine a entrega via AVA/Drive + pasta do laboratório.

Teste escrito-prático (15 pts) — modelo (S)

AVALIAÇÃO 1 — TESTE ESCRITO-PRÁTICO (15 pts) · WEB E HTML

ESCOLA: Instituto Técnico — Curso Técnico em Informática

ALUNO(A): _____

TURMA: 2/2026 DATA: ____ / ____ / ____ NOTA: _____

Instruções:

- Prova individual, sem consulta. Duração: 1 aula (50 min).
- Parte A: 6 questões objetivas (1 pt cada). Parte B: 3 questões práticas (3 pts cada).
- Nas questões de código, escreva com clareza; indentação e legibilidade também são avaliadas.

Parte A — questões objetivas (1 pt cada)**Questão 1. A Web (World Wide Web) é:**

- (A) sinônimo de Internet.
- (B) a infraestrutura física de cabos e servidores.
- (C) um dos serviços que funcionam sobre a Internet, formado por páginas interligadas.
- (D) um protocolo de criptografia de dados.

Resposta: C

A Web é um serviço sobre a Internet; e-mail, streaming etc. são outros serviços.

Questão 2. Na URL `https://loja.com.br/produtos/tv.php?cor=preta`, o trecho `"?cor=preta"` é:

- (A) o protocolo.
- (B) o domínio.
- (C) o caminho.
- (D) um parâmetro enviado à página.

Resposta: D

Parâmetros vêm após ? no formato nome=valor.

Questão 3. A função do DNS é:

- (A) criptografar a conexão HTTPS.
- (B) traduzir nomes de domínio em endereços IP.
- (C) hospedar os arquivos do site.
- (D) exibir as páginas no navegador.

Resposta: B

DNS converte nome (loja.com.br) em IP do servidor.

Questão 4. Qual grupo tem apenas elementos HTML semânticos de formatação?

- (A) b, i, font
- (B) strong, em, mark
- (C) div, span, p
- (D) table, tr, td

Resposta: B

strong/em/mark carregam significado; b/i são apenas aparência.

Questão 5. Para uma célula de tabela ocupar 2 colunas, usa-se:

- (A) rowspan="2"
- (B) colspan="2"
- (C) span="2"
- (D) merge="2"

Resposta: B

colspan mescla colunas; rowspan mesclaria linhas.

Questão 6. Em um formulário, o atributo que identifica o campo no processamento pelo servidor é:

- (A) id
- (B) class
- (C) name
- (D) for

Resposta: C

name é a chave usada pelo PHP (\$_POST/\$_GET); id é para label/CSS.

Parte B — questões práticas (3 pts cada)

Questão 7. Escreva o código HTML5 completo de uma página chamada “Cursos da Escola” que contenha: título principal (h1) “Cursos Técnicos”, um parágrafo com a palavra “gratuitos” em strong, uma lista não ordenada com os 3 cursos (Informática, Redes, Administração) e uma imagem cursos.jpg (pasta imagens/) com texto alternativo adequado.

GABARITO — código modelo

```
<!DOCTYPE html>
<html lang="pt-br">
<head>
<meta charset="UTF-8">
<title>Cursos da Escola</title>
</head>
<body>
<h1>Cursos Técnicos</h1>
<p>Nossos cursos são <strong>gratuitos</strong> </p>
<ul>
<li>Informática</li>
<li>Redes</li>
<li>Administração</li>
</ul>

</body>
</html>
```

Critérios: Estrutura completa (0,5) · h1 e parágrafo com strong (0,75) · ul/li com os 3 itens (0,75) · img com caminho relativo e alt descritivo (1,0).

Questão 8. O código abaixo contém 4 erros. Reescreva-o corrigido e sublinhe/numere as correções: (código: <html> sem DOCTYPE e sem lang; <p>Texto com importante</p>; <h1>Título</h3>; Sobre)

GABARITO — código modelo

```
<!DOCTYPE html>
<html lang="pt-br">
<head><meta charset="UTF-8"><title>Teste</title></head>
<body>
<h1>Título</h1>
<p>Texto com <strong>importante</strong> </p>
<a href="sobre.html">Sobre</a>
</body>
</html>
```

Crterios: Cada correção vale 0,75: 1) DOCTYPE (+lang/charset); 2) strong/p descruzados; 3) h1 fechado com /h1; 4) href em vez de src no link.

Questão 9. Construa (a) uma tabela 3×3 com cabeçalho (th) “Produto, Preço, Estoque”, uma linha de dados (Mouse, 25.00, 10) e umtfoot com “Total de itens” ocupando 2 colunas (colspan) e o valor 10; e (b) um formulário com campo de e-mail (obrigatório), um select “Turno” (Manhã/Tarde/Noite) e botão de envio.

GABARITO — código modelo

```
<table border="1">
<thead><tr><th>Produto</th><th>Preço</th><th>Estoque</th></tr></thead>
<tbody><tr><td>Mouse</td><td>25.00</td><td>10</td></tr></tbody>
<tfoot><tr><td colspan="2">Total de itens</td><td>10</td></tr></tfoot>
</table>
<form action="processar.php" method="post">
<label for="email">E-mail:</label>
<input type="email" id="email" name="email" required>
<label for="turno">Turno:</label>
<select id="turno" name="turno">
<option value="manha">Manha</option>
<option value="tarde">Tarde</option>
<option value="noite">Noite</option>
</select>
<button type="submit">Enviar</button>
</form>
```

Crterios: Tabela: thead/th (0,5) · tbody correto (0,5) · tfoot com colspan=2 (0,5). Formulário: type=email + required (0,5) · label/for (0,25) · select com 3 options (0,5) · button submit + action/method (0,25).

AVALIAÇÃO 2 (A2) — 30 PONTOS · SEMANA 16, AULA 48 (S)

Prova obrigatória (pode ser em duplas ou com consulta) — HTML + PHP + GET/POST

AVALIAÇÃO 2 — PROVA (30 pts) · HTML, PHP E FORMULÁRIOS

ESCOLA: Instituto Técnico — Curso Técnico em Informática

ALUNO(A): _____

TURMA: 2/2026 DATA: ____ / ____ / ____ NOTA: _____

Instruções:

- Conteúdo: HTML (estrutura a formulários), PHP (variáveis a arrays/funções) e GET/POST/validação.

- Modalidade definida pelo professor conforme o Plano de Ensino (duplas e/ou consulta permitidos).
- Questões 1–4: conceituais. Questões 5–10: práticas (prever saída, corrigir e escrever código).
- Duração sugerida: 2 aulas (100 min).

Questão 1. Sobre HTML e PHP, é correto afirmar:

- (A) HTML é processado no servidor e PHP no navegador.
- (B) HTML estrutura o conteúdo no navegador; PHP executa lógica no servidor e gera HTML.
- (C) Ambos são linguagens de programação com variáveis e decisões.
- (D) PHP substitui o HTML: páginas PHP não contêm HTML.

Resposta: B

HTML = marcação interpretada no navegador; PHP = programação executada no servidor, que gera HTML como resposta.

Questão 2. Para enviar um formulário de cadastro com senha, o método e a leitura corretos são:

- (A) method=get, lido com \$_GET — dados aparecem na URL.
- (B) method=post, lido com \$_POST — dados viajam no corpo da requisição.
- (C) method=submit, lido com \$_FORM.
- (D) method=get, lido com \$_POST.

Resposta: B

Dados sensíveis (senha) > POST; o PHP lê no array \$_POST pela chave name do campo.

Questão 3. O padrão seguro para receber dados de um formulário em PHP é:

- (A) usar \$_POST diretamente no echo, sem verificações.
- (B) confiar no required do HTML e dispensar validação no servidor.
- (C) verificar com isset/empty, validar as regras e exibir com htmlspecialchars.
- (D) criptografar a URL com GET.

Resposta: C

Validação no servidor (isset/empty + regras) + sanitização na saída (htmlspecialchars) — required do HTML pode ser contornado.

Questão 4. Sobre include e funções, é correto afirmar:

- (A) include serve para reutilizar partes de página (ex.: cabeçalho); funções reutilizam lógica com parâmetros e retorno.
- (B) include e function são sinônimos em PHP.
- (C) funções não podem receber parâmetros.
- (D) require emite apenas aviso se o arquivo não existir.

Resposta: A

include/require reutilizam trechos de código/página; function define lógica nomeada com parâmetros e return. require gera erro FATAL se faltar o arquivo.

Questão 5. Qual é a saída exibida pelo código abaixo?

Código para análise

PHP

```
<?php
$notas = [7.0, 5.5, 9.0];
$soma = 0;
foreach ($notas as $n) {
    if ($n >= 7) {
        $soma += $n;
    }
}
echo "Soma: $soma";
?>
```

GABARITO

Soma: 16 — apenas 7.0 e 9.0 passam na condição (5.5 é ignorado).

Critérios: Resultado 16 (2 pts) + justificativa do filtro (1 pt).

Questão 6. O que aparece na tela? Explique a diferença entre as duas comparações.

Código para análise

PHP

```
<?php
$a = 10;
$b = "10";
if ($a == $b) { echo "iguais "; }
if ($a === $b) { echo "identicos"; } else { echo "nao identicos"; }
?>
```

GABARITO

Saída: "iguais nao identicos" — == compara valor (10 = "10" > true); === compara valor E tipo (integer × string > false).

Critérios: Saída exata (1,5 pt) + explicação == × === (1,5 pt).

Questão 7. O programa abaixo deveria exibir a tabuada do 4 (4 x 1 = 4 ... 4 x 10 = 40), mas tem 2 erros. Reescreva-o corrigido.

Código para análise

PHP

```
<?php
$n = 4
for ($i = 1; $i <= 10; $i++) {
    echo "$n x $i = " . $n * $i . "<br>"
}
?>
```

GABARITO

Faltam os ponto e vírgula: após \$n = 4 e após o echo dentro do for.

Critérios: 1,5 pt por ponto e vírgula corrigido. Parênteses na multiplicação são opcionais (precedência correta), mas recomendados.

Questão 8. Escreva uma função PHP chamada `situacao($media)` que retorne "Aprovado" se a média for ≥ 7 , "Recuperação" se ≥ 5 e "Reprovado" caso contrário. Em seguida, chame a função com `$media = 6.5` e exiba o resultado.

GABARITO — código modelo

```
<?php
function situacao($media) {
    if ($media >= 7) {
        return "Aprovado";
    } elseif ($media >= 5) {
        return "Recuperação";
    } else {
        return "Reprovado";
    }
}

$media = 6.5;
echo situacao($media); // Recuperação
?>
```

Critérios: function com parâmetro (0,5) · 3 ramificações corretas na ordem certa (1,5) · return (não echo) dentro da função (0,5) · chamada e exibição (0,5).

Questão 9. Dado o array associativo multidimensional abaixo, escreva o código que percorre os alunos com foreach e exibe, para cada um, uma linha "Nome — Curso", usando `htmlspecialchars` na saída.

Código para análise

PHP

```
$alunos = [
    ["nome" => "Ana", "curso" => "Informática"],
    ["nome" => "Bruno", "curso" => "Redes"],
];
```

GABARITO — código modelo

```
<?php
foreach ($alunos as $aluno) {
    echo htmlspecialchars($aluno["nome"]) . " — "
    . htmlspecialchars($aluno["curso"]) . "<br>";
}
?>
```

Critérios: foreach correto (1,0) · acesso por chave (0,5) · htmlspecialchars nos dois campos (1,0) · formatação da linha (0,5).

Questão 10. Escreva o PHP de `processar.php` que: (a) recebe por POST os campos nome e idade; (b) valida — nome obrigatório (empty) e idade entre 10 e 120; (c) em caso de erro, exibe as mensagens e um link para voltar; (d) se válido, exibe a confirmação sanitizada "Cadastro de NOME (IDADE) anos realizado!".

GABARITO — código modelo

```

<?php
$erros = [];
if (empty $_POST["nome"]) {
$erros[] = "Nome é obrigatório.";
}
$idade = isset $_POST["idade"] ? (int) $_POST["idade"] : 0;
if ($idade < 10 || $idade > 120) {
$erros[] = "Idade deve estar entre 10 e 120.";
}
if (count $erros > 0) {
foreach $erros as $e {
echo "<p>$e</p>";
}
}
echo '<a href="cadastro.html">Voltar</a>';
exit;
}
$nome = htmlspecialchars($_POST["nome"]);
echo "Cadastro de $nome ($idade anos) realizado!";
?>

```

Crterios: Leitura de \$_POST (0,5) · validação nome (0,5) · validação faixa de idade (0,5) · mensagens + link voltar + exit (0,75) · sanitização e confirmação (0,75). Soluções equivalentes (if/else sem array \$erros) recebem nota integral se cumprirem os requisitos.

AValiação 3 (A3) — 40 PONTOS · SEMANA 24

Prova escrita final individual (30 pts, aulas 70–71 — S) + projeto e apresentação (10 pts, aula 72)

AValiação 3 — PROVA ESCRITA FINAL INDIVIDUAL (30 pts) · CUMULATIVA

ESCOLA: Instituto Técnico — Curso Técnico em Informática

ALUNO(A): _____

TURMA: 2/2026 DATA: ____ / ____ / ____ NOTA: _____

Instruções:

- Prova individual, sem consulta, cobrindo TODA a ementa (Web, HTML, PHP, formulários, projeto).
- Duração: 1 a 2 aulas (a definir pelo professor).
- Questões 1–5: objetivas/conceituais. Questões 6–10: práticas (prever, corrigir, escrever).

Questão 1. O ciclo de funcionamento da Web é:

- (A) resposta > requisição > processamento.
- (B) requisição do navegador > processamento no servidor > resposta em HTML.
- (C) processamento no navegador > requisição ao DNS > resposta do cliente.
- (D) requisição do servidor > resposta do navegador > exibição.

Resposta: B

Cliente requisita (HTTP), servidor processa (ex.: PHP) e responde com o HTML que o navegador exige.

Questão 2. Para exibir dados enviados pelo usuário com segurança contra XSS, usa-se:

- (A) `str_repeat()`
- (B) `htmlspecialchars()`
- (C) `count()`
- (D) `var_dump()`

Resposta: B

`htmlspecialchars` converte `<`, `>`, `&` e aspas em entidades: o dado vira texto inofensivo na saída.

Questão 3. Sobre laços de repetição em PHP:

- (A) `while` executa o bloco pelo menos uma vez sempre.
- (B) `for` só funciona com arrays.
- (C) `do...while` testa a condição após executar o bloco (mínimo 1 execução).
- (D) `foreach` requer contador manual (`$i++`).

Resposta: C

`do...while` = executa e depois testa. `while` pode executar 0 vezes; `for` usa contador para N voltas conhecidas; `foreach` percorre arrays sem contador manual.

Questão 4. A estrutura de projeto recomendada no curso organiza os arquivos assim:

- (A) todos os arquivos soltos na raiz, com nomes longos e descritivos.
- (B) páginas na raiz (`index`, `cadastro`, `processar`, `listar`) + pastas `css/`, `imagens/` e `includes/`.
- (C) um único arquivo com todo o código do sistema.
- (D) cada página em uma pasta própria, com cópias do cabeçalho.

Resposta: B

Padrão do módulo 10: páginas principais na raiz + pastas por tipo de arquivo; `includes/` centraliza cabeçalho/rodapé reutilizáveis.

Questão 5. Na tabela de testes de um sistema, os 'casos de limite' verificam:

- (A) a velocidade de carregamento das páginas.
- (B) o comportamento exatamente nas bordas das regras de validação (ex.: valor mínimo e máximo).
- (C) se o layout funciona em celulares.
- (D) se o DNS resolve o domínio corretamente.

Resposta: B

Limites/bordas (mínimo, máximo, um além) revelam erros de condição (`>=` vs `>`) — conteúdo das aulas 58 e 59.

Questão 6. Qual é a saída do código?

Código para análise

PHP

```
<?php
$precos = [10, 25, 5];
$maior = 0;
foreach ($precos as $p) {
    if ($p > $maior) {
        $maior = $p;
    }
}
echo "Maior preço: $maior";
echo "<br>Total: " . count($precos);
?>
```

GABARITO

Maior preço: 25 · Total: 3

Critérios: Cada linha correta: 1,5 pt.

Questão 7. O que o switch abaixo exhibe? Explique.

Código para análise

PHP

```
<?php
$opcao = 2;
switch ($opcao) {
    case 1:
        echo "Um ";
    case 2:
        echo "Dois ";
    case 3:
        echo "Tres ";
        break;
    default:
        echo "Outro";
}
?>
```

GABARITO

"Dois Tres" — faltam breaks nos cases 1 e 2: ao bater no case 2, a cascata continua executando o case 3 até encontrar o break.

Critérios: Saída exata (1,5 pt) + explicação da cascata por falta de break (1,5 pt).

Questão 8. O código deveria gerar uma tabela HTML com os nomes do array, mas tem 3 erros (sintaxe, lógica e segurança). Corrija-os.

Código para análise

PHP

```
<?php
$nomes = ["Ana", "Bruno"]
echo "<table>";
foreach ($nomes as $nome) {
echo "<tr><td>" . $nome . "</td></tr>";
}
echo "<tr><td>Total: " . count($nomes) + 1 . "</td></tr>";
?>
```

GABARITO — código modelo

```
<?php
$nomes = ["Ana", "Bruno"]; // 1) ; após o array
echo "<table>";
foreach ($nomes as $nome) {
echo "<tr><td>" . htmlspecialchars($nome) . "</td></tr>"; // 3) sanitizar
}
echo "<tr><td>Total: " . (count $nomes) . "</td></tr>"; // 2) sem + 1
echo "</table>"; // (bônus) fechar a tabela
?>
```

Critérios: 1 pt por erro corrigido: sintaxe (;), lógica (+1 indevido no total), segurança (htmlspecialchars). Fechar </table> não pontua, mas deve ser elogiado.

Questão 9. Escreva um arquivo pagina.php que: inclua includes/cabecalho.php; declare a função media(\$a, \$b) que retorna a média; chame a função com 7.5 e 9.0 guardando em \$resultado; e exiba “Média: X” dentro de um parágrafo HTML. Finalize incluindo includes/rodape.php.

GABARITO — código modelo

```
<?php include "includes/cabecalho.php"; ?>
<?php
function media $a, $b) {
return ($a + $b) / 2;
}
$resultado = media(7.5, 9.0);
?>
<p>Média: <?= $resultado ?></p>
<?php include "includes/rodape.php"; ?>
```

Critérios: include do cabeçalho (0,5) · função com return correto (1,0) · chamada guardando em \$resultado (0,5) · exibição em <p> com echo/<?= (0,5) · include do rodapé (0,5).

Questão 10. Escreva o HTML de um formulário de busca de produtos que envie por GET para buscar.php, com: campo texto name=produto (obrigatório), select name=categoria (Todas/Eletrônicos/Livros) e botão “Buscar”. Em seguida, escreva o PHP de buscar.php que lê e exibe (sanitizado) “Buscando PRODUTO na categoria CATEGORIA”, tratando campo vazio com mensagem de erro.

GABARITO — código modelo

```

<!-- buscar.html -->
<form action="buscar.php" method="get">
<label for="produto">Produto:</label>
<input type="text" id="produto" name="produto" required>
<label for="categoria">Categoria </label>
<select id="categoria" name="categoria">
<option value="todas">Todas</option>
<option value="eletronicos">Eletrônicos</option>
<option value="livros">Livros</option>
</select>
<button type="submit">Buscar</button>
</form>
<!-- buscar.php -->
<?php
if (empty($_GET["produto"])) {
echo "Erro: informe o produto.";
exit;
}
$produto = htmlspecialchars($_GET["produto"]);
$categoria = htmlspecialchars($_GET["categoria"] ?? "todas");
echo "Buscando $produto na categoria $categoria";
?>

```

Crêterios: HTML: form get/action (0,5) · input name+required (0,5) · select com options (0,5) · button (0,25). PHP: leitura \$_GET (0,5) · validação empty + exit (0,5) · htmlspecialchars (0,5) · mensagem final (0,25).

Rubrica do projeto e apresentação (10 pts — critério do professor)

Crêterio	Pontos	Descrição
Funcionalidade	2,0	Cadastro, listagem e validação funcionando de ponta a ponta.
Organização dos arquivos	2,0	Estrutura em pastas (css/, imagens/, includes/), includes reutilizados, nomes padronizados.
Qualidade do código	2,0	Nomes significativos, indentação consistente, comentários úteis.
Uso correto de HTML e PHP	2,0	HTML semântico e válido; PHP com validação (isset/empty) e sanitização (htmlspecialchars).
Apresentação e demonstração	2,0	Roteiro seguido, demonstração funcional (válido + inválido + listagem), todos participam, dentro do tempo.

- As entregas parciais (1 a 5) alimentam a nota de funcionalidade e organização — acompanhe com vistos.
- Na apresentação (aula 72), o grupo deve demonstrar: cadastro válido, tentativa inválida (mensagens de erro) e listagem dinâmica.
- Sugestão: plateia registra uma pergunta por grupo; respostas também compõem a avaliação da apresentação.

TERMOS ESSENCIAIS DA DISCIPLINA

Glossário

Array	Estrutura que guarda vários valores em uma única variável, acessados por índice (numérico) ou por chave (associativo).
Atributo	Configuração extra de uma tag HTML, escrita na tag de abertura: nome="valor" (ex.: href, src, alt).
Boolean	Tipo de dados com apenas dois valores: true (verdadeiro) ou false (falso).
Checkbox	Campo de formulário de múltipla escolha (marca/desmarca opções independentes).
Cliente	Papel de quem solicita recursos na Web — tipicamente o navegador do usuário.
colspan	Atributo de célula de tabela que a faz ocupar N colunas.
Concatenação	Junção de textos. Em PHP usa-se o ponto: "Olá, " . \$nome.
Constante	Valor que não muda durante o programa; em PHP: define("NOME", valor).
CSS	Folhas de estilo — definem a aparência (cores, fontes, espaçamento) das páginas HTML.
DNS	Sistema que traduz nomes de domínio em endereços IP (a "agenda" da Internet).
Domínio	Nome registrado que identifica um site na Web (ex.: meusite.com.br).
DOCTYPE	Declaração <!DOCTYPE html> que indica ao navegador o uso de HTML5.
echo	Comando PHP que envia conteúdo (texto/HTML) para a resposta da página.
e-Commerce	Comércio eletrônico — compra e venda on-line (loja virtual).
e-Business	Negócio conduzido de forma digital (conceito amplo, inclui o e-commerce).
Elemento HTML	Conjunto: tag de abertura + conteúdo + tag de fechamento (ex.: texto).
float	Tipo numérico com casas decimais em PHP (ex.: 1.65).
foreach	Laço PHP que percorre todos os elementos de um array (as \$item ou as \$chave => \$valor).
Formulário	Conjunto de campos (form + inputs) para o usuário enviar dados ao servidor.
FTP	Protocolo de transferência de arquivos, usado para publicar sites na hospedagem.
Função	Bloco de código nomeado, com parâmetros de entrada e valor de retorno (return).
GET	Método de envio de formulário cujos dados viajam visíveis na URL; lido via \$_GET.
Hospedagem	Serviço que armazena os arquivos do site em um servidor ligado 24 h.
htmlspecialchars()	Função PHP que converte caracteres especiais em entidades — sanitização contra XSS.
HTTP / HTTPS	Protocolo de comunicação da Web / sua versão criptografada (segura).
include / require	Comandos PHP que inserem o conteúdo de outro arquivo (reutilização: cabeçalho, rodapé, funções).

integer	Tipo numérico inteiro em PHP (ex.: 17).
IP (endereço)	Número que identifica um dispositivo em uma rede (ex.: 142.250.78.14).
isset() / empty()	Funções PHP de validação: o dado foi enviado? / o dado está vazio?
Laço (loop)	Estrutura de repetição: for, while, do...while, foreach.
Label	Rótulo de campo de formulário; associado ao campo pelo par for/id.
localhost	Endereço do próprio computador — usado para testar o servidor local (http://localhost).
Markup (marcação)	Sistema de anotação de texto que define a estrutura do conteúdo (HTML).
Método (HTTP)	Modo de envio dos dados do formulário: GET ou POST.
Parâmetro	Valor de entrada de uma função; na chamada, é o argumento.
Portal	Site que concentra conteúdo e serviços (institucional, educacional, de notícias).
POST	Método de envio cujos dados viajam no corpo da requisição (invisíveis na URL); lido via \$_POST.
Radio (botão)	Campo de escolha única dentro de um grupo (mesmo name).
Requisição / Resposta	Pedido enviado pelo navegador ao servidor / conteúdo devolvido pelo servidor.
return	Palavra-chave que devolve o resultado de uma função e a encerra.
Sanitização	Tratamento dos dados do usuário para exibição/armazenamento seguros (ex.: htmlspecialchars).
Semântica (HTML)	Uso de tags que transmitem significado (strong, em, nav), não apenas aparência.
Servidor Web	Software (ex.: Apache) que atende requisições e entrega páginas; roda na hospedagem.
string	Tipo de dados de texto em PHP (entre aspas).
switch	Estrutura de decisão que compara uma variável com valores fixos (case/break/default).
Tag	Marcador HTML entre sinais de menor/maior: de abertura (), fechamento () ou vazia ().
URL	Endereço completo de um recurso na Web (protocolo + domínio + caminho + recurso + parâmetros).
var_dump()	Função PHP que exibe tipo e valor de uma variável — essencial para depuração.
Variável	Espaço nomeado na memória que guarda um valor; em PHP começa com \$.
XAMPP	Pacote gratuito com servidor Apache + PHP + MySQL para desenvolvimento local.
XSS	Ataque de injeção de script em páginas Web — prevenido com sanitização da saída.

OFICIAL DO PLANO DE ENSINO**Bibliografia****Básica**

- ALVES, William Pereira. Desenvolvimento e design de sites. São Paulo: Erica, 2014.
- ALVES, William Pereira. Linguagem e lógica de programação. São Paulo: Erica, 2014.
- LAUREANO, Marcos Aurelio Pchek; CORDELLI, Rosa Lantmann. Fundamentos de software. São Paulo: Erica, 2019.

Complementar

- ALVES, William Pereira. Projetos de sistemas Web. São Paulo: Erica, 2019.
- ALMEIDA, Rodrigo Maximiano A. de. Programação de sistemas embarcados. Rio de Janeiro: GEN LTC, 2016.
- MACIEL, Francisco Marcelo de Barros. Python e Django. Rio de Janeiro: Alta Books, 2020.
- SOUZA, Diogo B. da Costa (Org.). Sistemas digitais. Porto Alegre: SER - SAGAH, 2018.
- STAIR, Ralph M.; REYNOLDS, George W. Princípios de sistemas de informação. São Paulo: Cengage Learning, 2016.

COMO LER ESTE DOCUMENTO

Itens marcados com (S) neste material são propostas pedagógicas de organização (datas de avaliações, entregas e distribuição das horas não presenciais) — não especificadas no Plano de Ensino original. As demais informações reproduzem o Plano de Ensino oficial da disciplina.

Bons estudos e bom código!

Programação para Internet I · Curso Técnico em Informática · Turma 2/2026 — material elaborado a partir do Plano de Ensino e do Cronograma oficiais. Itens marcados com (S) são propostas pedagógicas de organização.